Neural networks and large language models

Computational modeling of language
Spring 2026

Bureaucracy

Bureaucracy

- **Thursday** will be **different** this week!

Bureaucracy

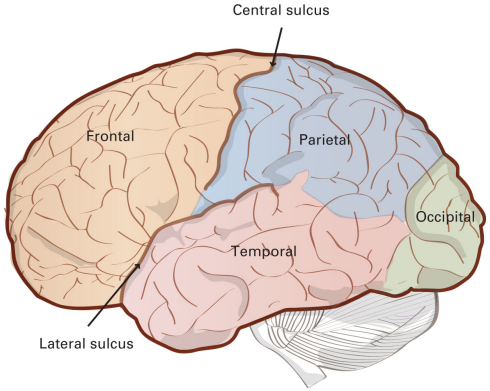
- **Thursday** will be **different** this week!
- In-class **discussion** of neural nets, LLMs, etc.

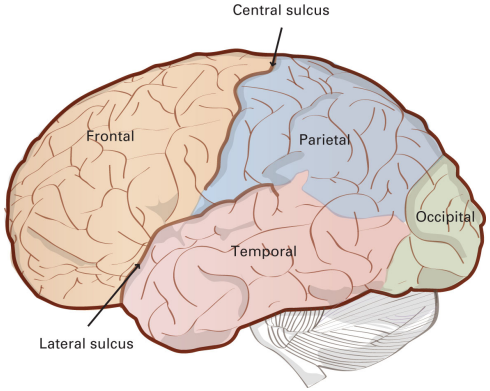
Bureaucracy

- **Thursday** will be **different** this week!
- In-class **discussion** of neural nets, LLMs, etc.
- Maxent OT homework is due **next** week, not this week.

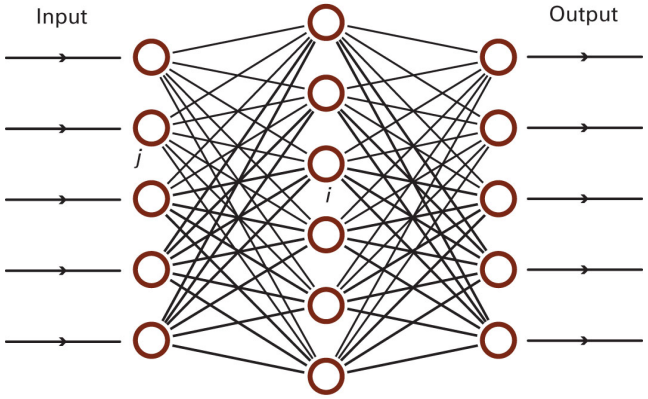
Bureaucracy

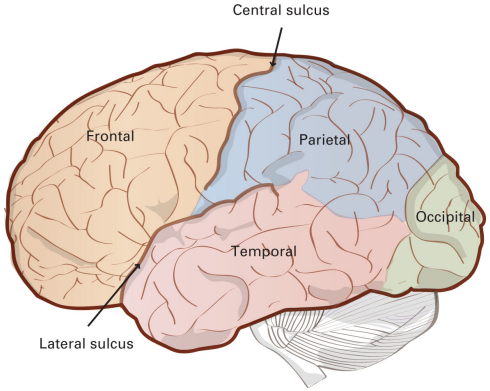
- **Thursday** will be **different** this week!
- In-class **discussion** of neural nets, LLMs, etc.
- Maxent OT homework is due **next** week, not this week.
- **No new homework** this week.


Central sulcus
Frontal
Parietal
Occipital
Temporal
Lateral sulcus

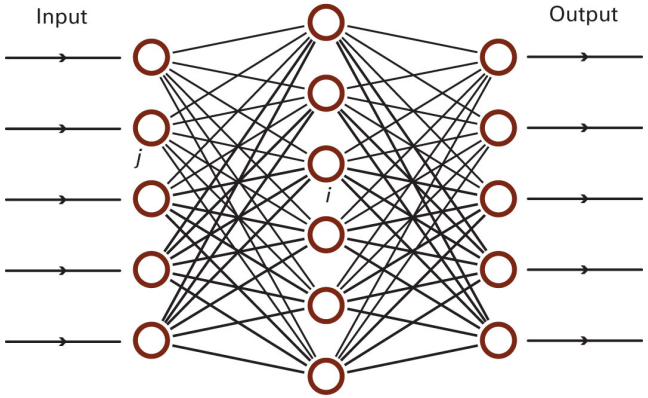


Neural inspiration





Neural inspiration

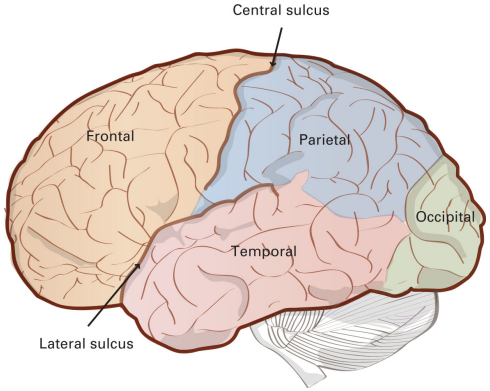


Neural computation

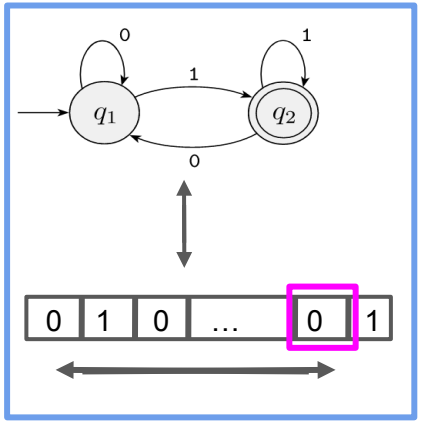
We often speak of **computation** in terms of **symbolic rules** – e.g. finite state machines, regular expressions, Turing machines, the Chomsky hierarchy, OT.

We often speak of **computation** in terms of **symbolic rules** – e.g. finite state machines, regular expressions, Turing machines, the Chomsky hierarchy, OT.

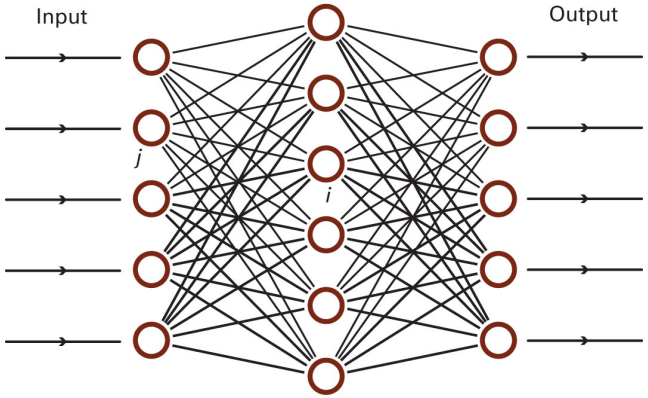
Neural networks offer a **different view** of computation: **non-symbolic** representations, **massively parallel** processing, **multiple soft constraints**.



Turing machine

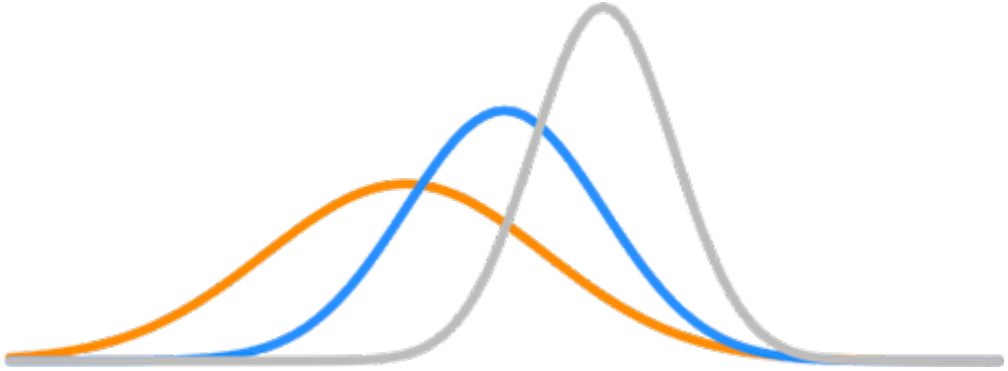


Neural inspiration



Symbolic to non-symbolic

Neural computation



Compare with **explicitly probabilistic models** we have seen: you will see both similarities and differences.

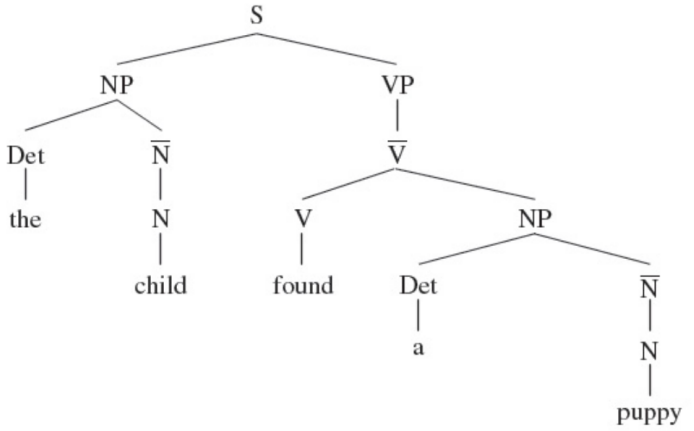
Possible relations of structure and numbers

Arranged in a cline of increasing importance for numbers

Possible relations of structure and numbers

Arranged in a cline of increasing importance for numbers

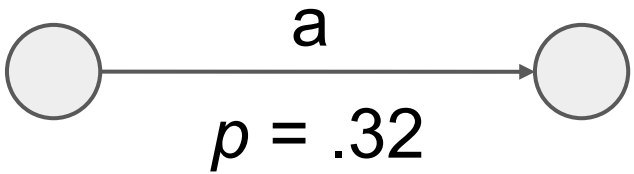
- All symbols, **no** numbers



Possible relations of structure and numbers

Arranged in a cline of increasing importance for numbers

- All symbols, **no** numbers
- Symbols **augmented** by numbers

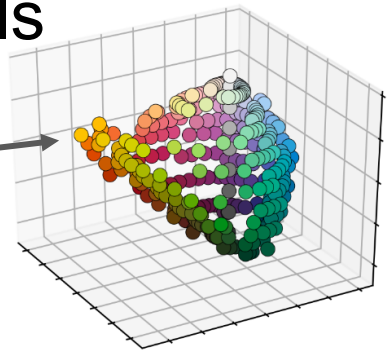


Possible relations of structure and numbers

Arranged in a cline of increasing importance for numbers

- All symbols, **no** numbers
- Symbols **augmented** by numbers
- Numbers as **content providers** for symbols

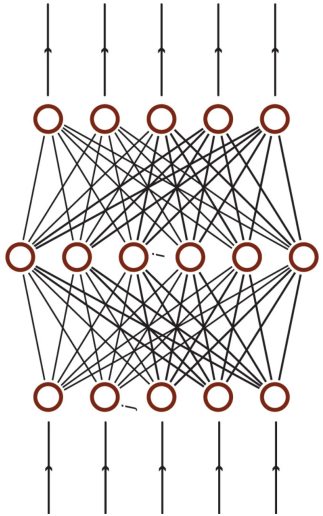
“yellow”



Possible relations of structure and numbers

Arranged in a cline of increasing importance for numbers

- All symbols, **no** numbers
- Symbols **augmented** by numbers
- Numbers as **content providers** for symbols
- **All** numbers, no symbols



- 1943: Linking neuronal function to logic.
- 1958: Perceptrons, learning rule.
- 1969: Limitations of single-layer perceptrons.
- 1980s: Learning in multi-layer perceptrons - back-propagation, connectionism.
- 2000s: Deep learning.
- Late 2010s: GANs, transformers, LLMs.

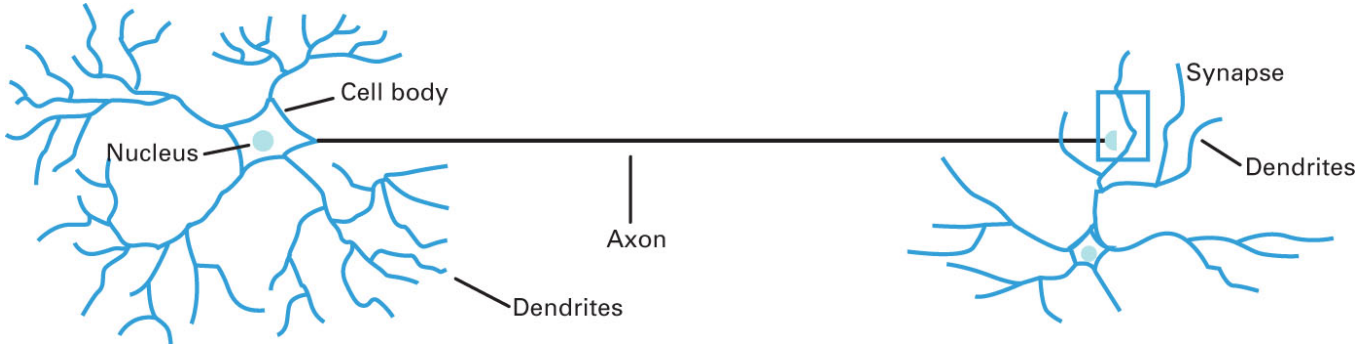
Overview

1. Neurons and logic
2. Perceptrons and learning
3. Generative AI
4. Transformers and large language models

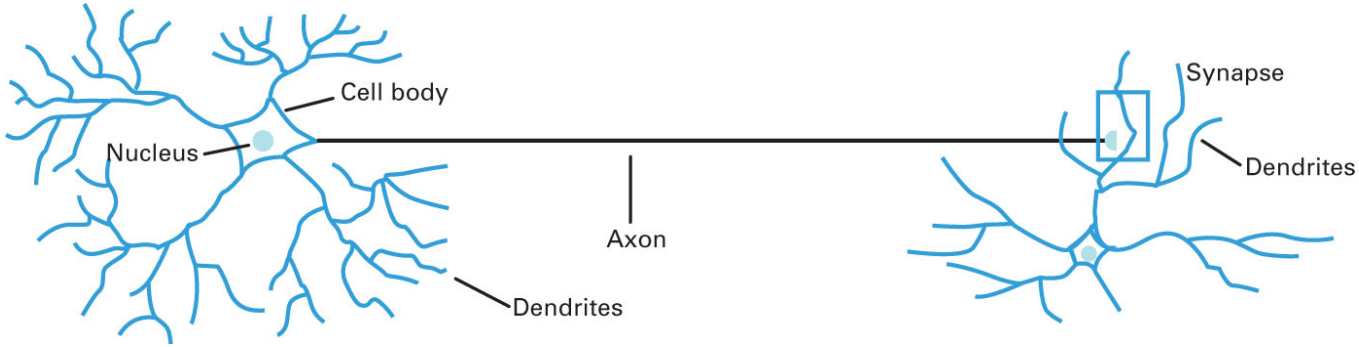
Overview

1. Neurons and logic
2. Perceptrons and learning
3. Generative AI
4. Transformers and large language models

Neuron



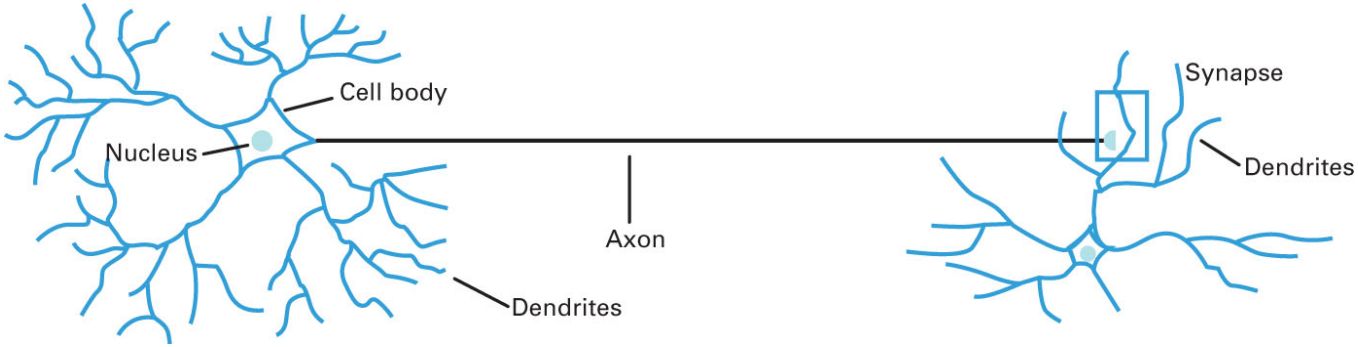
Neuron

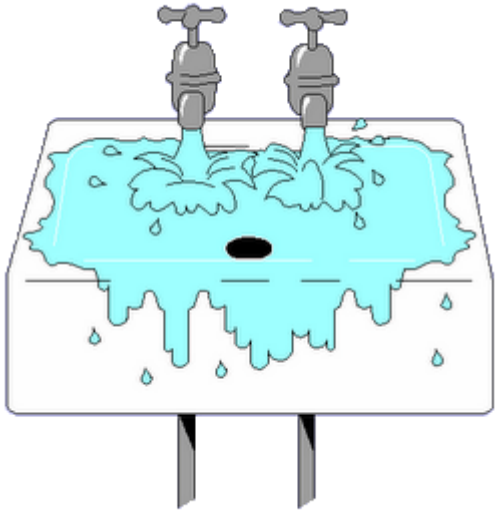


The nervous system is a net of neurons, each having a soma [cell body] and an axon. Their adjunctions, or synapses, are always between the axon of one neuron and the soma of another. **At any instant a neuron has some threshold, which excitation must exceed to initiate an impulse.**

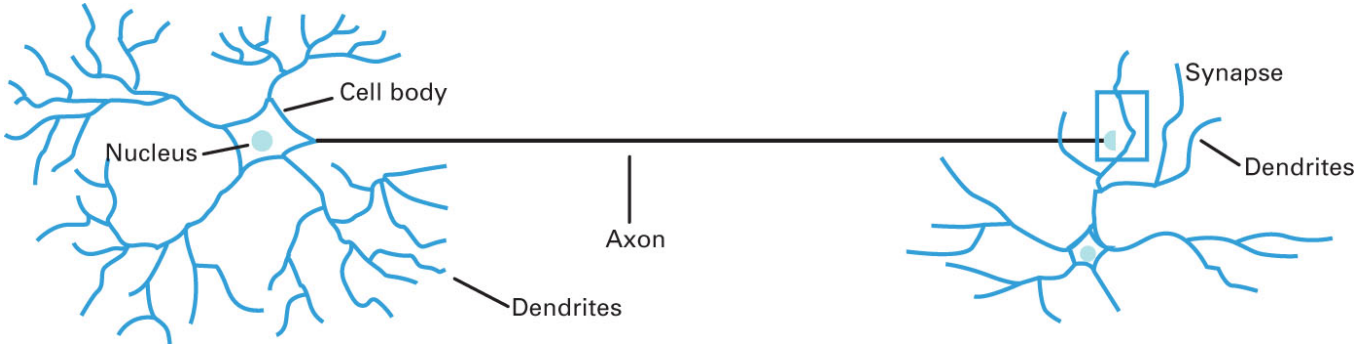
McCulloch & Pitts 1943

Neuron

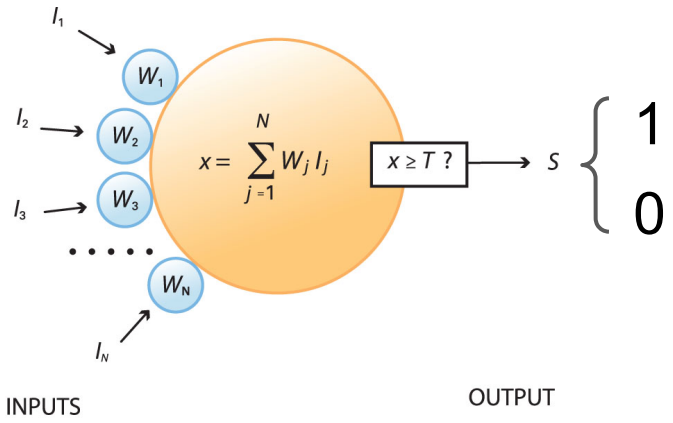




Neuron



Model neuron



I_i	Input $_i$
W_i	The weight attached to input $_i$
T	The threshold of the neuron
X	The total input to the neuron
S	The output signal

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY*

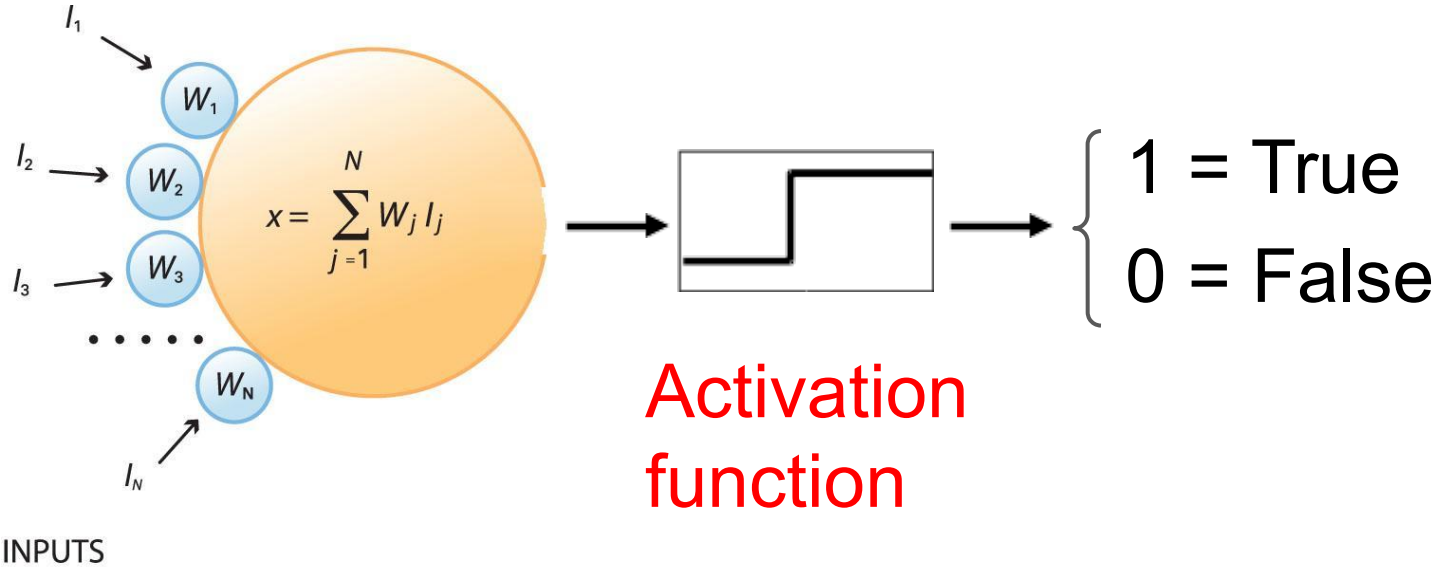
■ WARREN S. MCCULLOCH AND WALTER PITTS
University of Illinois, College of Medicine,
Department of Psychiatry at the Illinois Neuropsychiatric Institute,
University of Chicago, Chicago, U.S.A.

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.

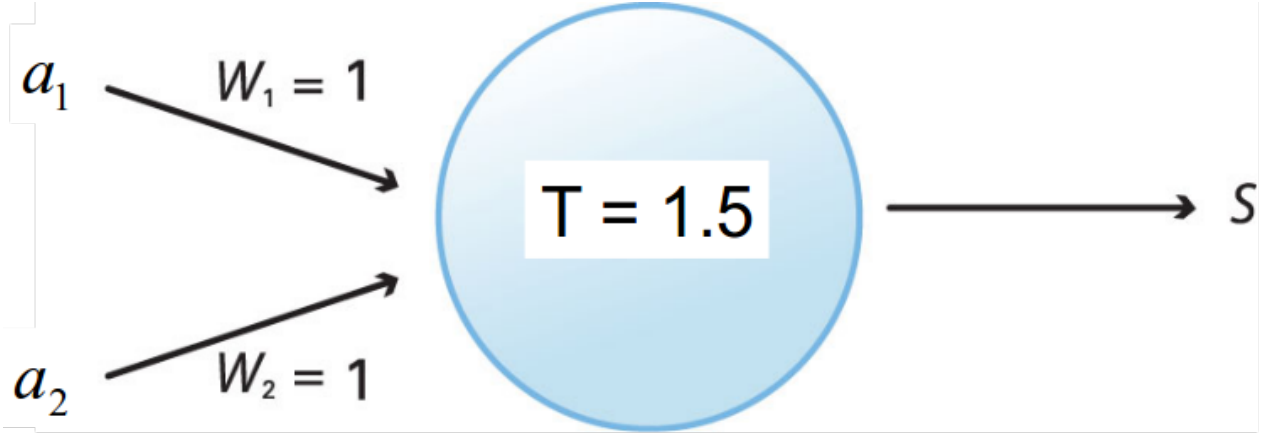
* Reprinted from the *Bulletin of Mathematical Biophysics*, Vol. 5, pp. 115–133 (1943).

McCulloch & Pitts listed only 3 references. All concerned logic:

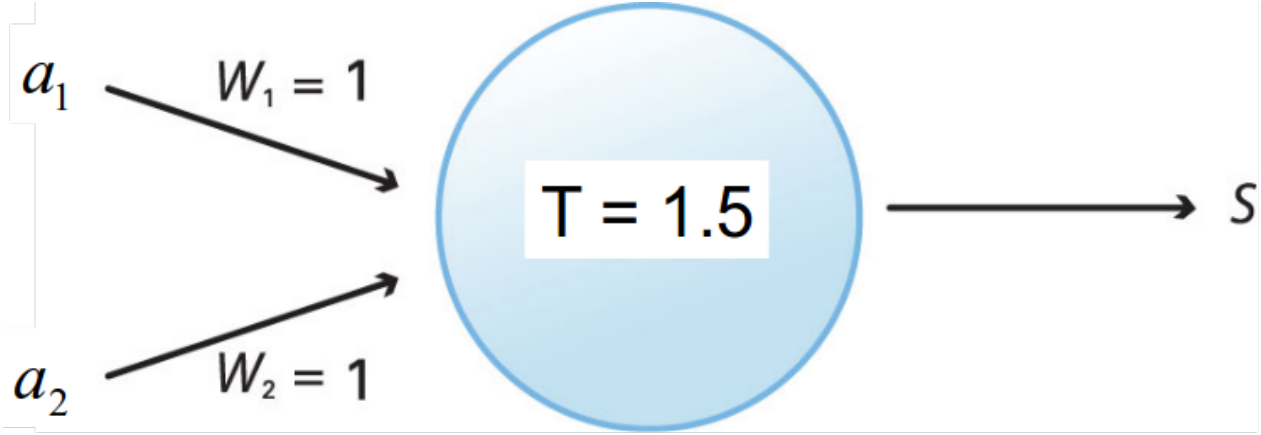
- Carnap, R. 1938. *The logical syntax of language*. New York: Harcourt-Brace.
- Hilbert, D. & W. Ackermann. 1927. *Grundzüge der Theoretischen Logik*. Berlin: Springer.
- Russell, B. & A. N. Whitehead. 1925. *Principia Mathematica*. Cambridge University Press.



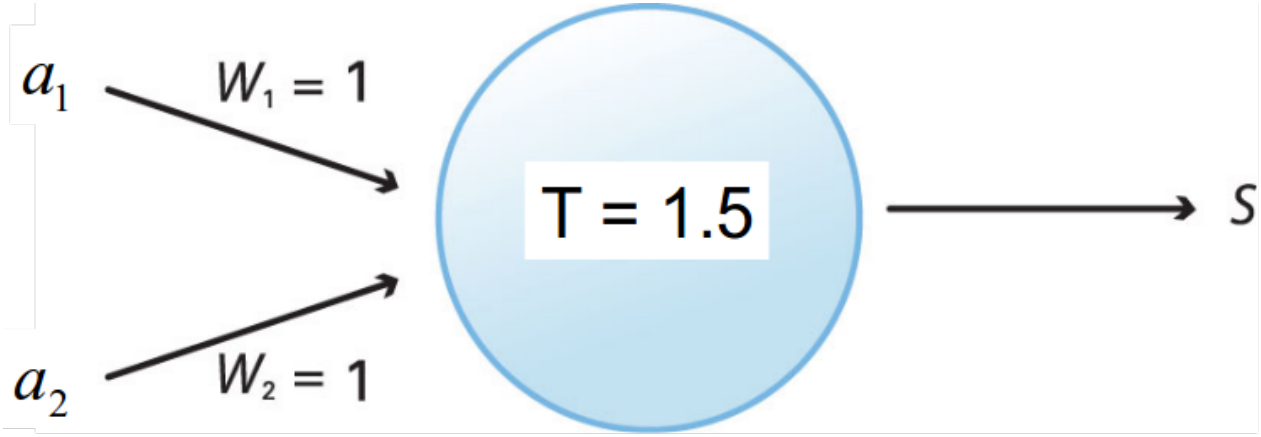
Implementing **propositional logic** using **neuron-like** computing elements.



a_1	a_2	S
0	0	
0	1	
1	0	
1	1	

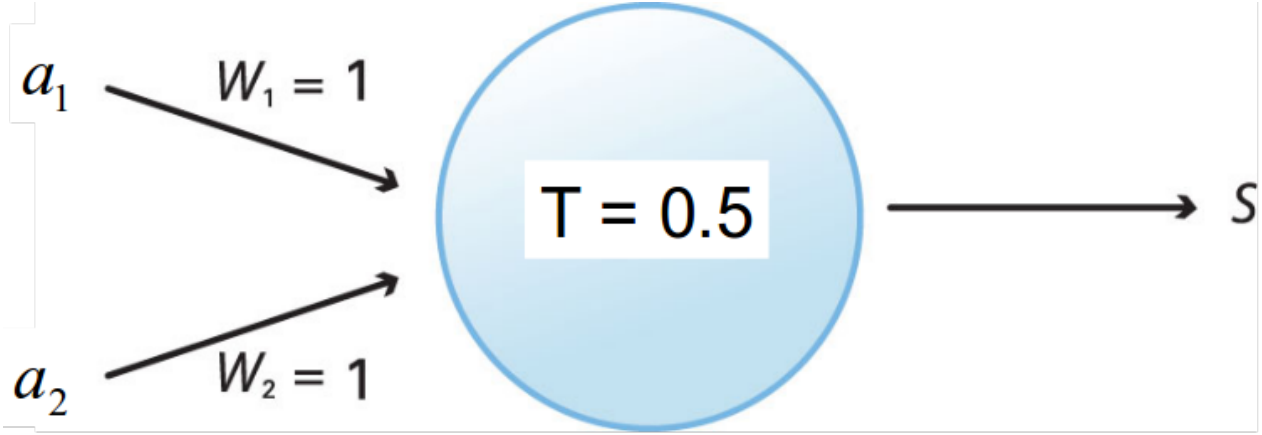


a_1	a_2	S
0	0	0
0	1	0
1	0	0
1	1	1

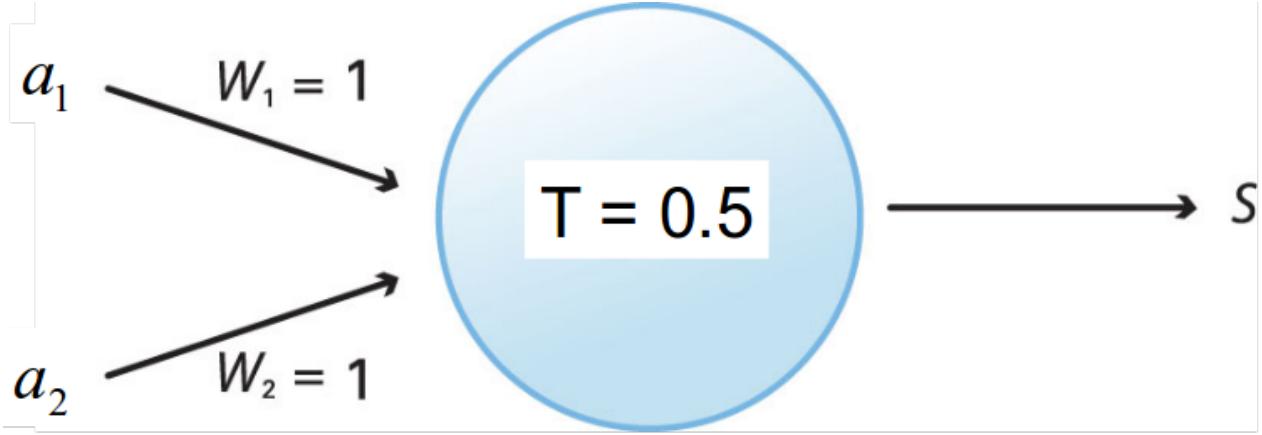


a_1	a_2	S
0	0	0
0	1	0
1	0	0
1	1	1

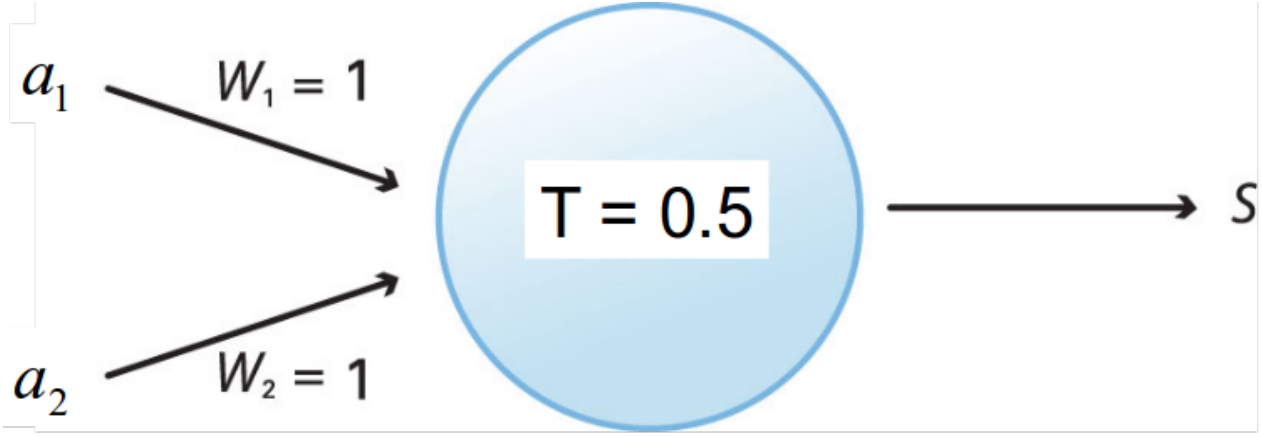
AND



a_1	a_2	S
0	0	
0	1	
1	0	
1	1	

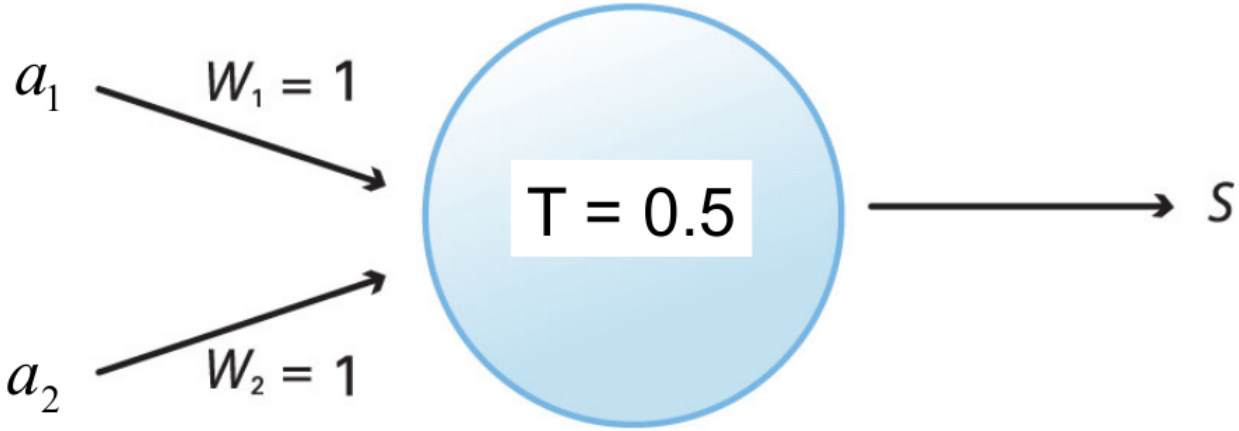


a_1	a_2	S
0	0	0
0	1	1
1	0	1
1	1	1

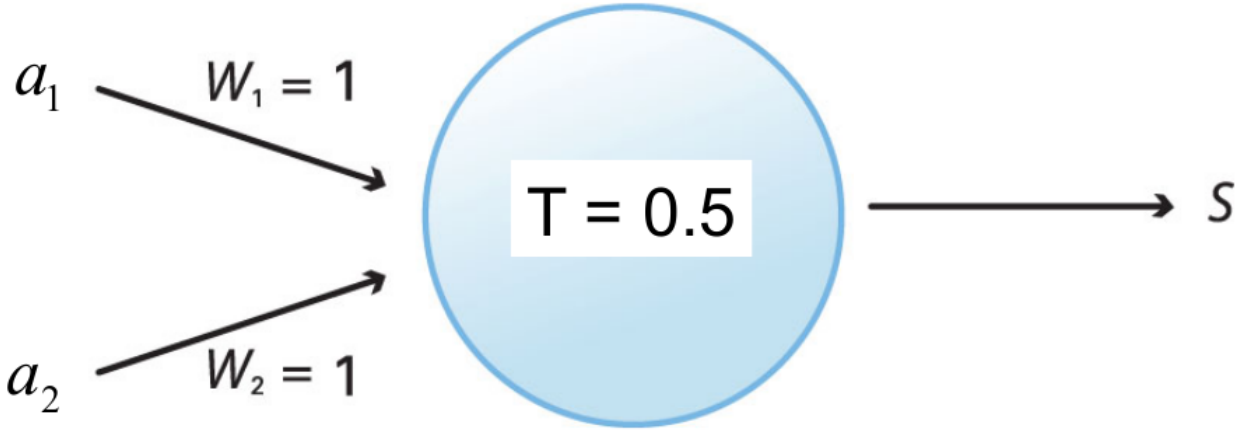


a_1	a_2	S
0	0	0
0	1	1
1	0	1
1	1	1

OR



The **boundary** between inputs yielding responses of 0 vs. 1 is: $w_1 a_1 + w_2 a_2 = T$

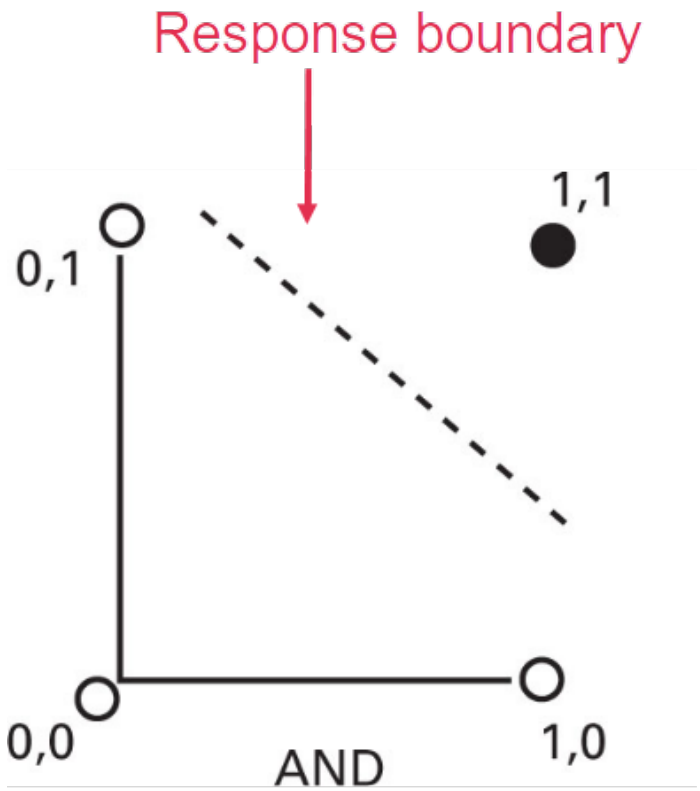


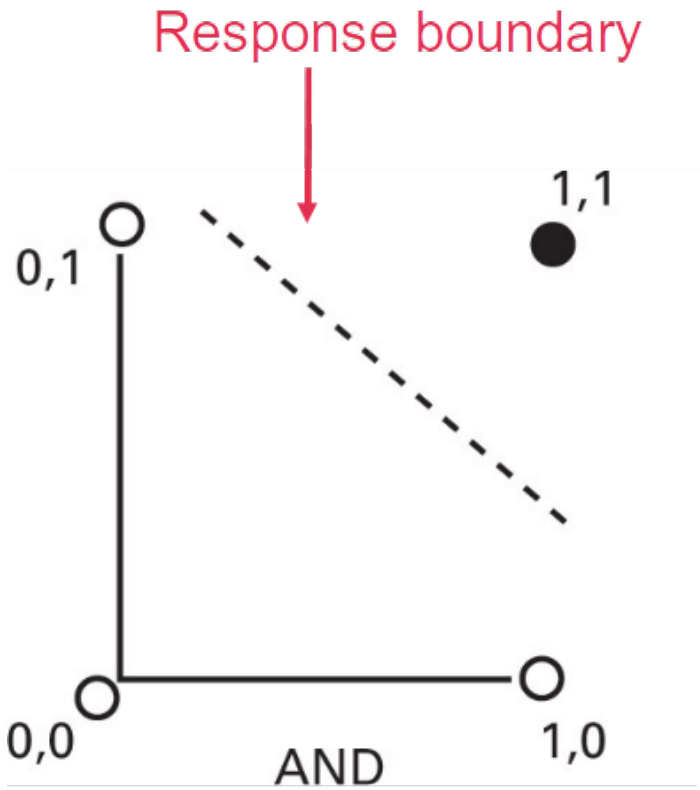
The **boundary** between inputs yielding responses of 0 vs. 1 is: $w_1 a_1 + w_2 a_2 = T$

slope → intercept

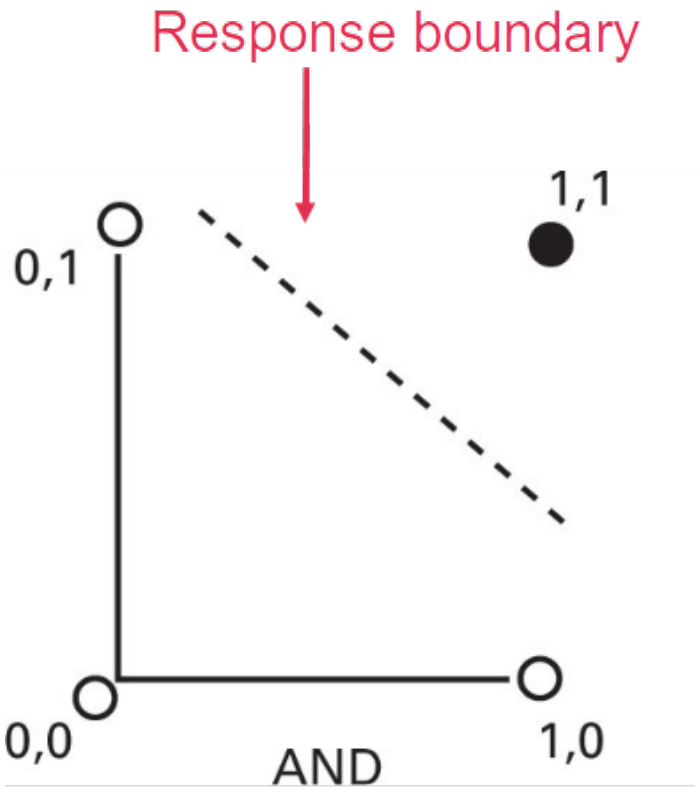
$$a_2 = \left(-\frac{w_1}{w_2} \right) a_1 + \left(\frac{T}{w_2} \right)$$

Defines a **line** in 2-d (a_1,a_2) **input space**.


Response boundary
1,1
0,1
0,0
1,0
AND

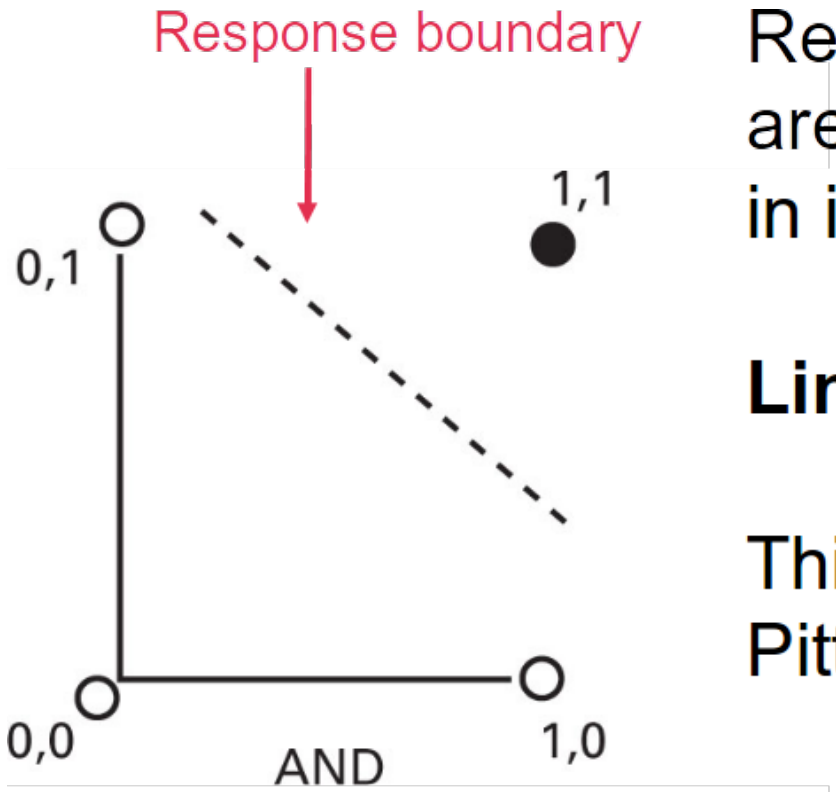


Responses of 0 vs. 1 are separated by a **line** in input space.



Responses of 0 vs. 1 are separated by a **line** in input space.

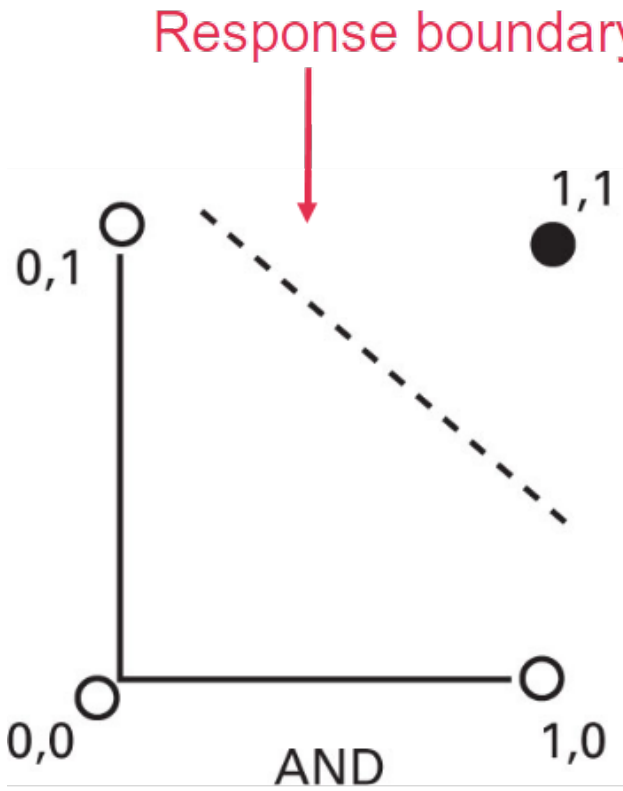
Linear separation



Responses of 0 vs. 1 are separated by a **line** in input space.

Linear separation

This is what McCulloch-Pitts model neurons do.

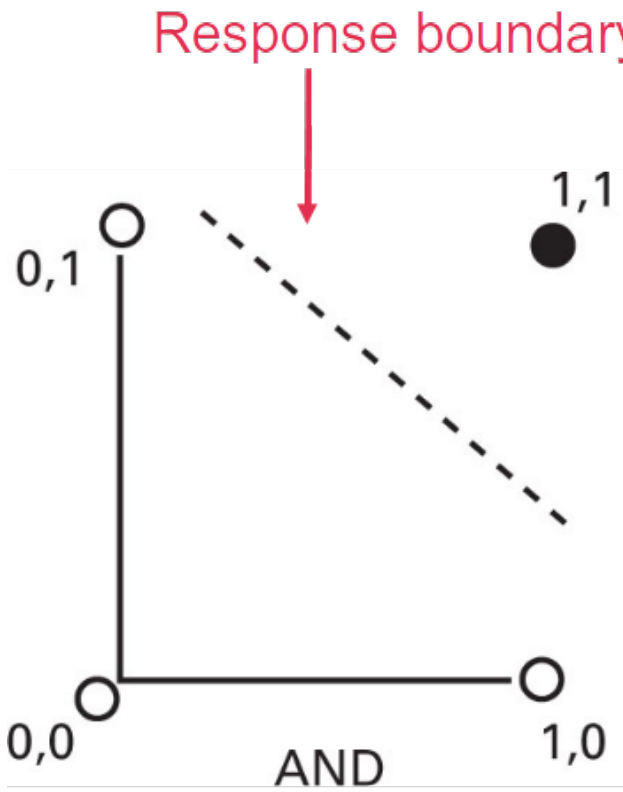


Responses of 0 vs. 1 are separated by a **line** in input space.

Linear separation

This is what McCulloch-Pitts model neurons do.

Can change line by **changing weights, threshold**.



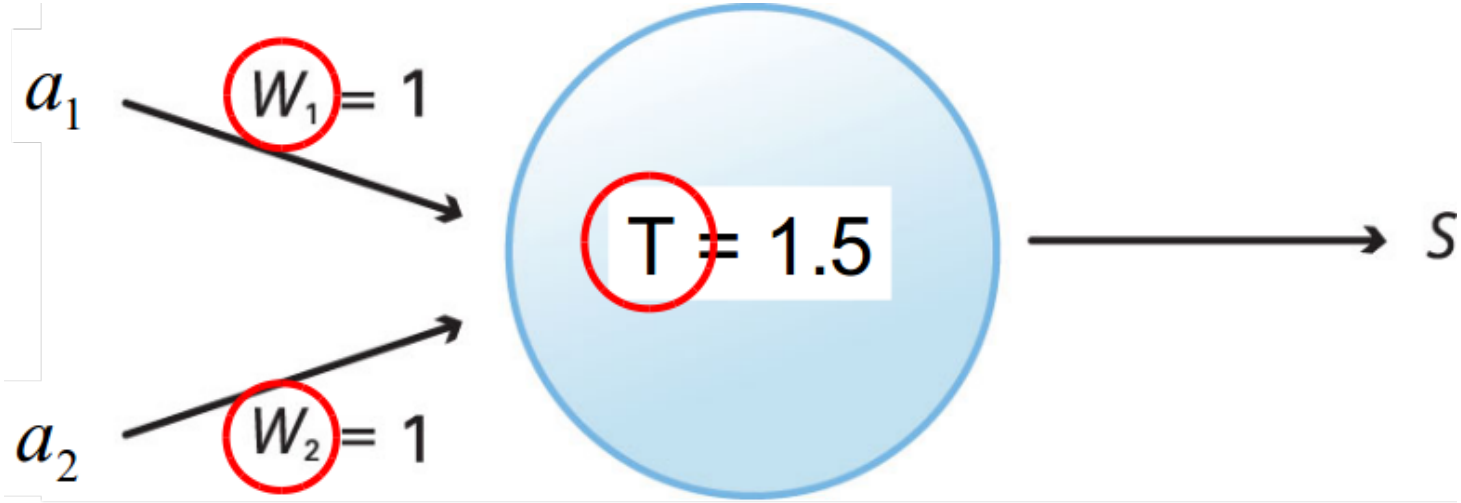
Responses of 0 vs. 1 are separated by a **line** in input space.

Linear separation

This is what McCulloch-Pitts model neurons do.

Learning ⟶

Can change line by **changing weights, threshold**.



Learning is a process of adjusting these quantities.

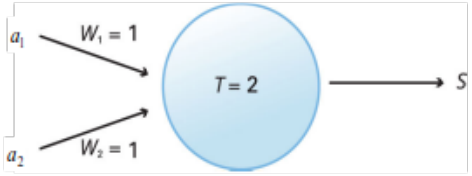
What **questions** do you have?

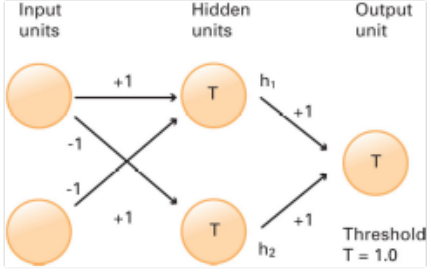
What **comments** do you have?

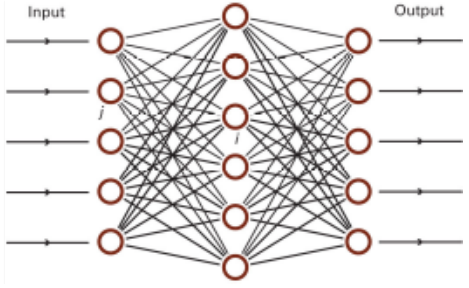
Overview

1. Neurons and logic
2. Perceptrons and learning
3. Generative AI
4. Transformers and large language models

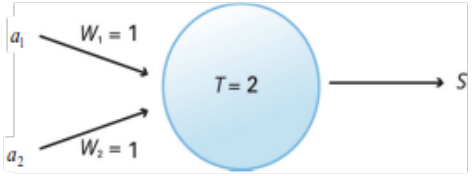
Types of neural networks

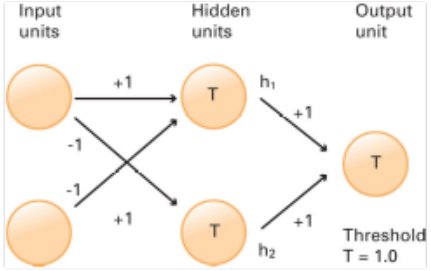




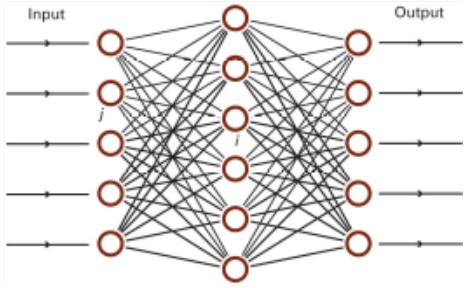


Types of neural networks

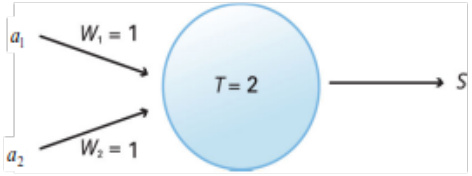


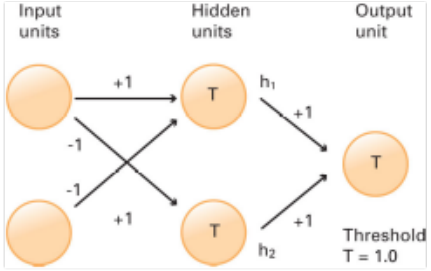


Instead of a **complex processor**, we have **simple ones,** interconnected. The “smarts” are in the **connections**, not the processors.



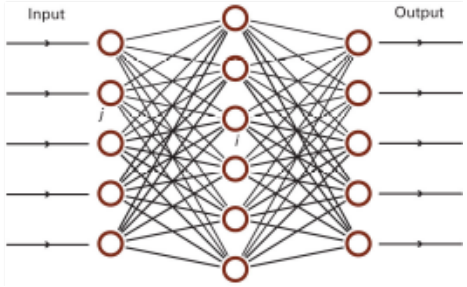
Types of neural networks



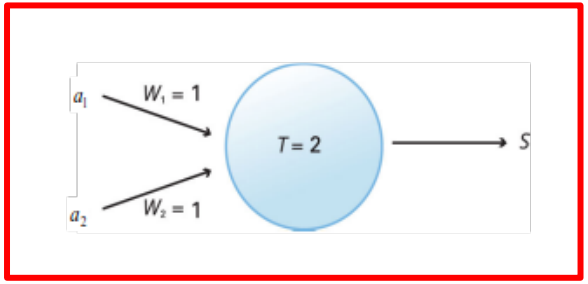


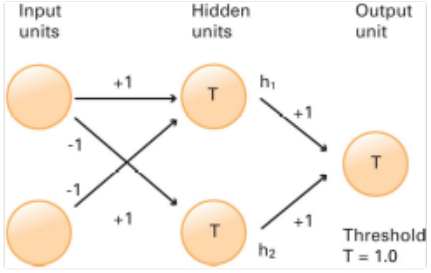
Connectionism

Instead of a **complex processor**, we have **simple ones,** interconnected. The “smarts” are in the **connections**, not the processors.



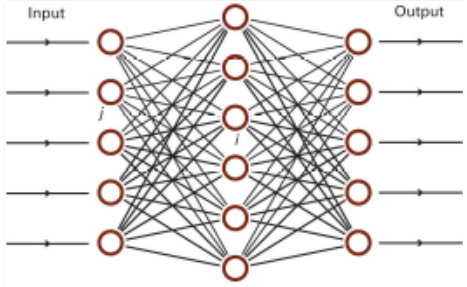
Types of neural networks





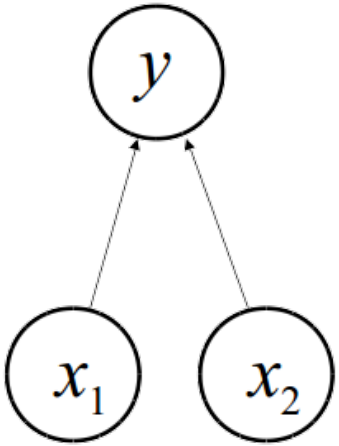
Connectionism

Instead of a **complex processor**, we have **simple ones,** interconnected. The “smarts” are in the **connections**, not the processors.



Perceptrons

+1 = cat, -1 = dog

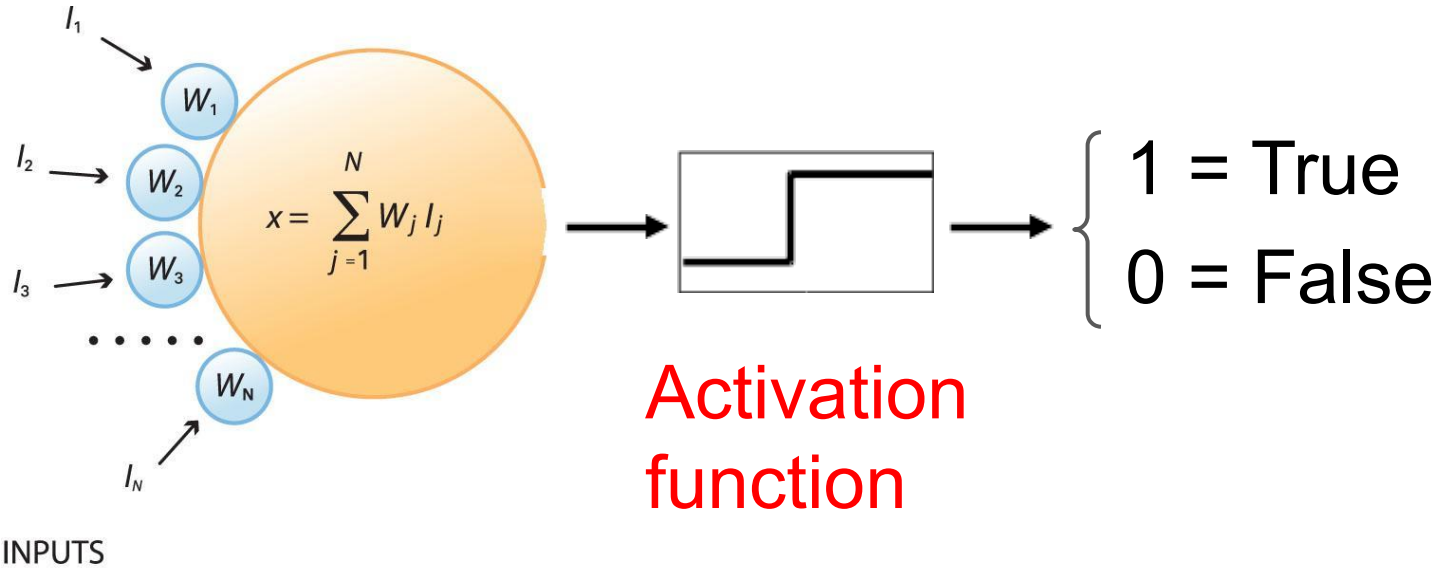


perceptual features

Trainable! Need not set weights by hand.

The simplest neural network

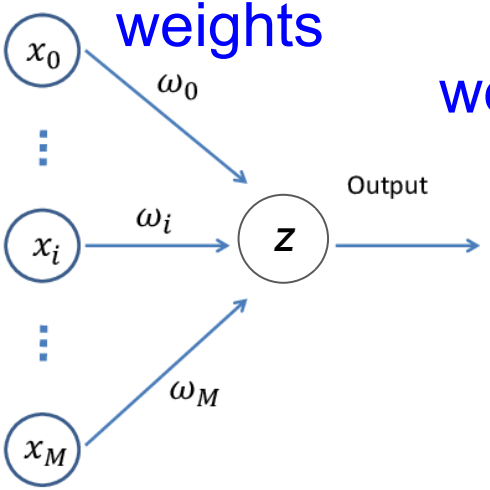
Originally introduced by Frank Rosenblatt (1958)


I_1
W_1
I_2
W_2
$x = \sum_{j=1}^{N} W_j I_j$
I_3
W_3
W_N
I_N
INPUTS
1 = True
0 = False
Activation function

Logistic regression as a pipeline

features

weights

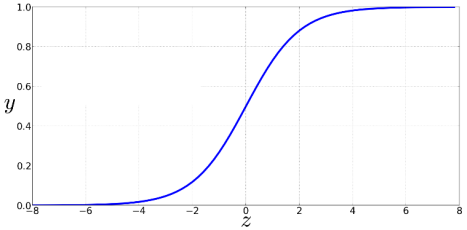


weighted sum, z

$$\sum_{i=0}^{M} \omega_i x_i = \boldsymbol{w} \cdot \boldsymbol{x}$$

sigmoid

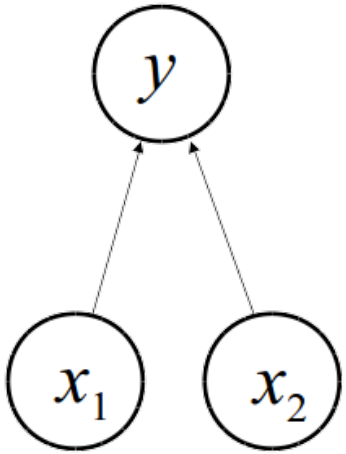
$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

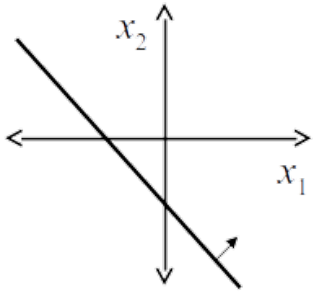


classification probability

$P(y = 1 \mid x)$

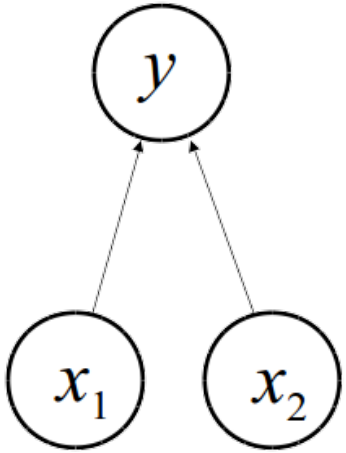
Limitation of perceptrons

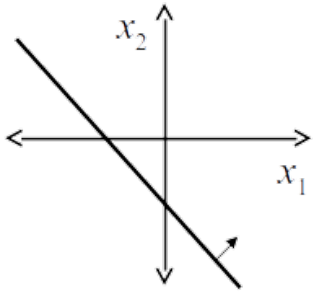




Can only learn **linearly separable** categories

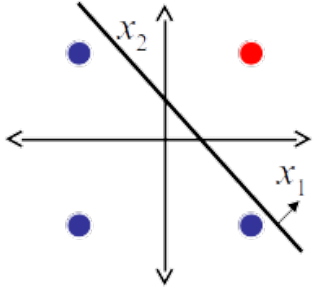
Limitation of perceptrons



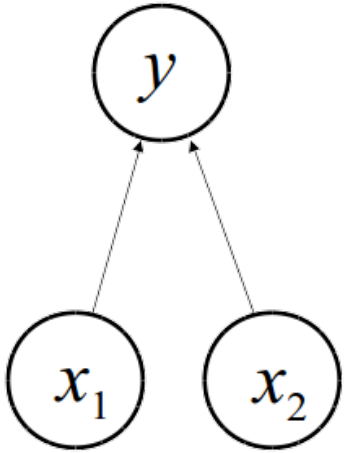


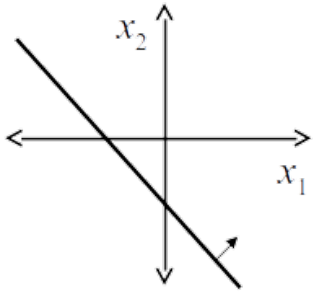
Can only learn **linearly separable** categories

AND



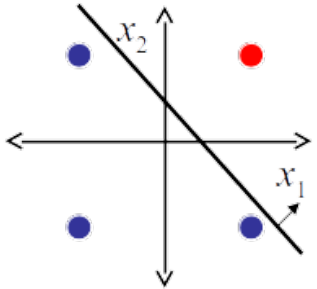
Limitation of perceptrons



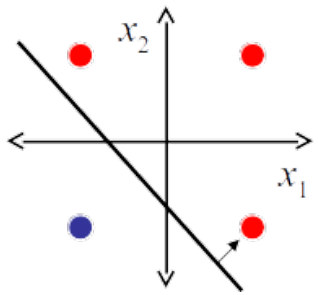


Can only learn **linearly separable** categories

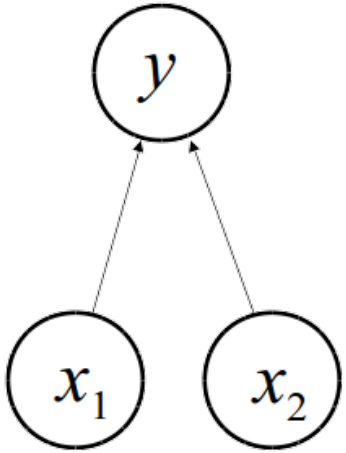
AND

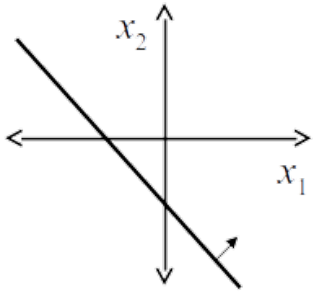


OR



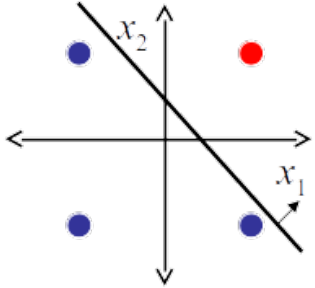
Limitation of perceptrons



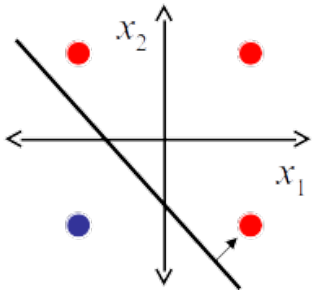


Can only learn **linearly separable** categories

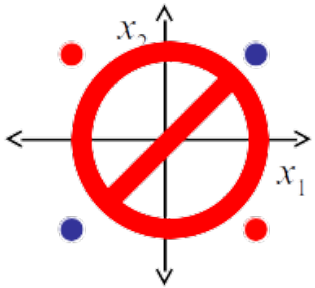
AND



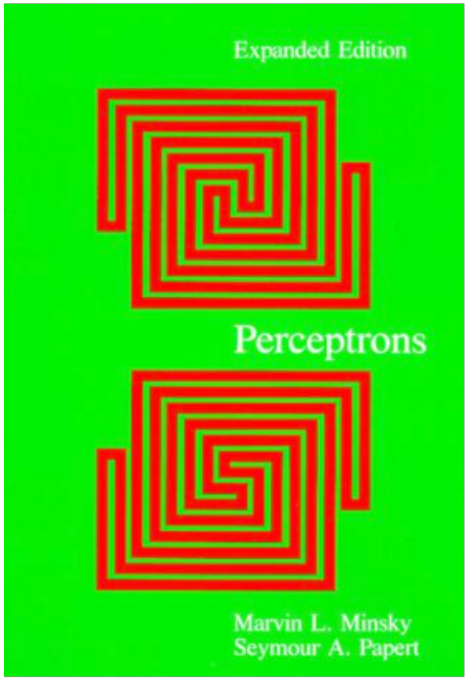
OR



XOR

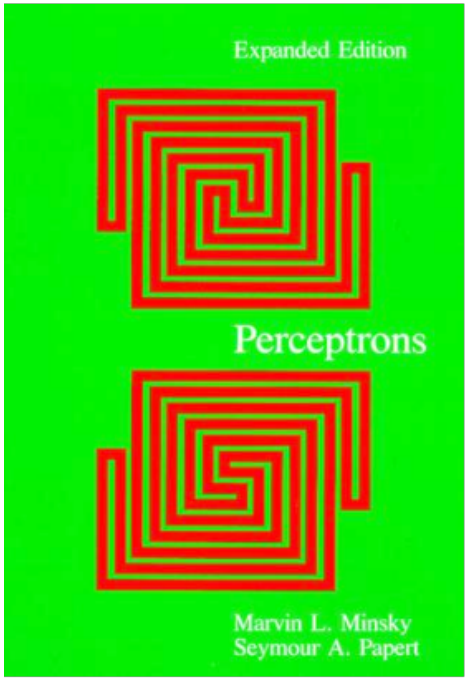


Limitation of perceptrons



The simple logical XOR function cannot be **represented**, let alone **learned**, by a 1-layer perceptron.

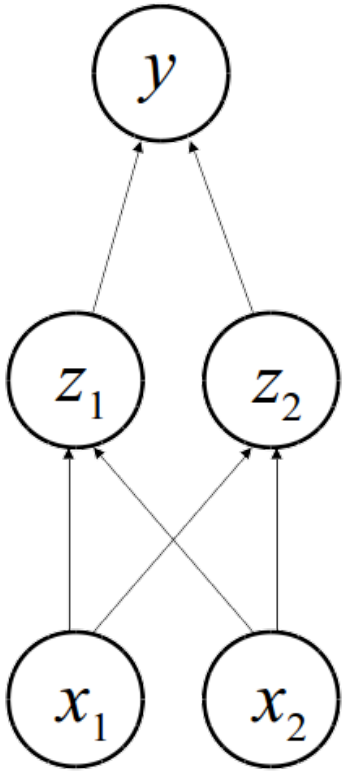
Limitation of perceptrons



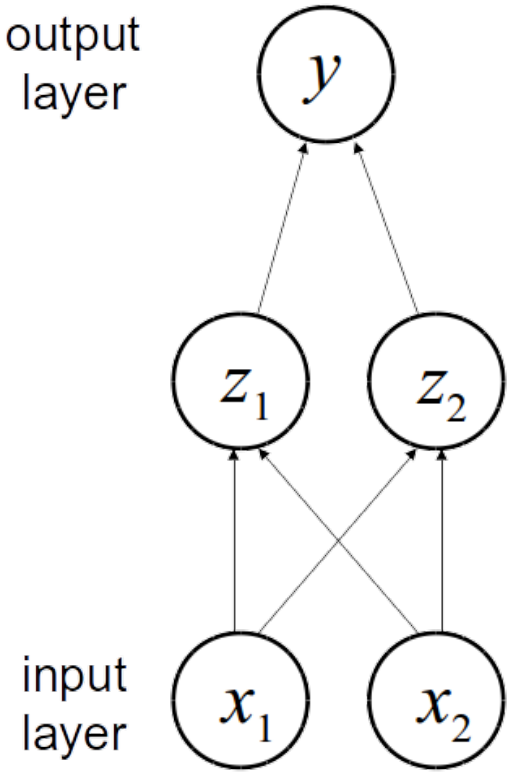
The simple logical XOR function cannot be **represented**, let alone **learned**, by a 1-layer perceptron.

What would it take to do this, and more interesting complex functions?

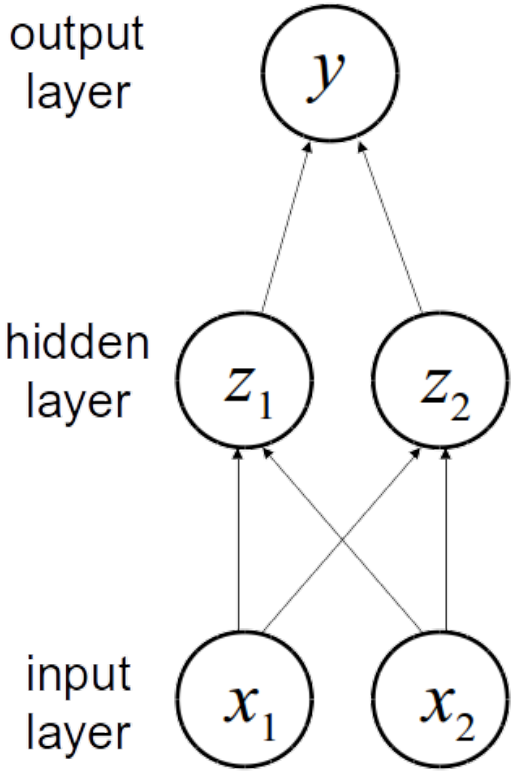
A solution: multiple layers



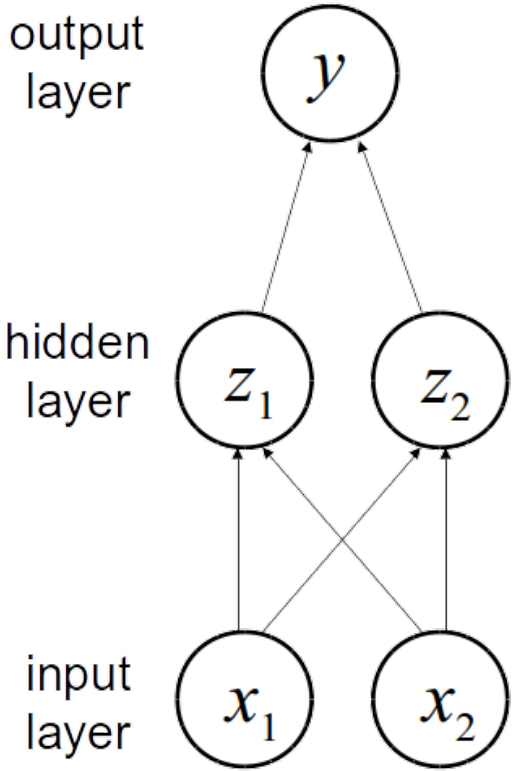
A solution: multiple layers

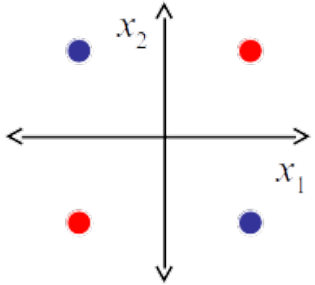


A solution: multiple layers

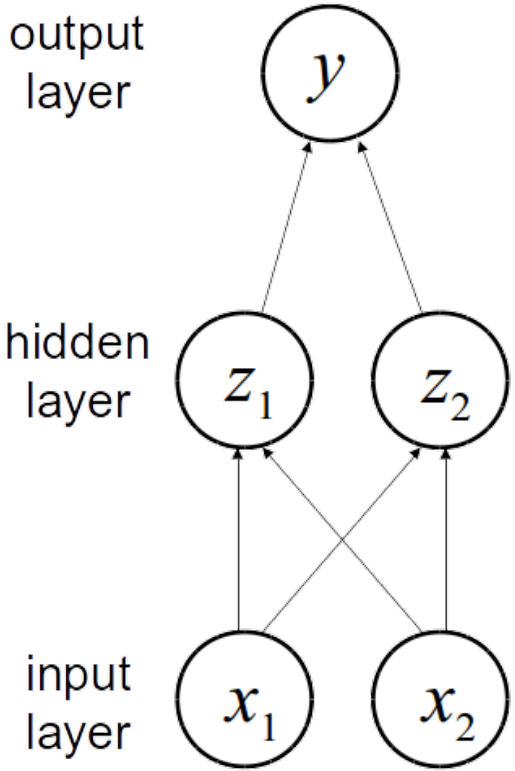


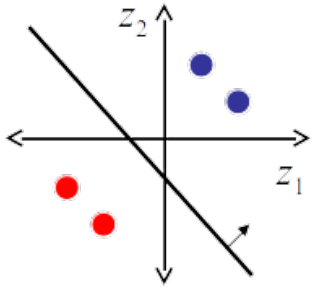
A solution: multiple layers

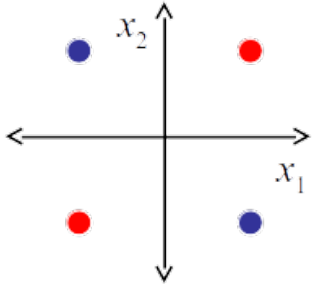




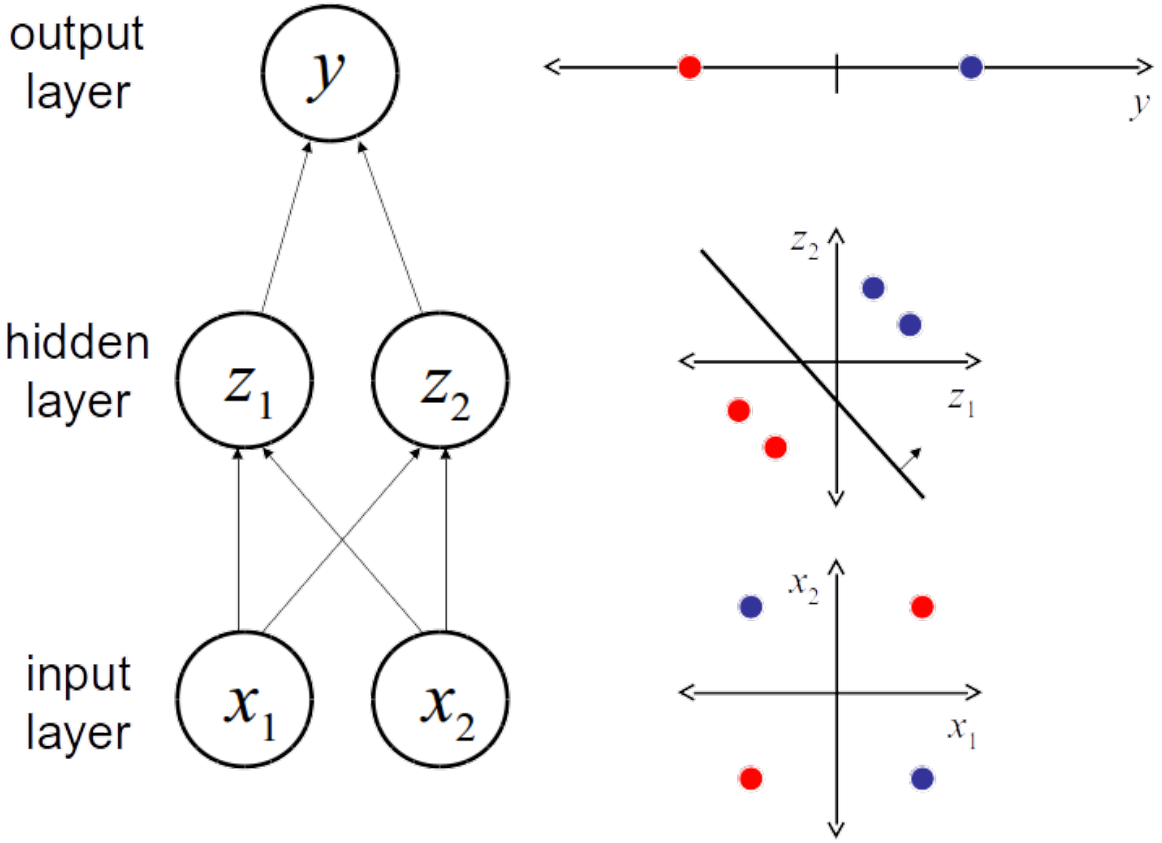
A solution: multiple layers



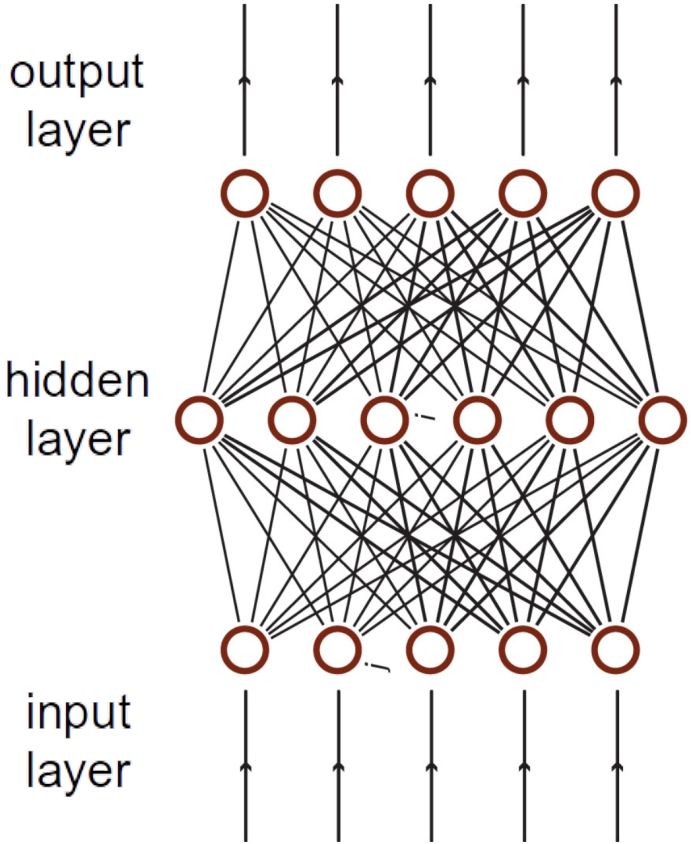




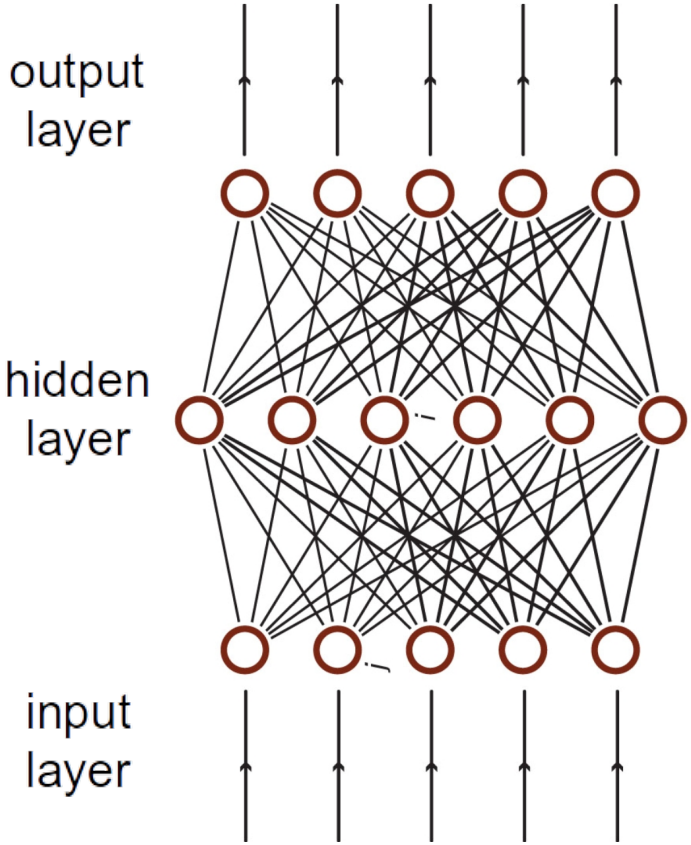
A solution: multiple layers



A solution: multiple layers

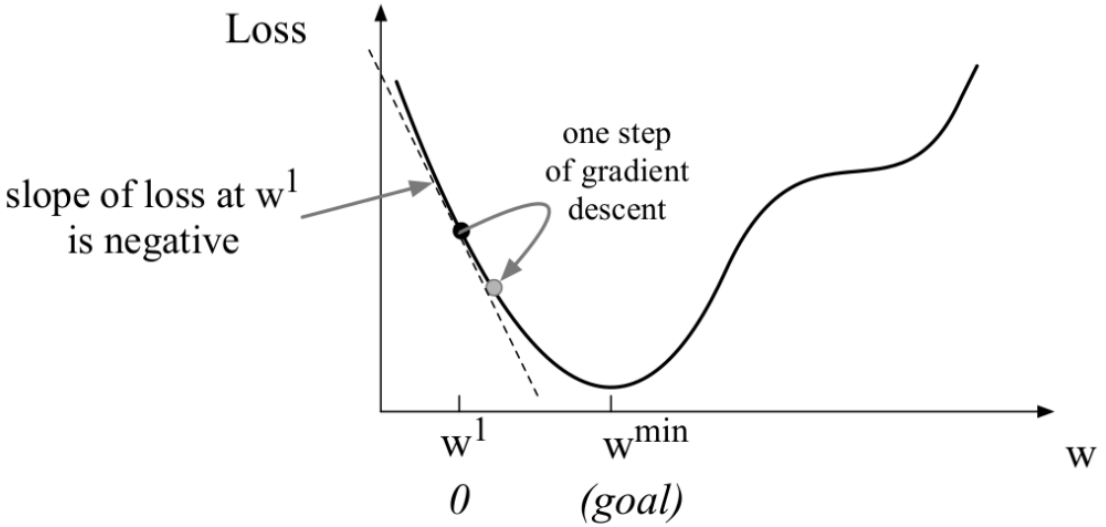


A solution: multiple layers

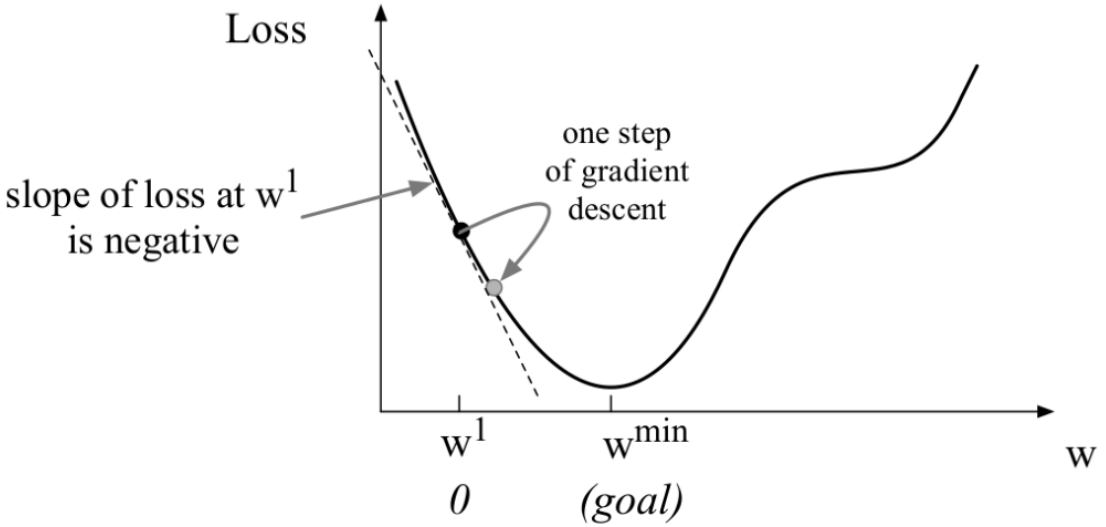


A more complex example of the same idea:

a **multi-layer perceptron**.

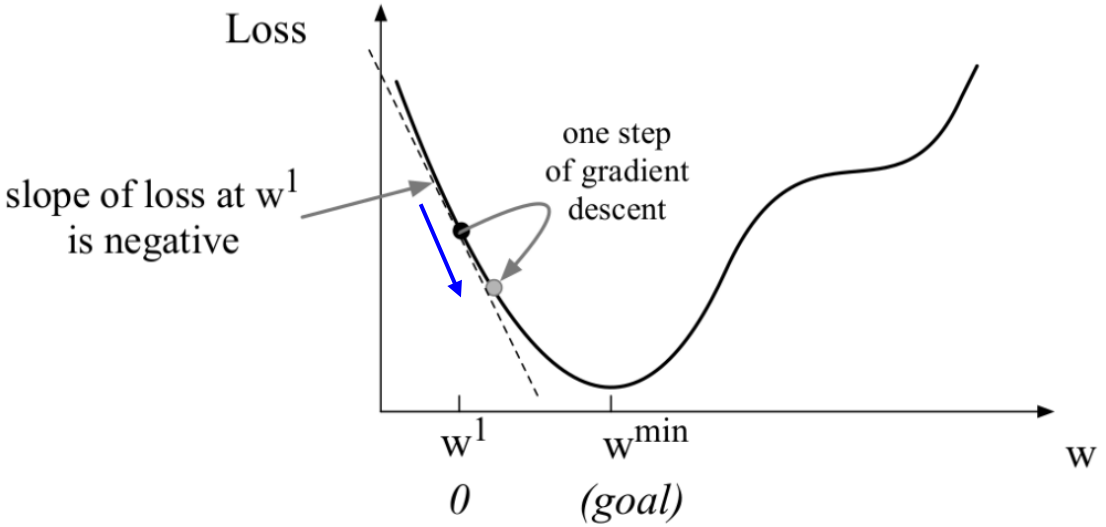


Learning via **gradient descent**, as with logistic regression.



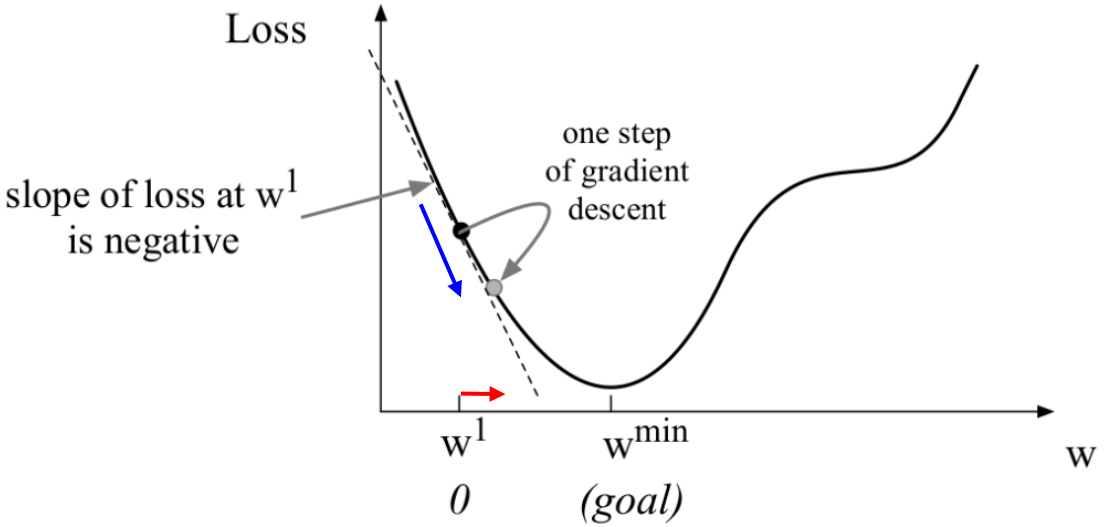
$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

Learning via **gradient descent**, as with logistic regression.



$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

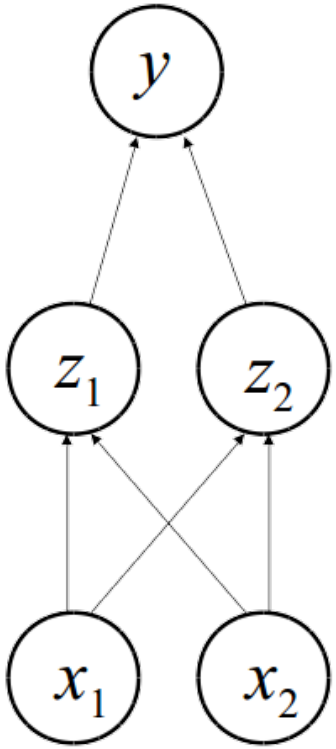
Learning via **gradient descent**, as with logistic regression.



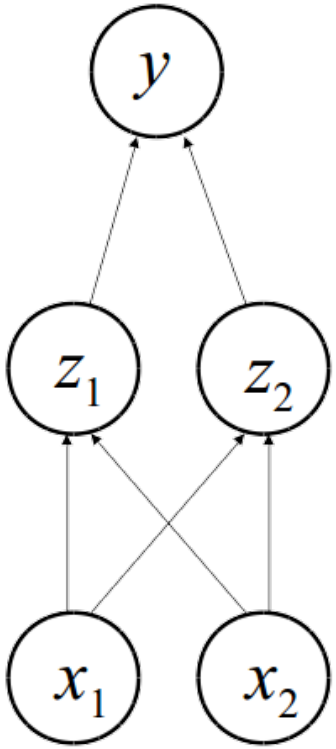
$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

Learning via **gradient descent**, as with logistic regression.

Training multi-layer networks

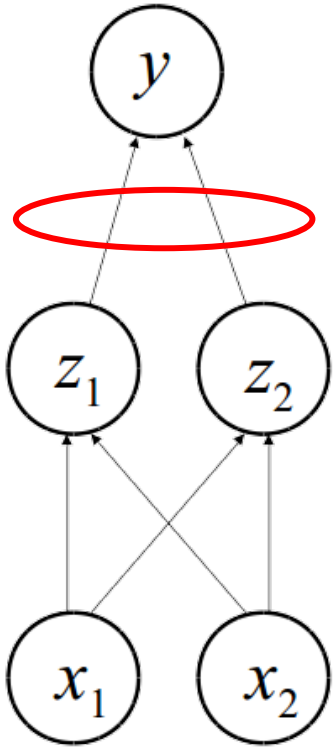


Training multi-layer networks



compute error

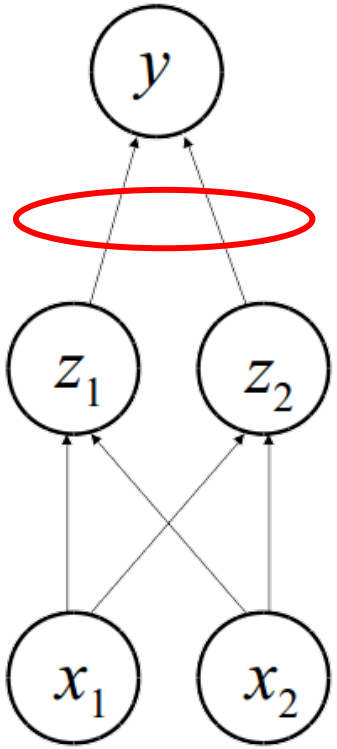
Training multi-layer networks



compute error

use gradient descent

Training multi-layer networks

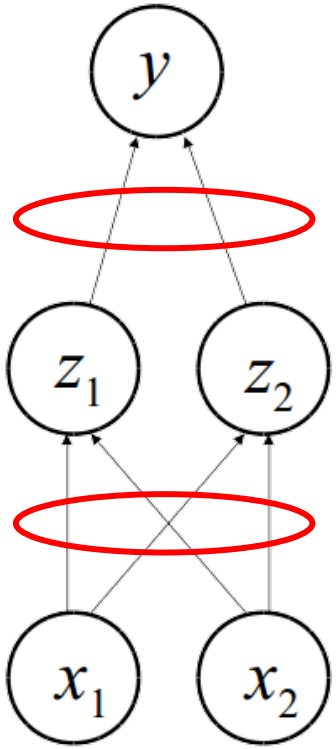


compute error

use gradient descent

propagate error to hidden layer

Training multi-layer networks



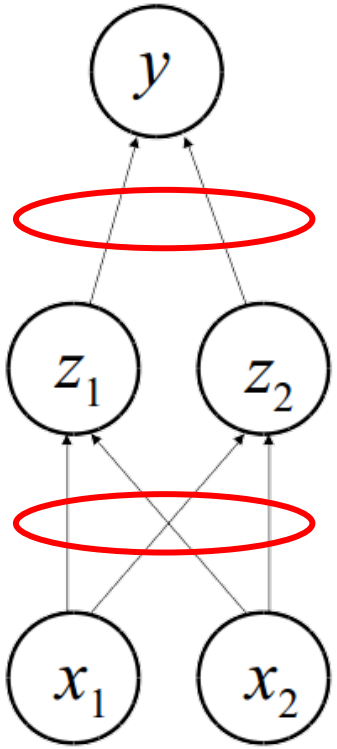
compute error

use gradient descent

propagate error to hidden layer

use gradient descent **again**

Training multi-layer networks



compute error

use gradient descent

propagate error to hidden layer

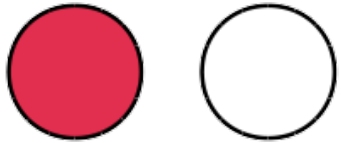
use gradient descent **again**

Back-propagation

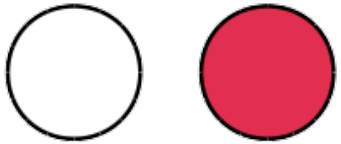
Rumelhart, Hinton, & Wilson, 1986

Different representations

dog



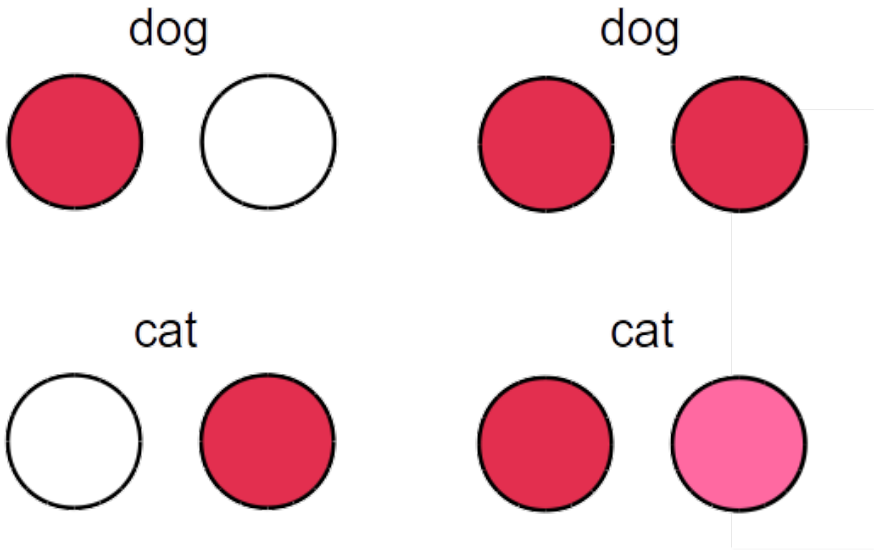
cat



Localist

One node
per concept

Different representations



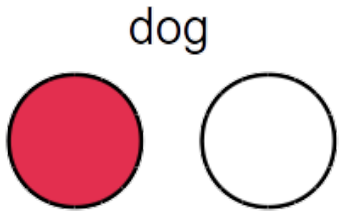
Localist

One node
per concept

Distributed

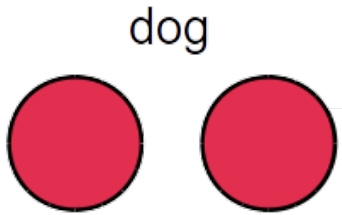
One **pattern of activation** per concept

Different representations



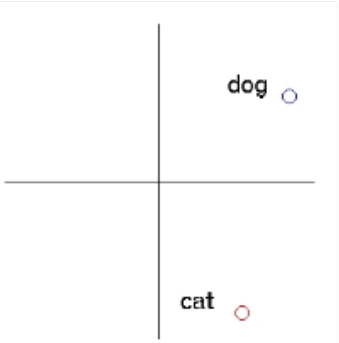
Localist

One node
per concept



Distributed

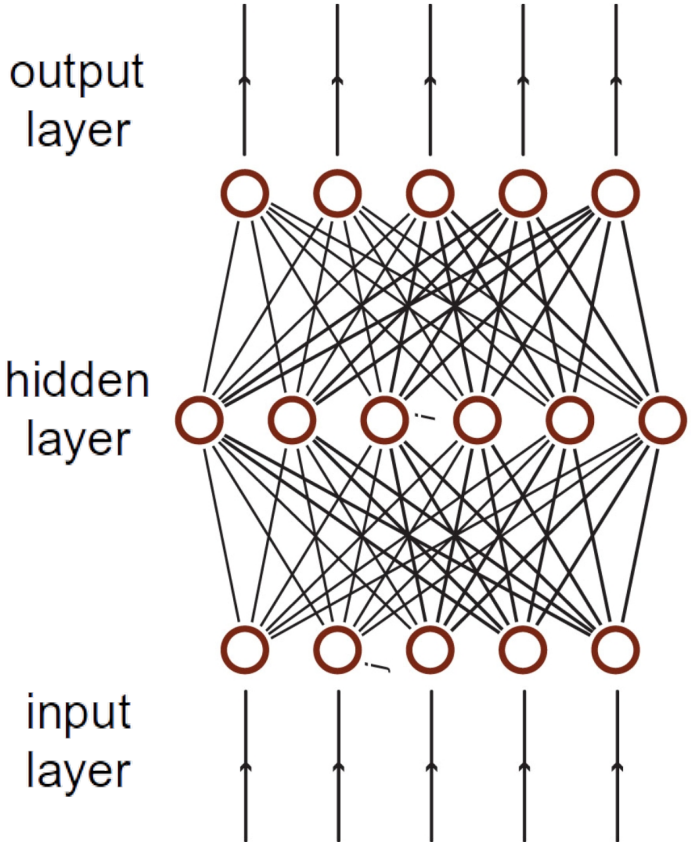
One **pattern of activation** per concept

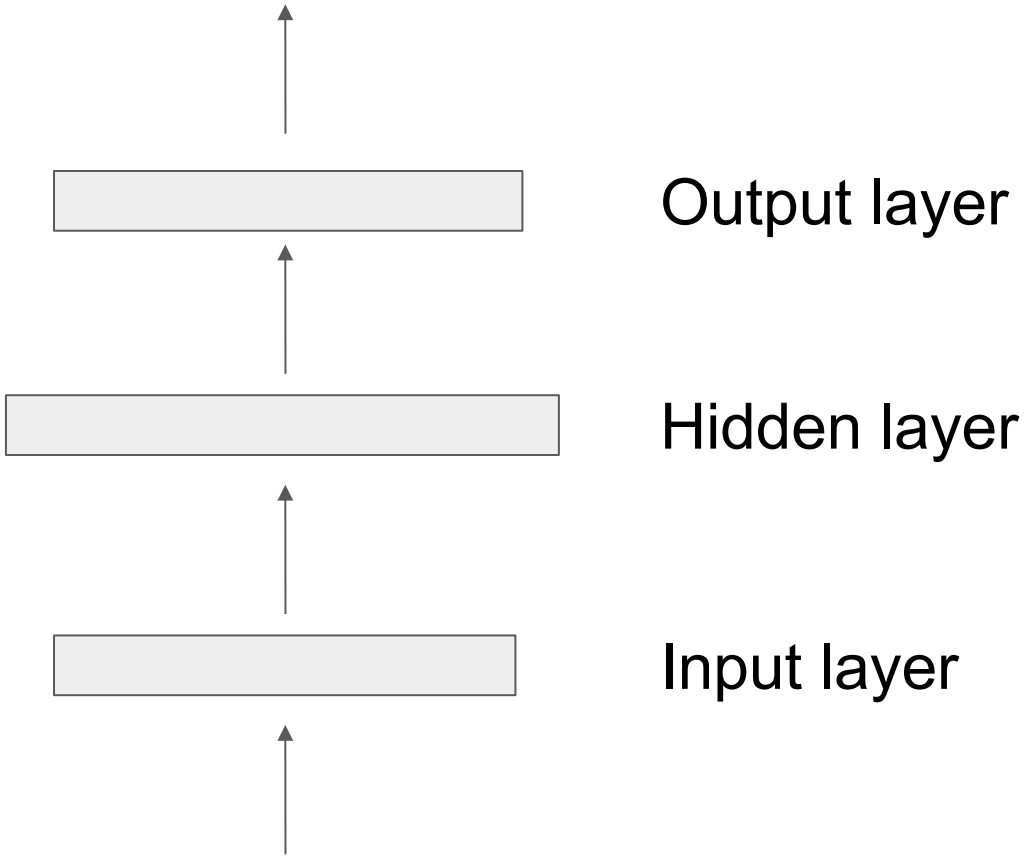


Distributed

Interpretable
as points in
space

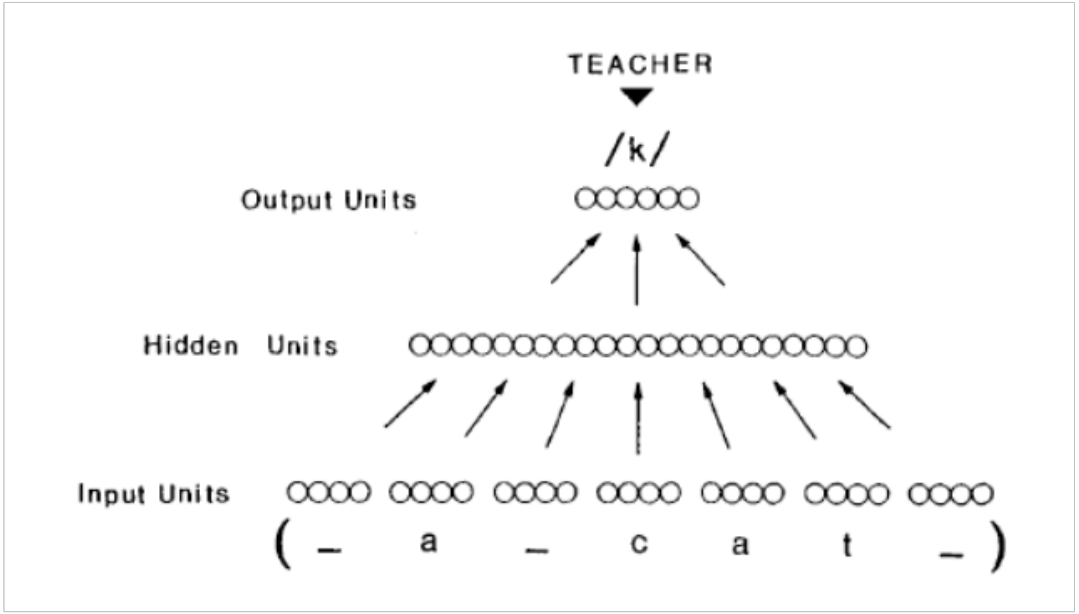
A solution: multiple layers



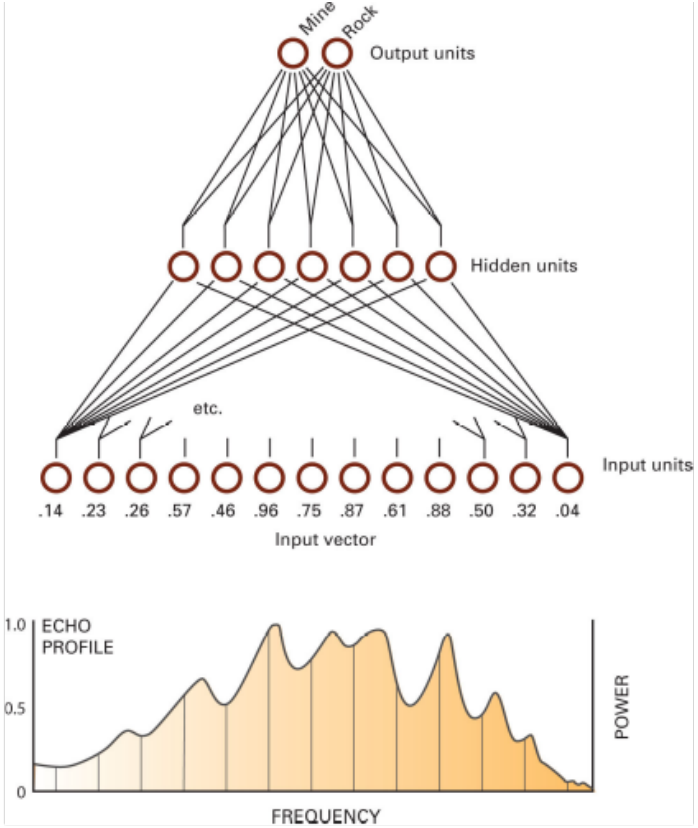

Output layer
Hidden layer
Input layer

A learning example: NETtalk

Sejnowski & Rosenberg, 1987

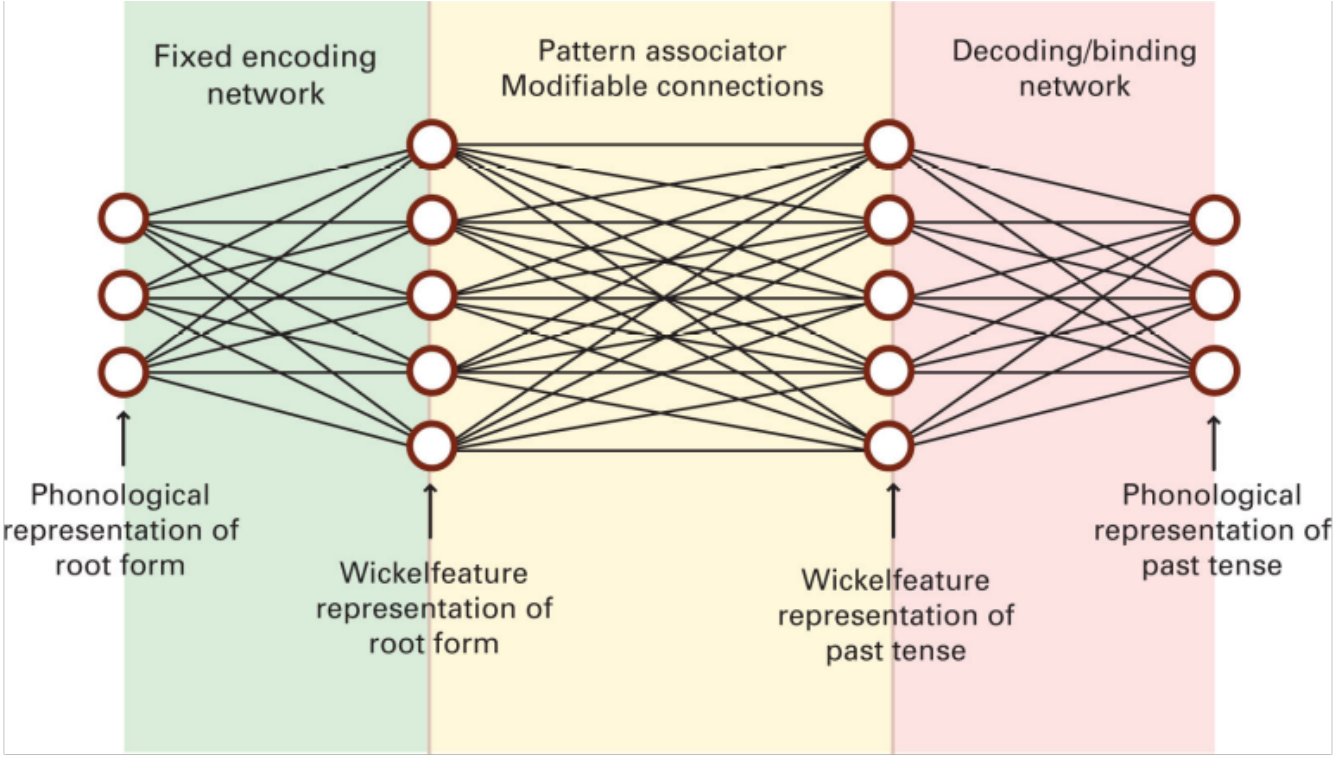


Learned to pronounce English words from written forms (which is pretty hard to do!)



Discriminating sonar pings of mines vs. rocks

Gorman & Sejnowski 1988



Learning rules of language: English past tense

Rumelhart et al. 1986

Recurrent networks

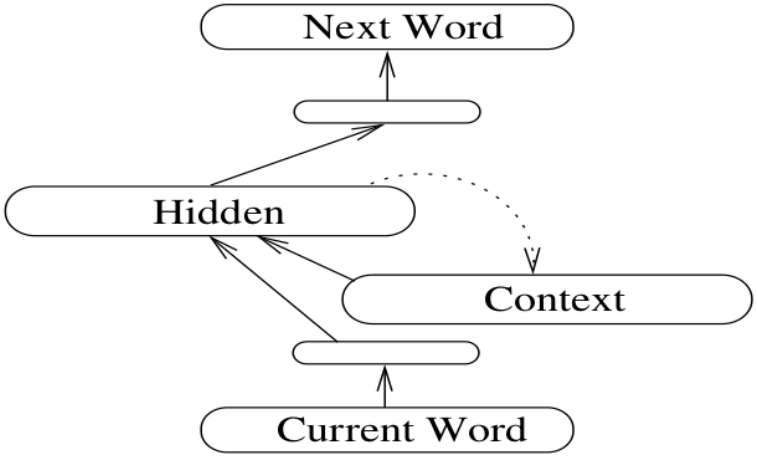


Figure 1: An SRN. Solid lines represent full connectivity; the dashed line indicates unit-to-unit connections. The unlabeled layers are reduction layers.

Recurrent networks

“distributed representations which encode … grammatical relations and hierarchical constituent structure”

Elman 1991

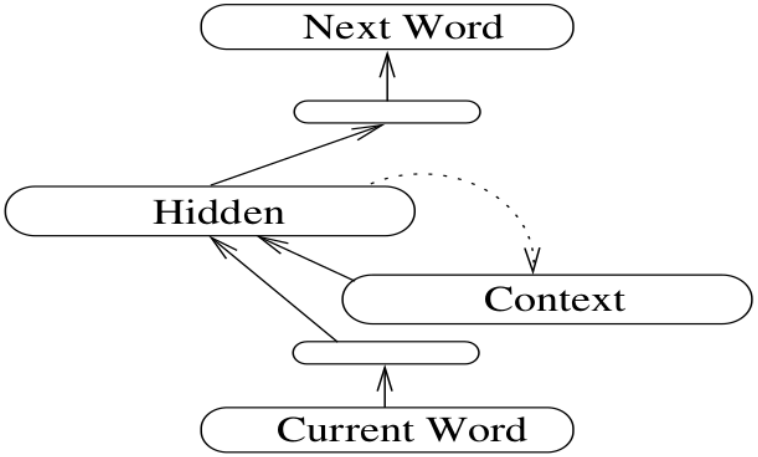


Figure 1: An SRN. Solid lines represent full connectivity; the dashed line indicates unit-to-unit connections. The unlabeled layers are reduction layers.

A prominent line of argument

- Neural networks are **domain-general** learners.
- Therefore, if they succeed at learning some aspect of language from realistic input, perhaps that aspect of language **need not be innate**.
- A response to the **argument from poverty of the stimulus**.

What **questions** do you have?

What **comments** do you have?

David Rumelhart (1942-2011)

- Explored mathematical models of cognition.
- One of the discoverers of backpropagation, which is widely used.
- Inspiration for the annual Rumelhart Prize.

The **Rumelhart Prize** is given annually at the Cognitive Science Society conference, to honor contributions to **theoretical foundations of human cognition**.

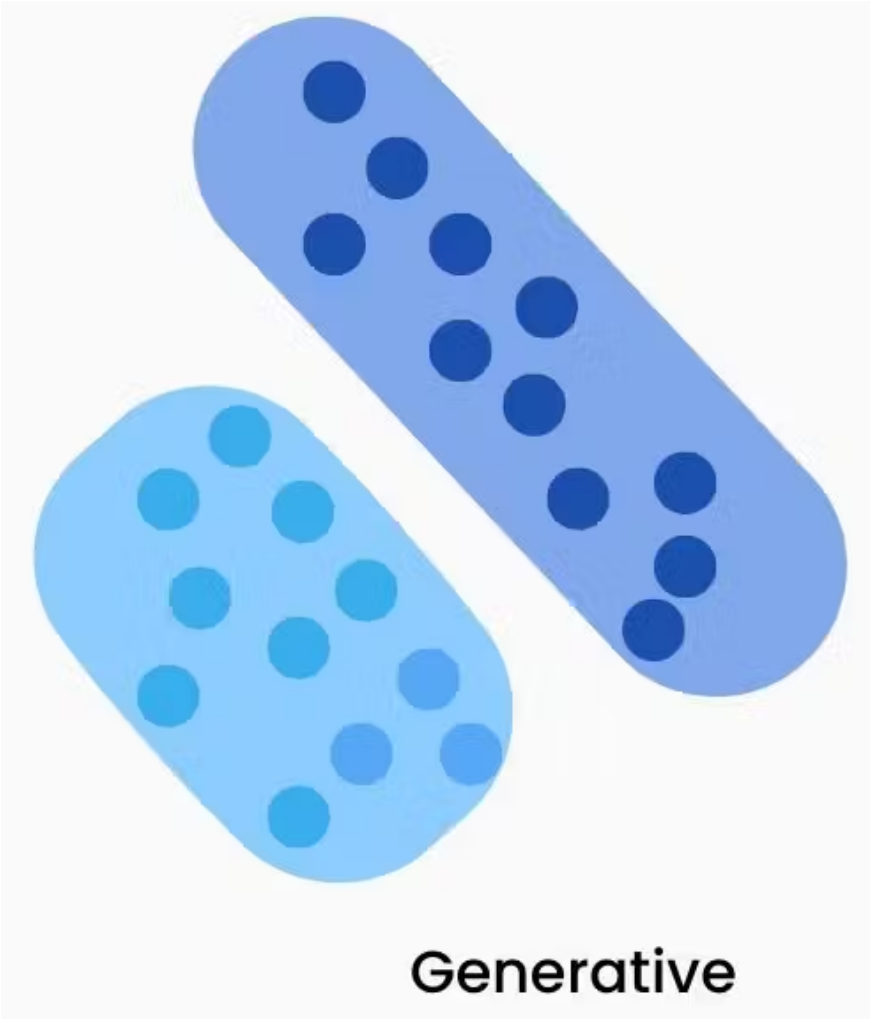
http://rumelhartprize.org/

Douglas Medin

2026 recipient of the Rumelhart Prize

Overview

1. Neurons and logic
2. Perceptrons and learning
3. Generative AI
4. Transformers and large language models



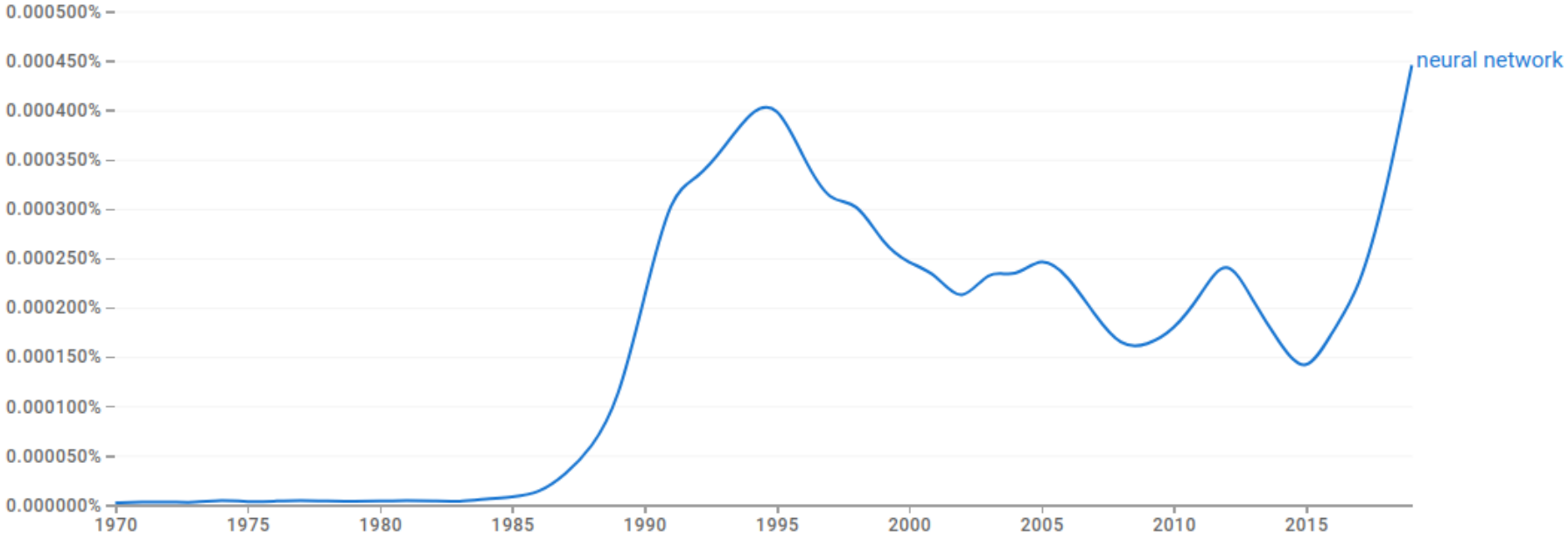
Generative classifier
(as in generative AI)

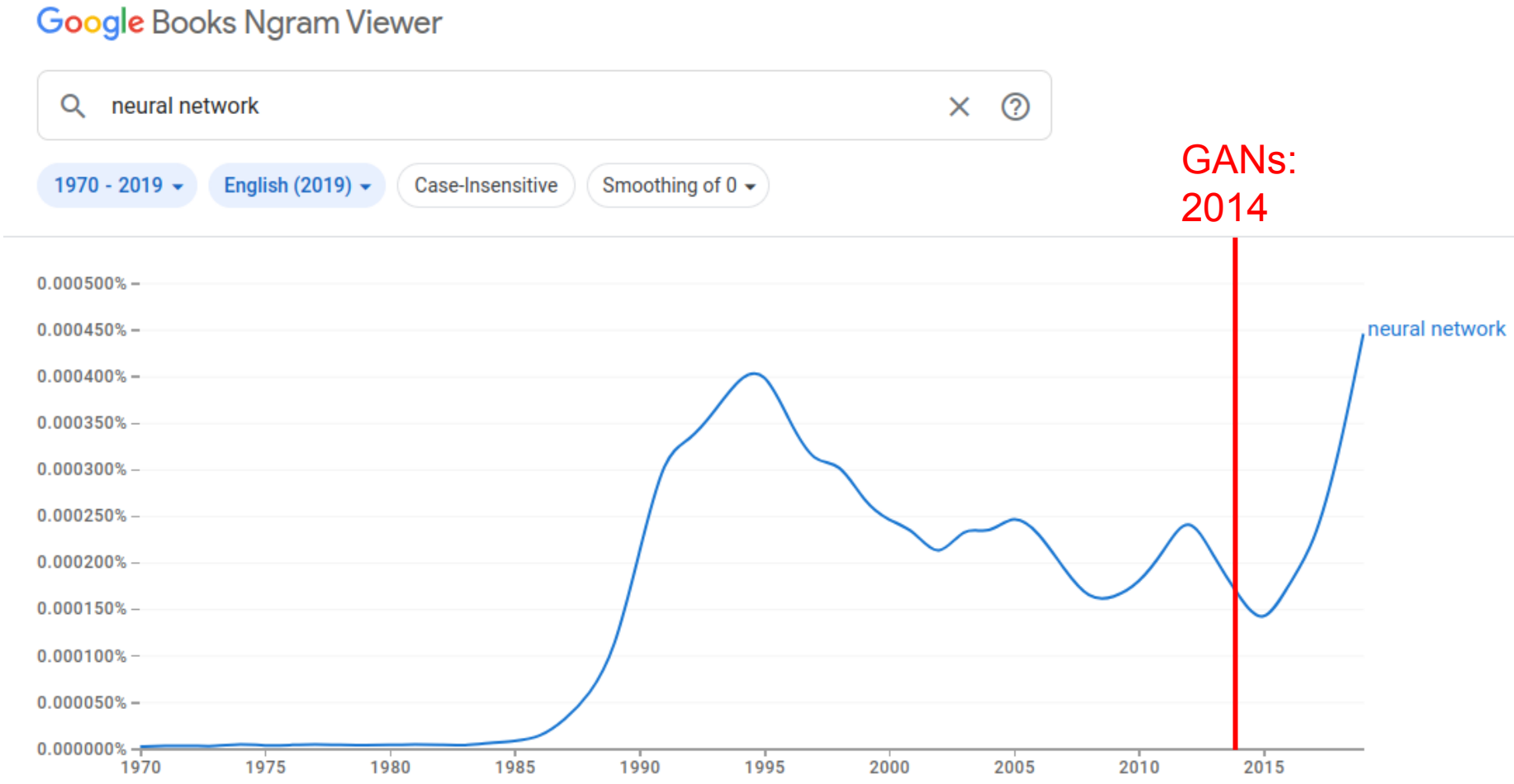
Classifies by modeling **how the data were generated**: there is a term for ***P*(data|class)**.
E.g. **naive Bayes**.

Google Books Ngram Viewer

neural network

1970 - 2019 | English (2019) | Case-Insensitive | Smoothing of 0



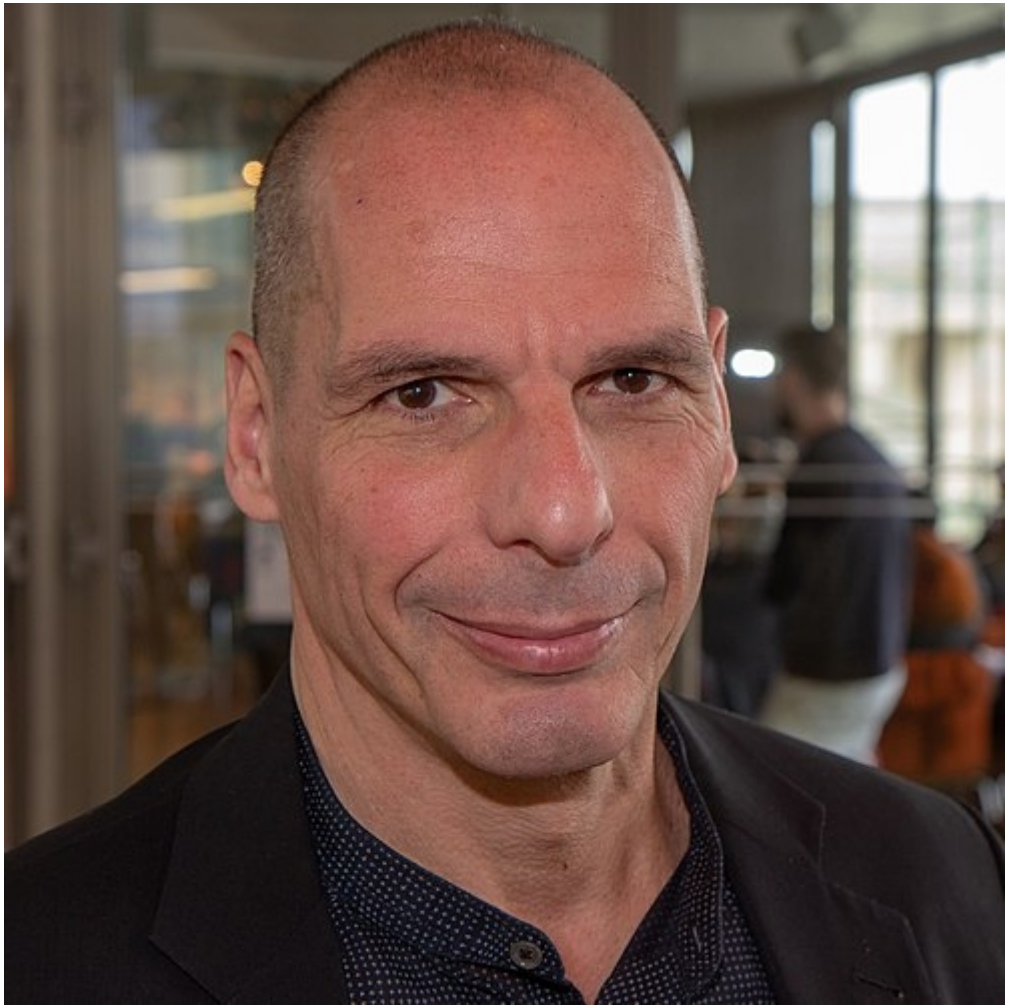

Google Books Ngram Viewer
neural network
1970 - 2019
English (2019)
Case-Insensitive
Smoothing of 0
GANs:
2014
0.000500%
0.000450%
0.000400%
0.000350%
0.000300%
0.000250%
0.000200%
0.000150%
0.000100%
0.000050%
0.000000%
1970
1975
1980
1985
1990
1995
2000
2005
2010
2015
neural network

This person
does not exist

This person
does not exist

This person **does not exist**

Deep fakes, via generative adversarial networks (GANs).



This person
does exist, but...

Yanis Varoufakis

This person
does exist, but...

“I’m watching myself on YouTube saying things I would never say. This is the deepfake menace we must confront.”

Yanis Varoufakis

Generated Data

Discriminator

Real Data

FAKE

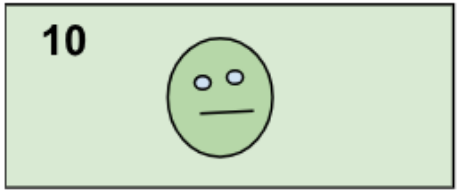
REAL

Generated Data

Discriminator

Real Data

FAKE REAL



FAKE REAL

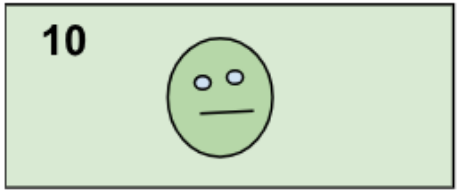
Generated Data

Discriminator

Real Data

FAKE

REAL



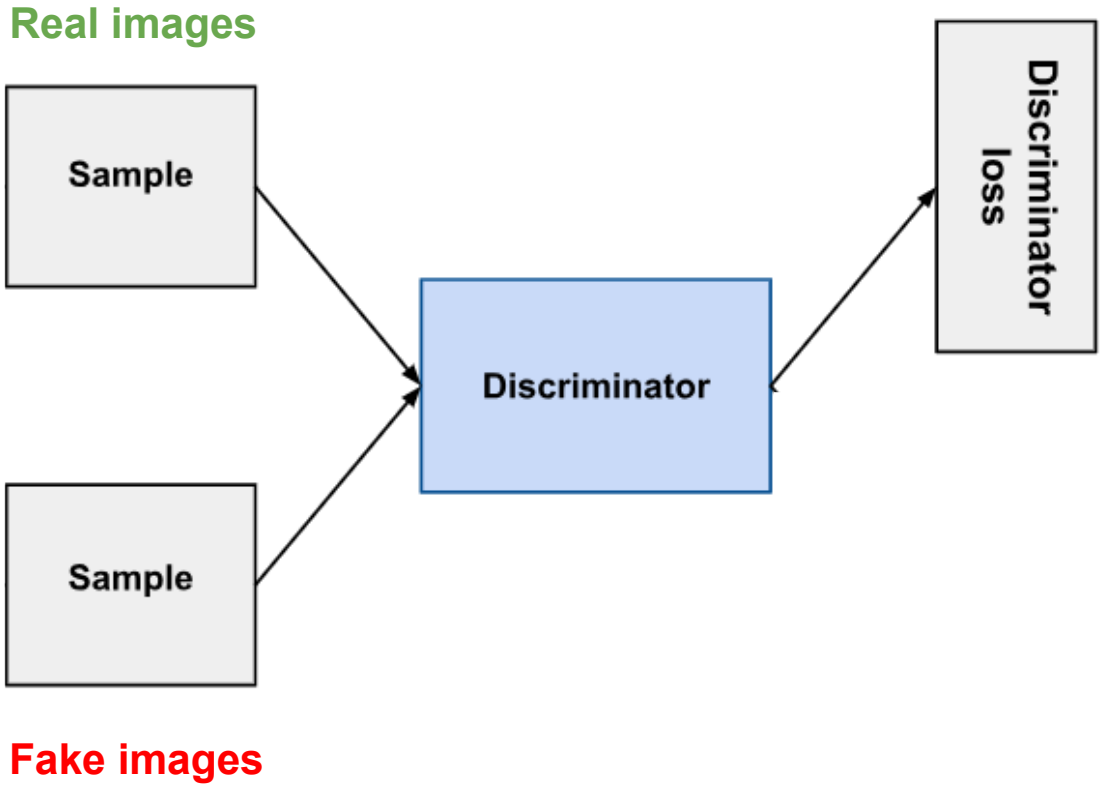
FAKE

REAL

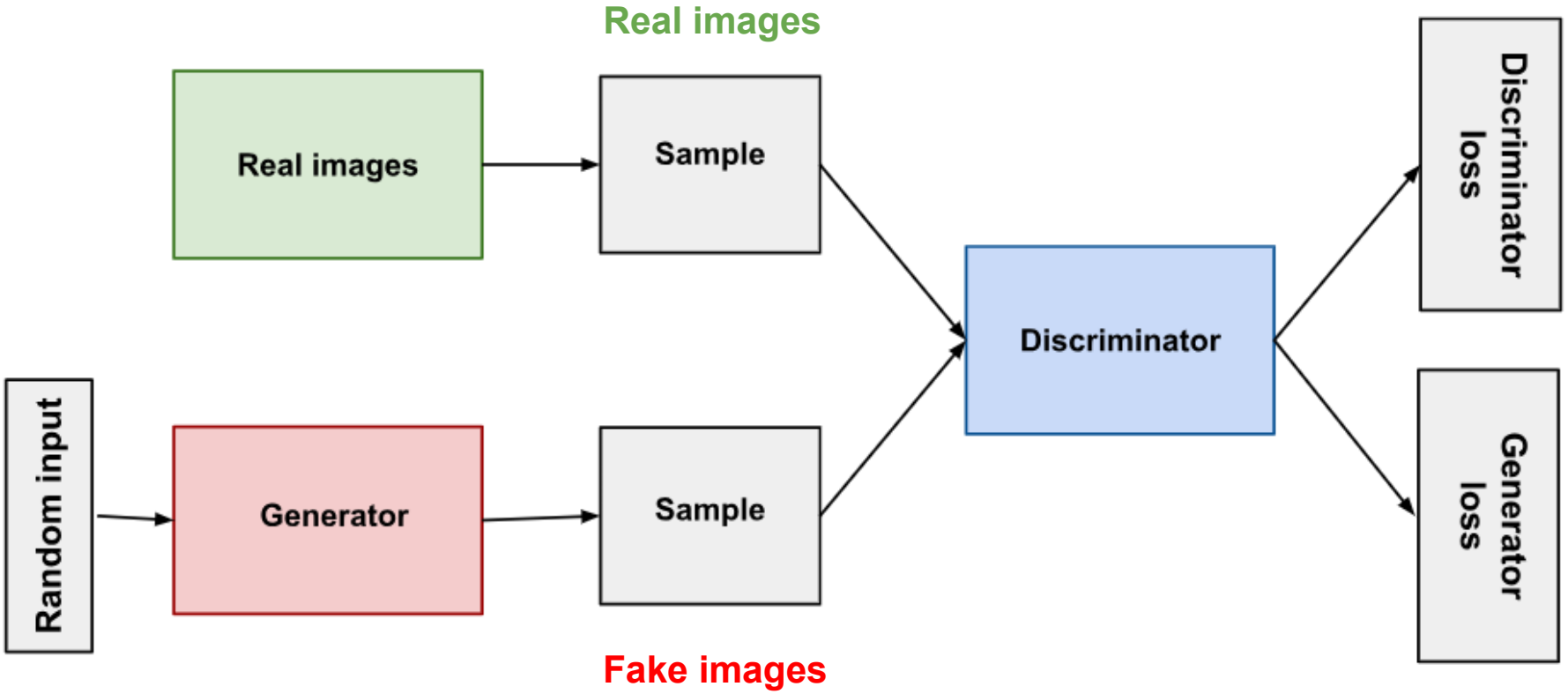
REAL

REAL

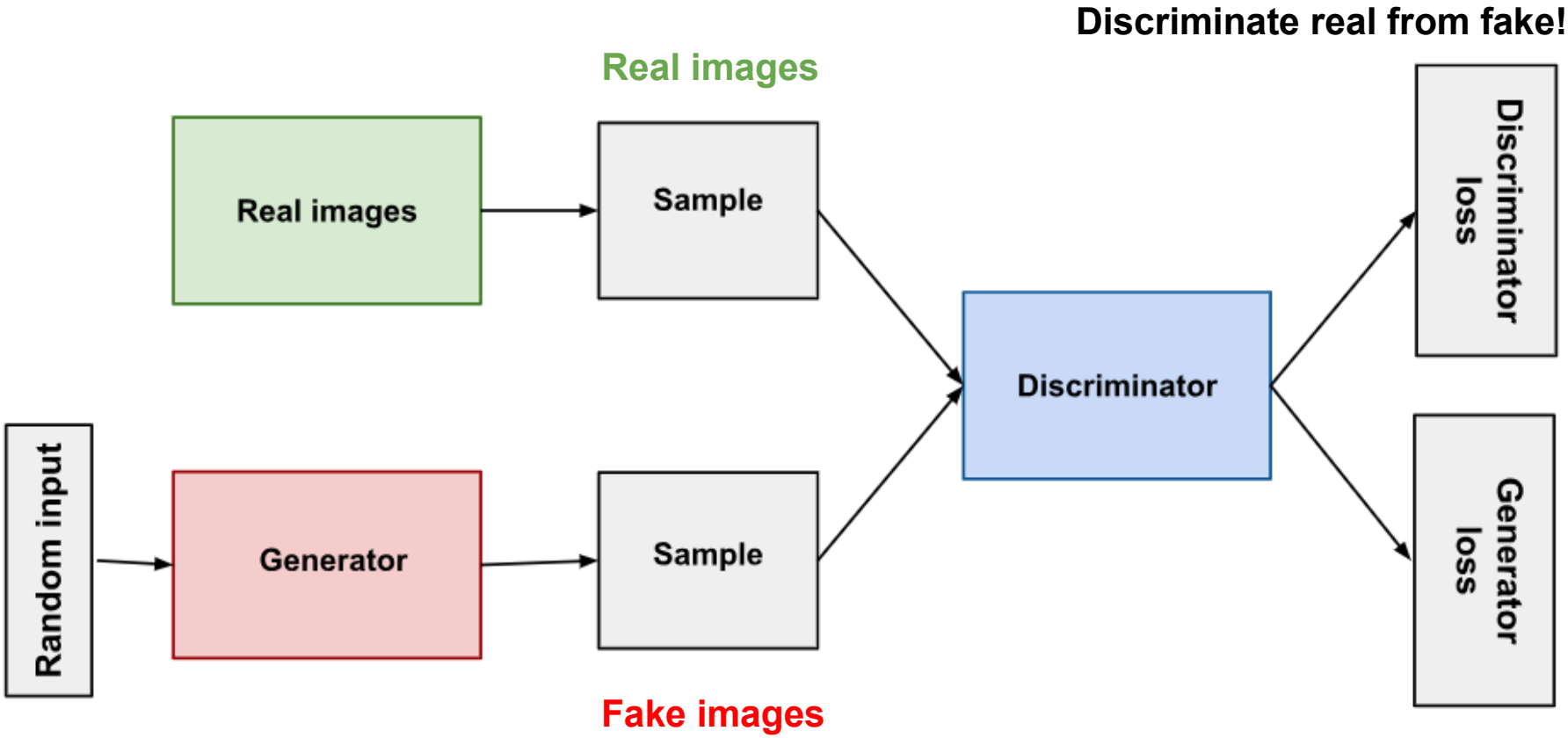
Generative adversarial network



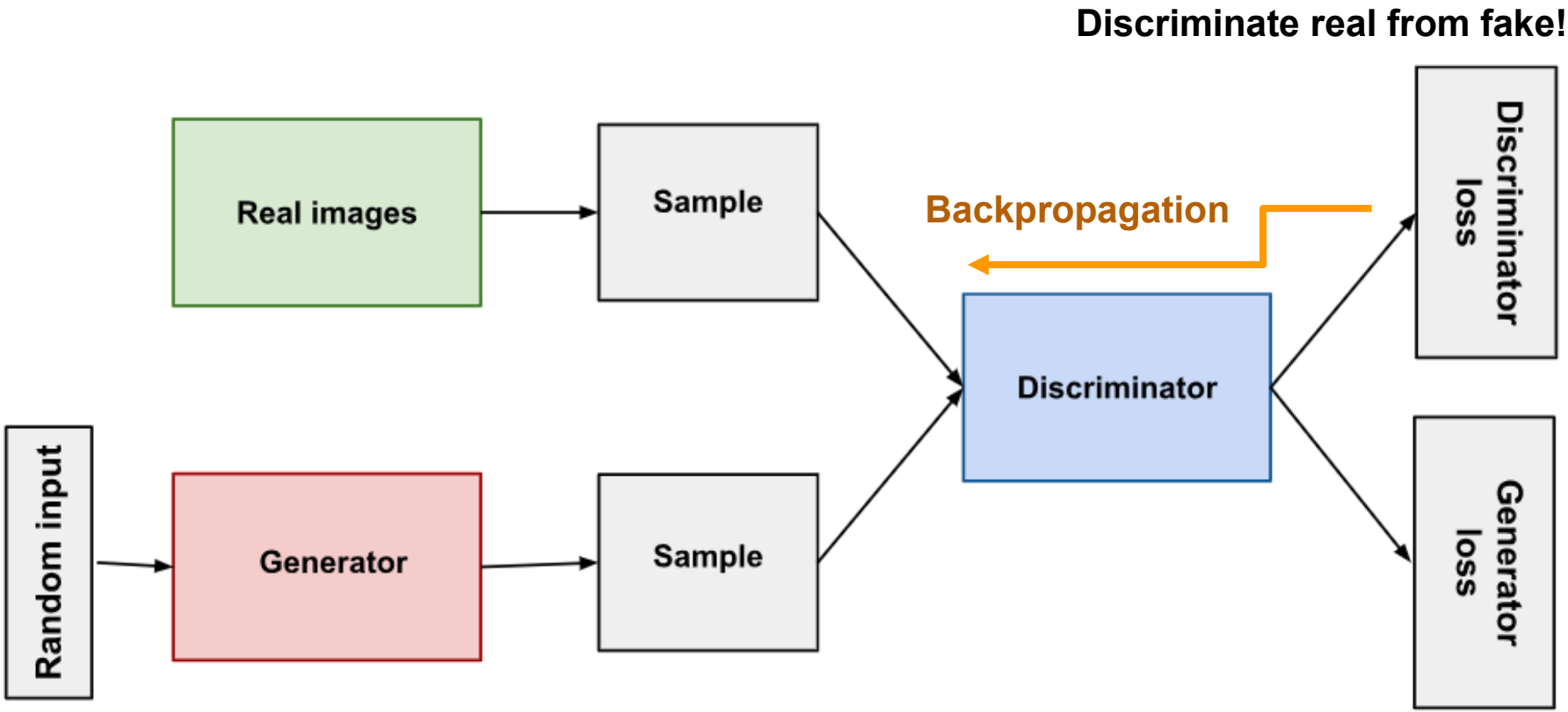
Generative adversarial network



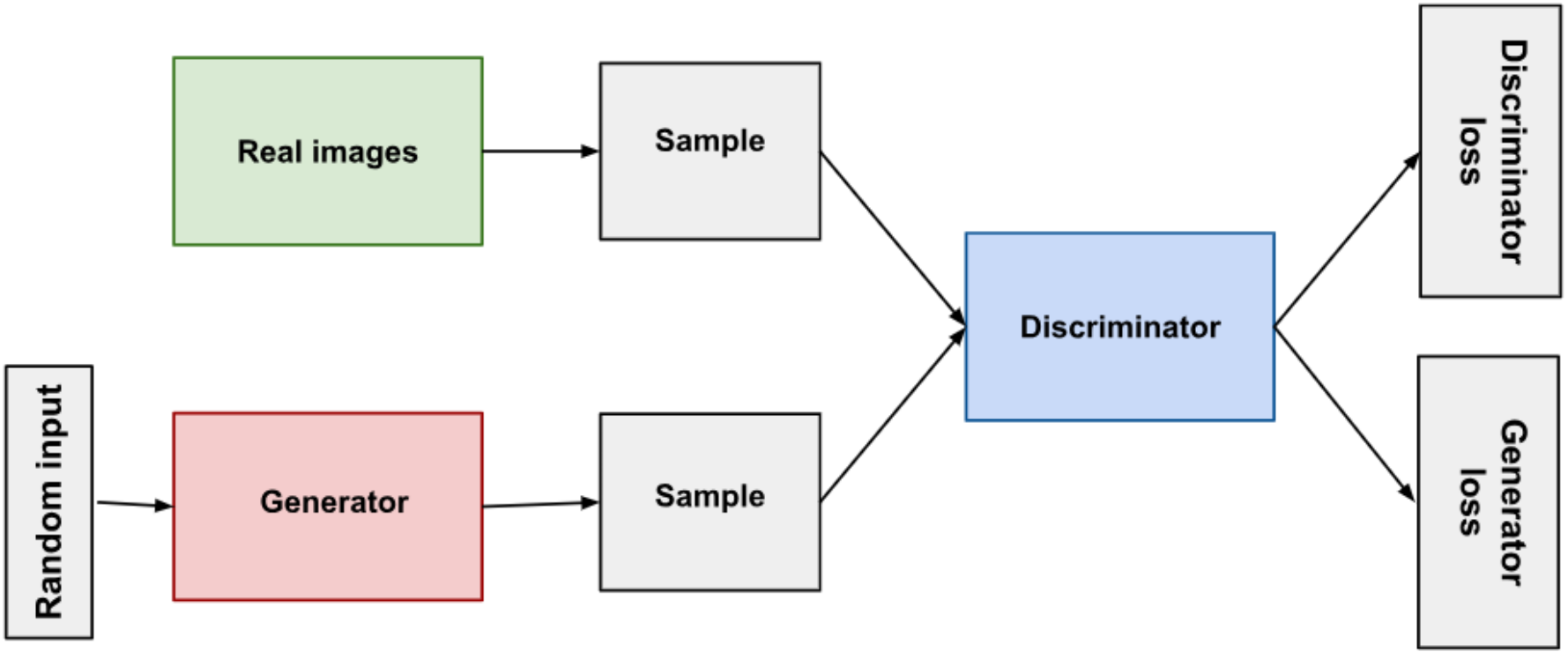
Generative adversarial network



Generative adversarial network

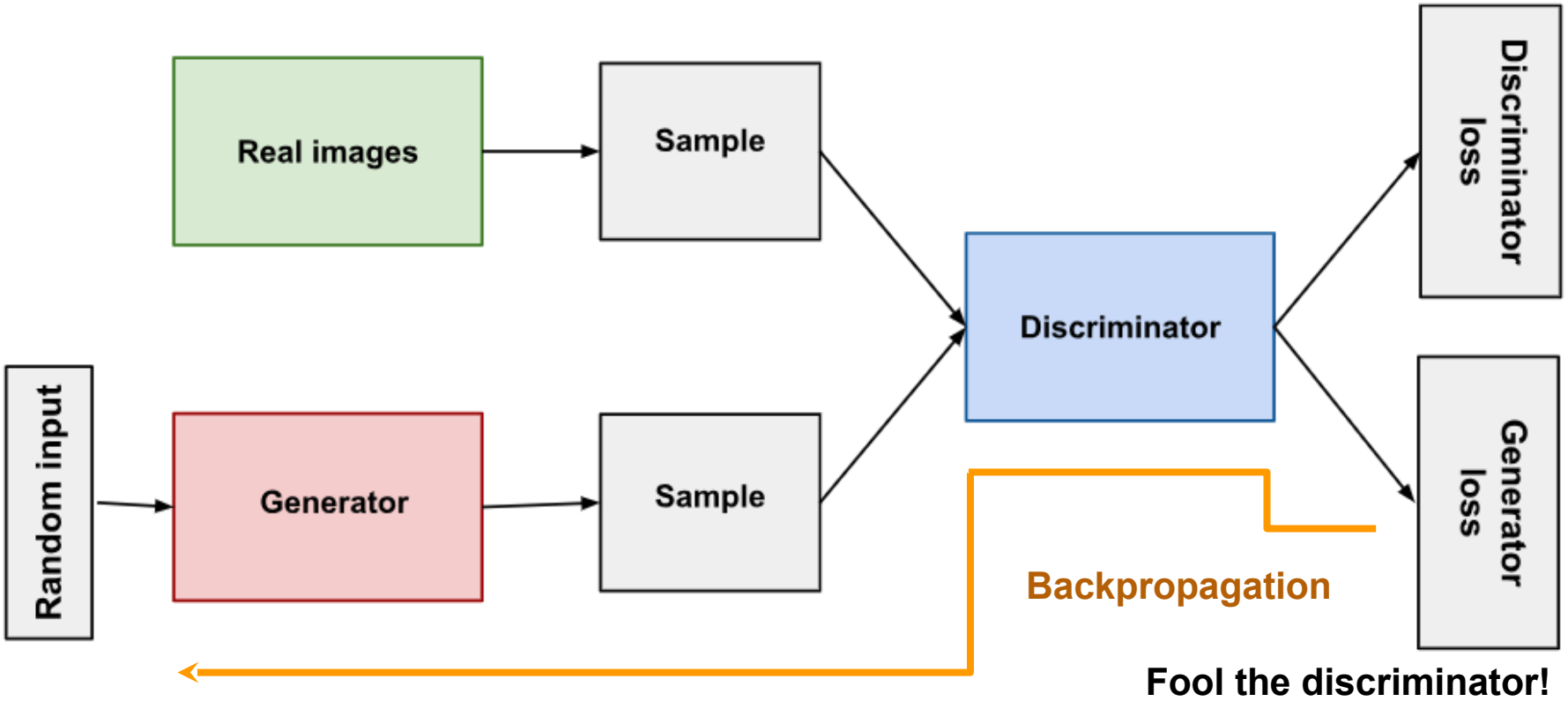


Generative adversarial network



Fool the discriminator!

Generative adversarial network



Generative adversarial network

- An **evolutionary arms race**: discriminator and generator are **adversaries**, each learning to try to outsmart the other.

Generative adversarial network

- An **evolutionary arms race**: discriminator and generator are **adversaries**, each learning to try to outsmart the other.
- GANs, introduced in 2014, were a major advance in generative AI.

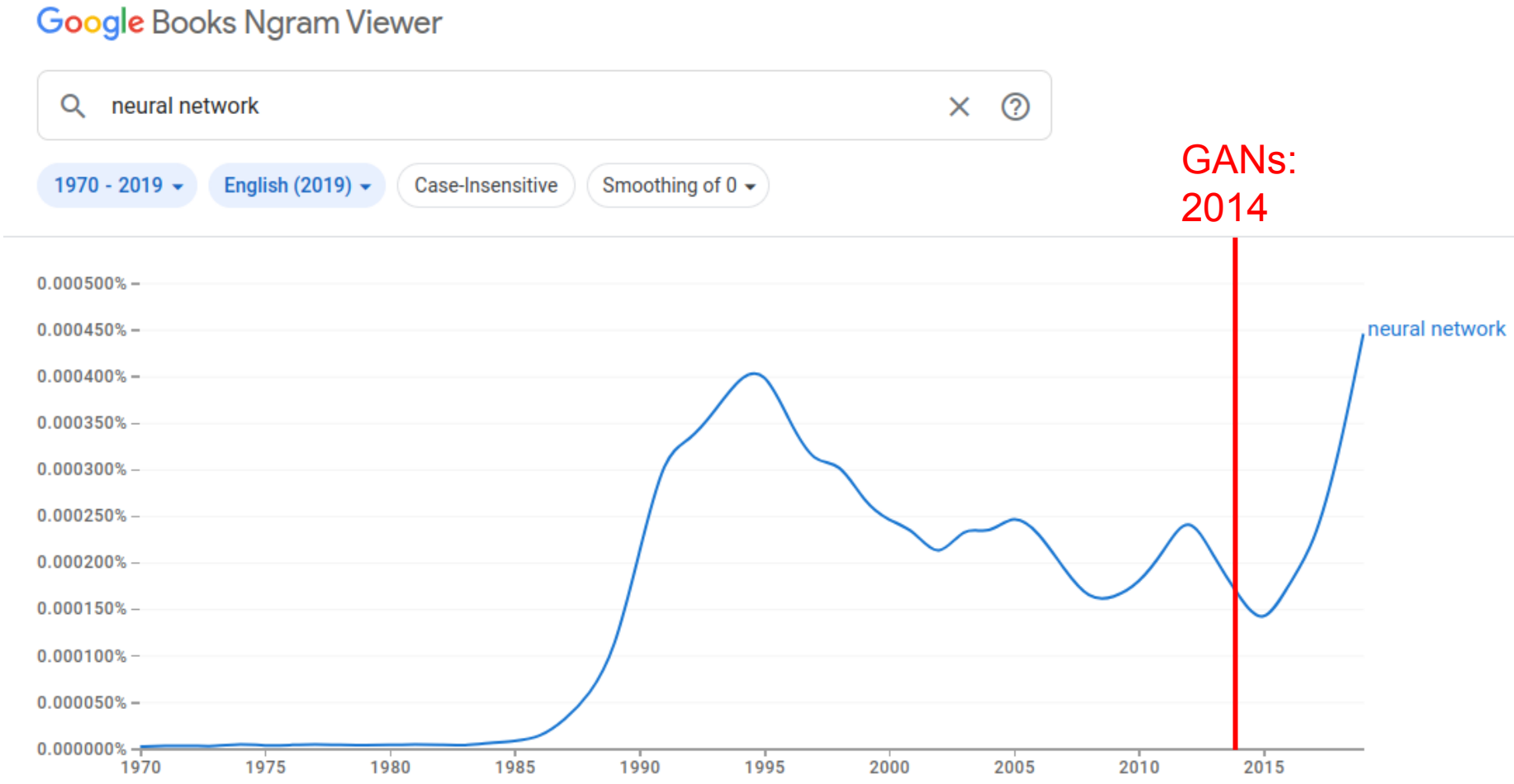
Generative adversarial network

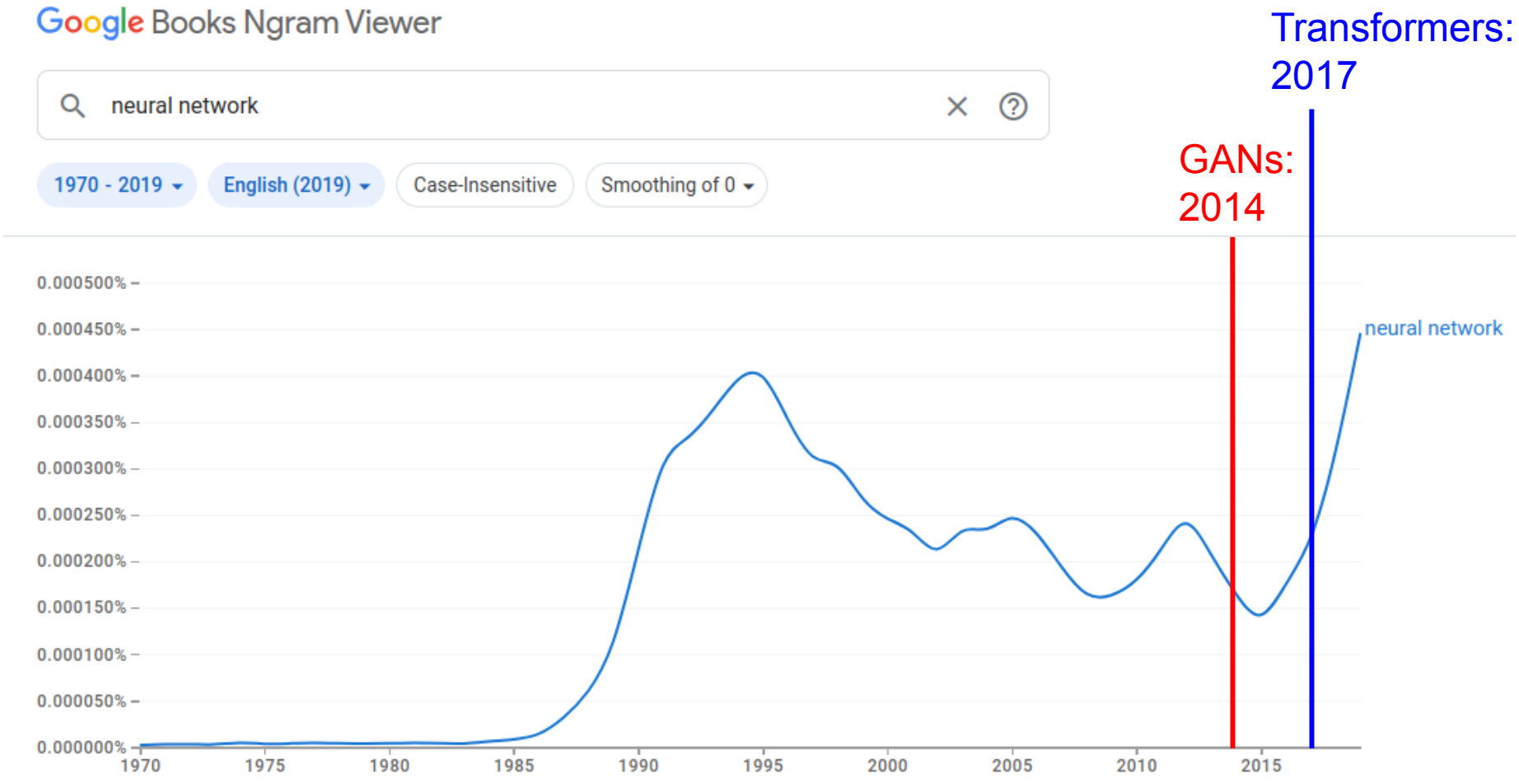
- An **evolutionary arms race**: discriminator and generator are **adversaries**, each learning to try to outsmart the other.
- GANs, introduced in 2014, were a major advance in generative AI.
- Often applied to image generation but…

Gasper Begus at Berkeley is a proponent of GANs as applied to **audio speech data**, for the purpose of modeling phonological processes, word learning, etc.

Overview

1. Neurons and logic
2. Perceptrons and learning
3. Generative AI
4. Transformers and large language models


Google Books Ngram Viewer
neural network
1970 - 2019
English (2019)
Case-Insensitive
Smoothing of 0
GANs:
2014
0.000500%
0.000450%
0.000400%
0.000350%
0.000300%
0.000250%
0.000200%
0.000150%
0.000100%
0.000050%
0.000000%
1970
1975
1980
1985
1990
1995
2000
2005
2010
2015
neural network


Google Books Ngram Viewer
neural network
1970 - 2019
English (2019)
Case-Insensitive
Smoothing of 0
Transformers: 2017
GANs: 2014
neural network
0.000500%
0.000450%
0.000400%
0.000350%
0.000300%
0.000250%
0.000200%
0.000150%
0.000100%
0.000050%
0.000000%
1970
1975
1980
1985
1990
1995
2000
2005
2010
2015

Also generative

- Produces novel yet plausible **text**, rather than images.

Also generative

- Produces novel yet plausible **text**, rather than images.
- ChatGPT et al. have arguably done to the **written word** what GANs have done to photos.

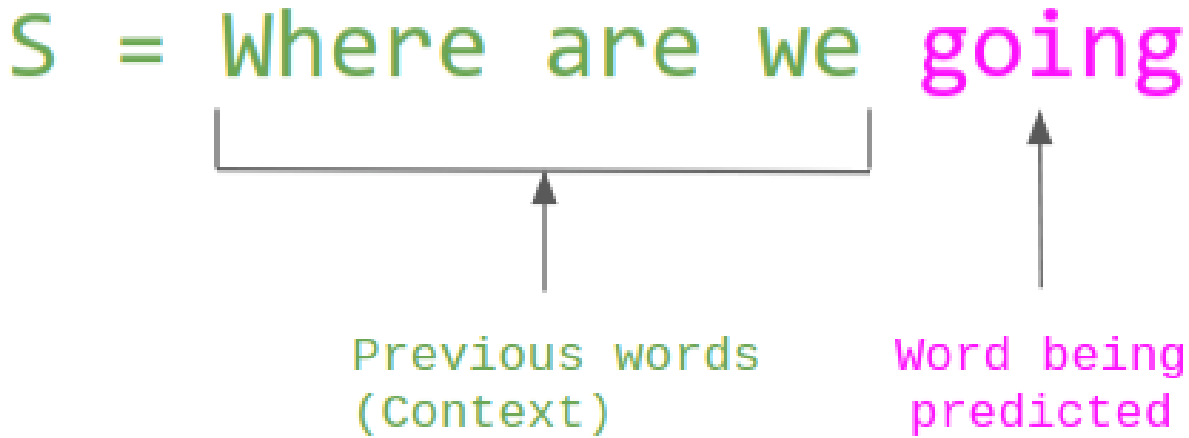
Also generative

- Produces novel yet plausible **text**, rather than images.
- ChatGPT et al. have arguably done to the **written word** what GANs have done to photos.
- Tremendous social impact – and attracting the attention of **linguists** and **cognitive scientists**.

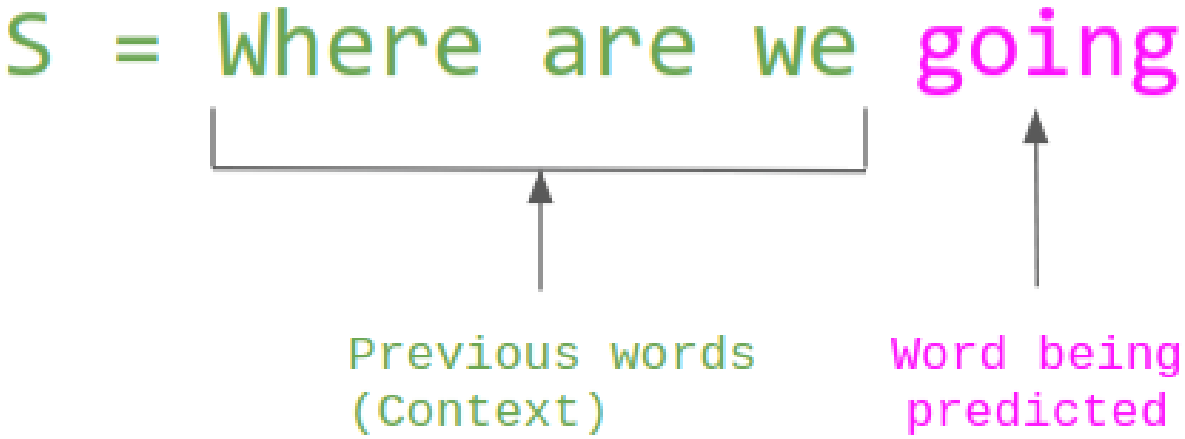
Also generative

- Produces novel yet plausible **text**, rather than images.
- ChatGPT et al. have arguably done to the **written word** what GANs have done to photos.
- Tremendous social impact – and attracting the attention of **linguists** and **cognitive scientists**.
- Goal: **understand the basics**, largely based on Jurafsky & Martin reading — then **discuss**.

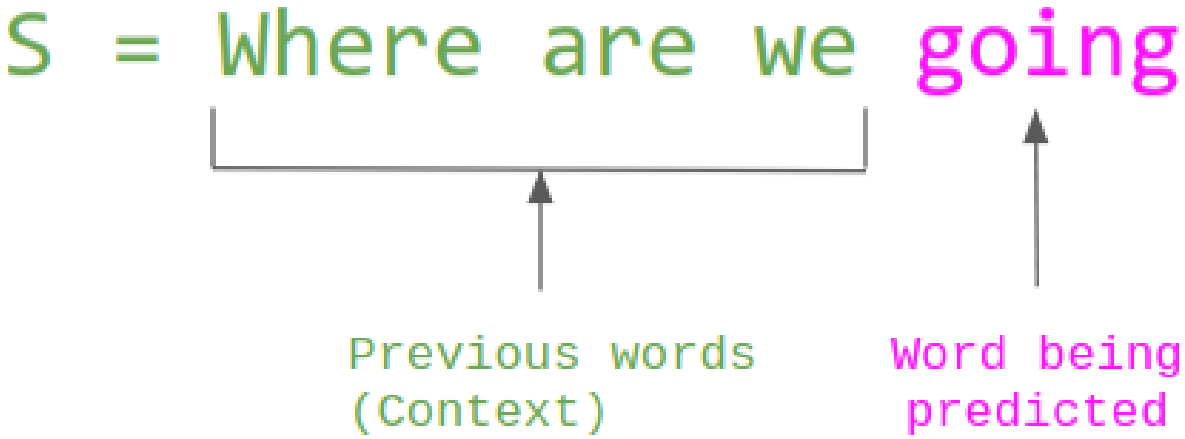

OH MY GOD, THE VOLCANO IS ERU_
ERUDITE?


S = Where are we going
Previous words
(Context)
Word being
predicted

Language modeling



Language modeling



Self-supervised learning

Language modeling

- The task of **predicting the next word** (or character, or phoneme, or other symbol) in text, given preceding text.

Language modeling

- The task of **predicting the next word** (or character, or phoneme, or other symbol) in text, given preceding text.
- **N-gram models** are a simple example.

Language modeling

- The task of **predicting the next word** (or character, or phoneme, or other symbol) in text, given preceding text.
- **N-gram models** are a simple example.
- **Simple recurrent networks** are another.

Language modeling

- The task of **predicting the next word** (or character, or phoneme, or other symbol) in text, given preceding text.
- **N-gram models** are a simple example.
- **Simple recurrent networks** are another.
- Large language models (LLMs) such as **transformers** are a more powerful example of **the same thing**.

Recurrent networks

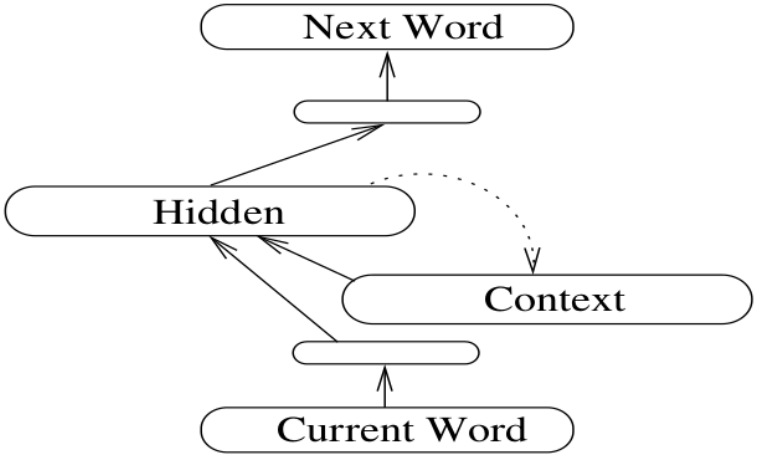


Figure 1: An SRN. Solid lines represent full connectivity; the dashed line indicates unit-to-unit connections. The unlabeled layers are reduction layers.

Recurrent networks

Vanishing gradient: Gradient gets smaller as we trickle back through the network, and through **time**.

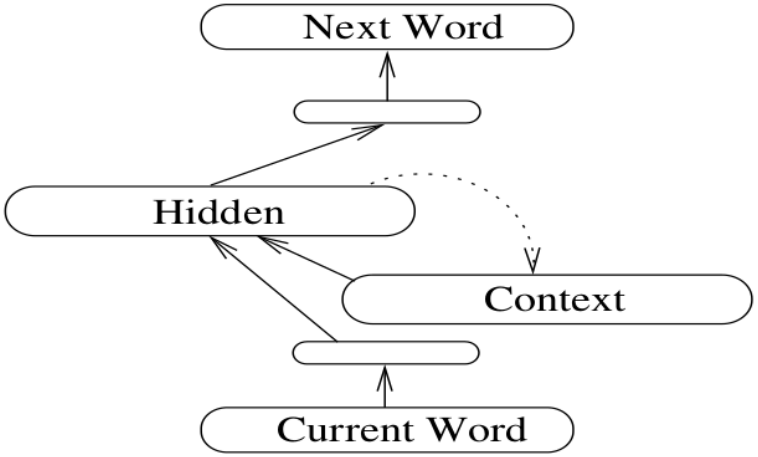


Figure 1: An SRN. Solid lines represent full connectivity; the dashed line indicates unit-to-unit connections. The unlabeled layers are reduction layers.

Recurrent networks

Vanishing gradient: Gradient gets smaller as we trickle back through the network, and through **time**.

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

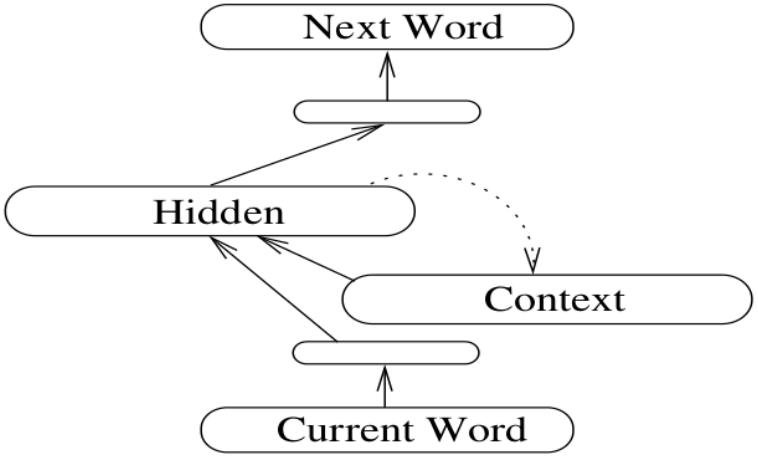


Figure 1: An SRN. Solid lines represent full connectivity; the dashed line indicates unit-to-unit connections. The unlabeled layers are reduction layers.

Vanishing gradient: Gradient gets smaller as we trickle back through the network, and through **time**.

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

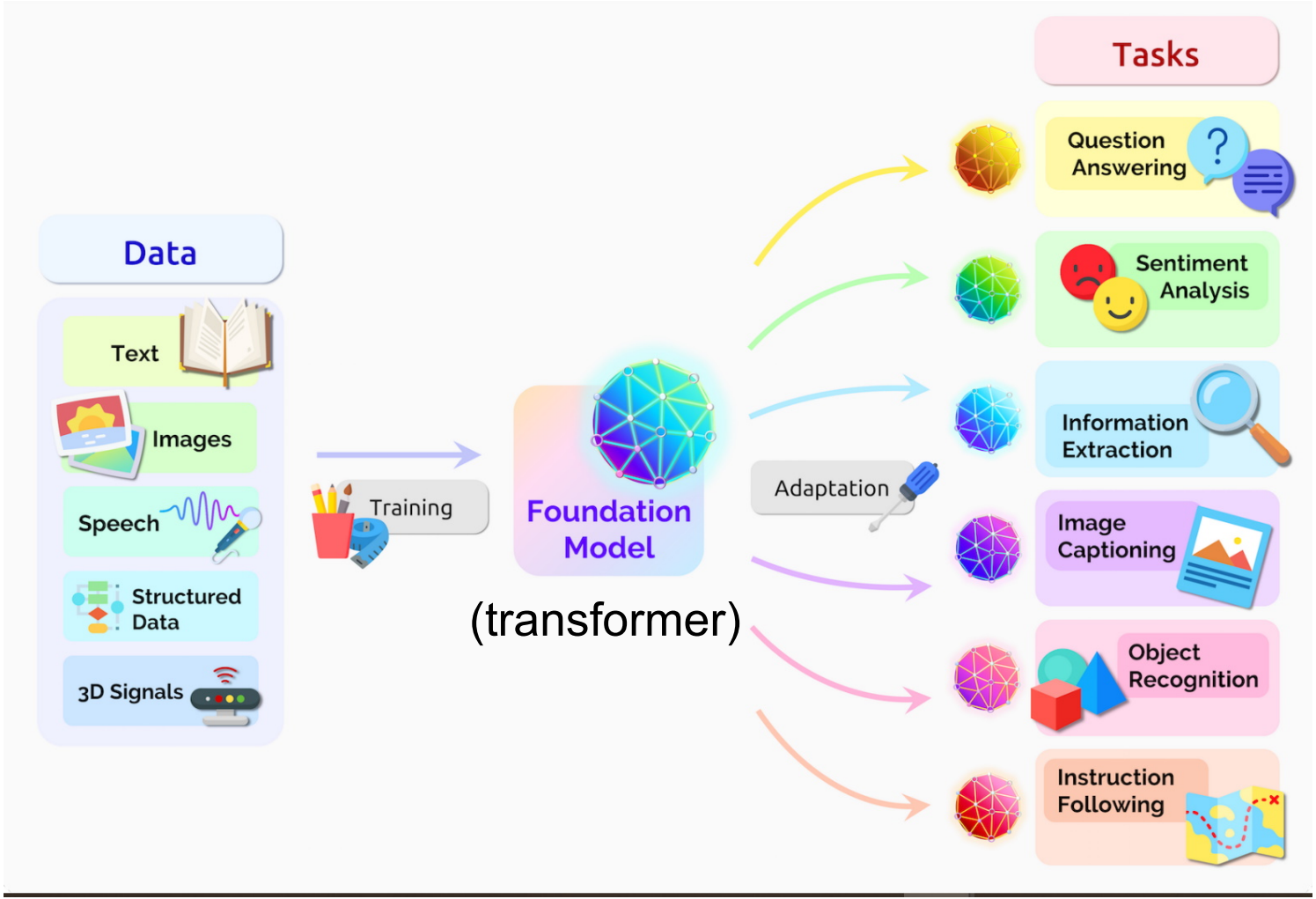
Transformers avoid this problem by having **all words available at once,** in parallel, and learning to **attend** to relevant words. So there is no trickling back through time.

What is a transformer model?

What is a transformer model?

- A transformer model is a neural network that **learns context** and thus **meaning** by **tracking relationships** in sequential data like the words in this sentence.

From blogs.nvidia.com


Data
Text
Images
Speech
Structured Data
3D Signals
Training
Foundation Model
(transformer)
Adaptation
Tasks
Question Answering
Sentiment Analysis
Information Extraction
Image Captioning
Object Recognition
Instruction Following

Some motivating examples

- She poured water from the pitcher to the cup until it was full.

Some motivating examples

- She poured water from the pitcher to the cup until it was full.

Some motivating examples

- She poured water from the pitcher to the cup until it was full.

Some motivating examples

- She poured water from the pitcher to the cup until it was full.
- She poured water from the pitcher to the cup until it was empty.

Some motivating examples

- She poured water from the pitcher to the cup until it was full.
- She poured water from the pitcher to the cup until it was empty.

Some motivating examples

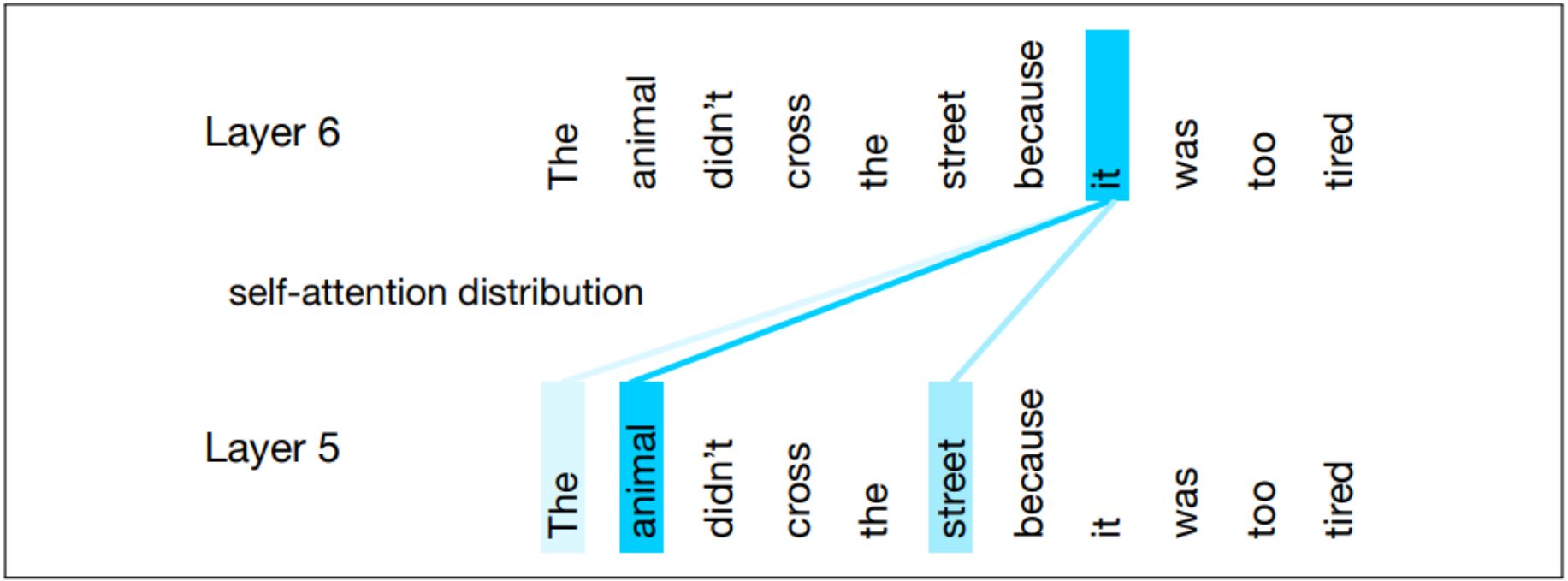
- She poured water from the pitcher to the cup until it was full.
- She poured water from the pitcher to the cup until it was empty.

Some motivating examples

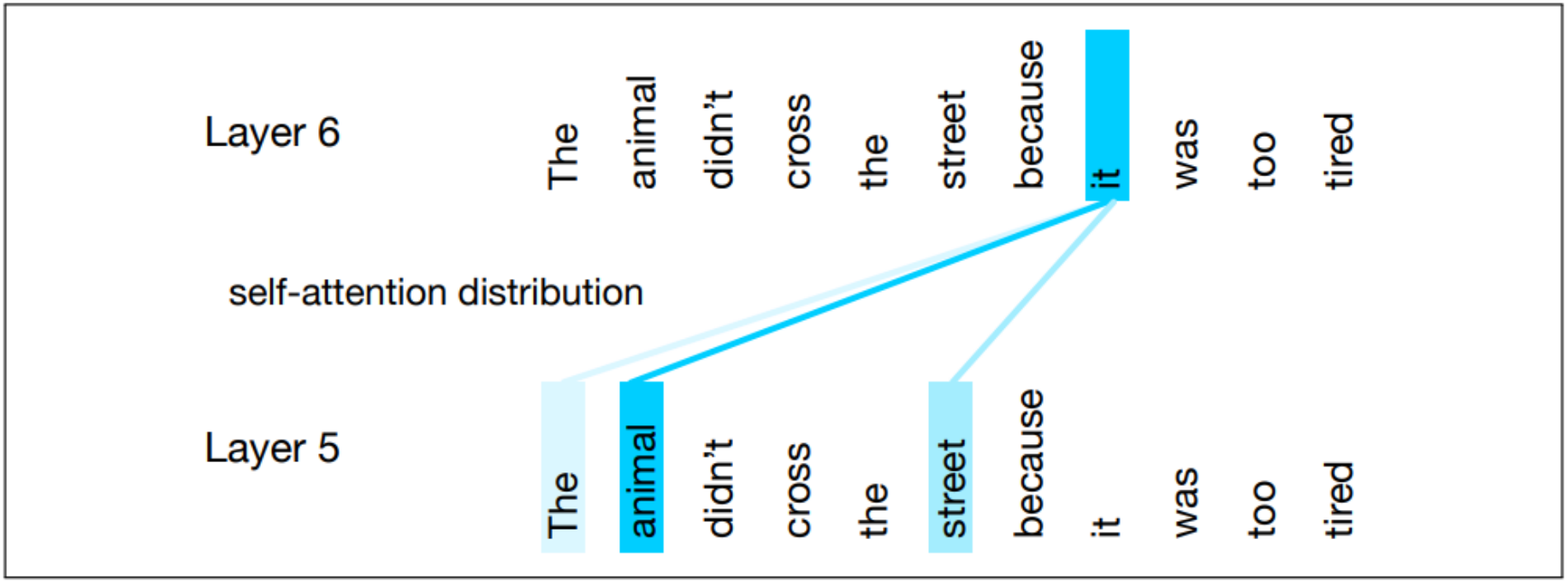
- She poured water from the pitcher to the cup until it was full.
- She poured water from the pitcher to the cup until it was empty.

“We need a mechanism that can look broadly in the context to help compute representations for words” (J&M): **self-attention**

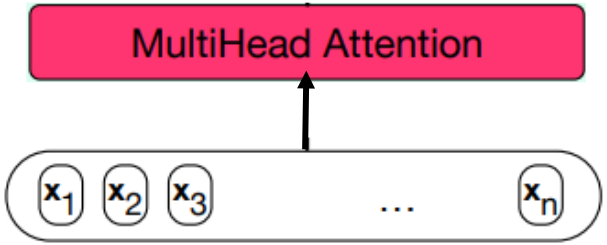
Self-attention weight distribution

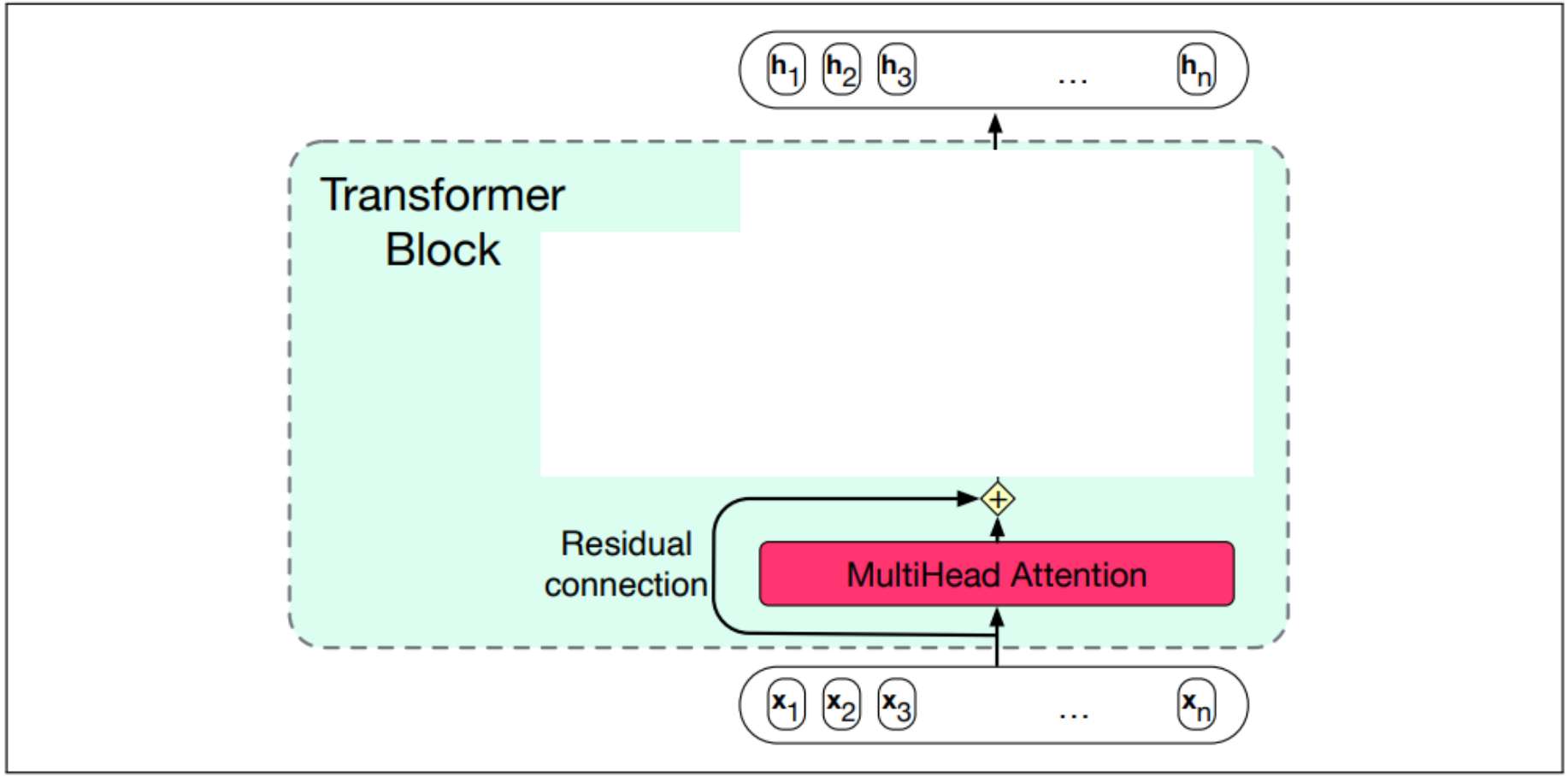


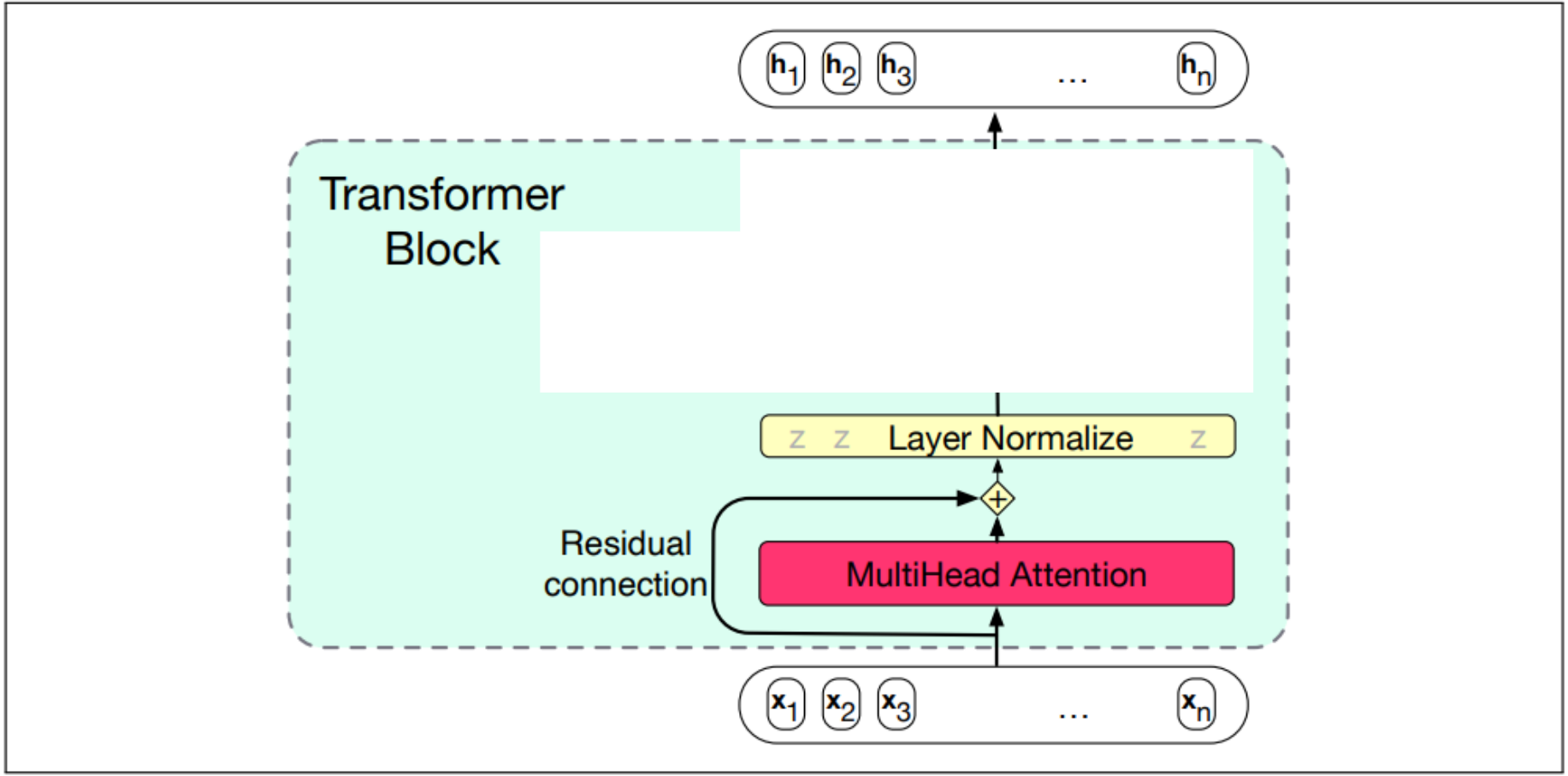
Self-attention weight distribution

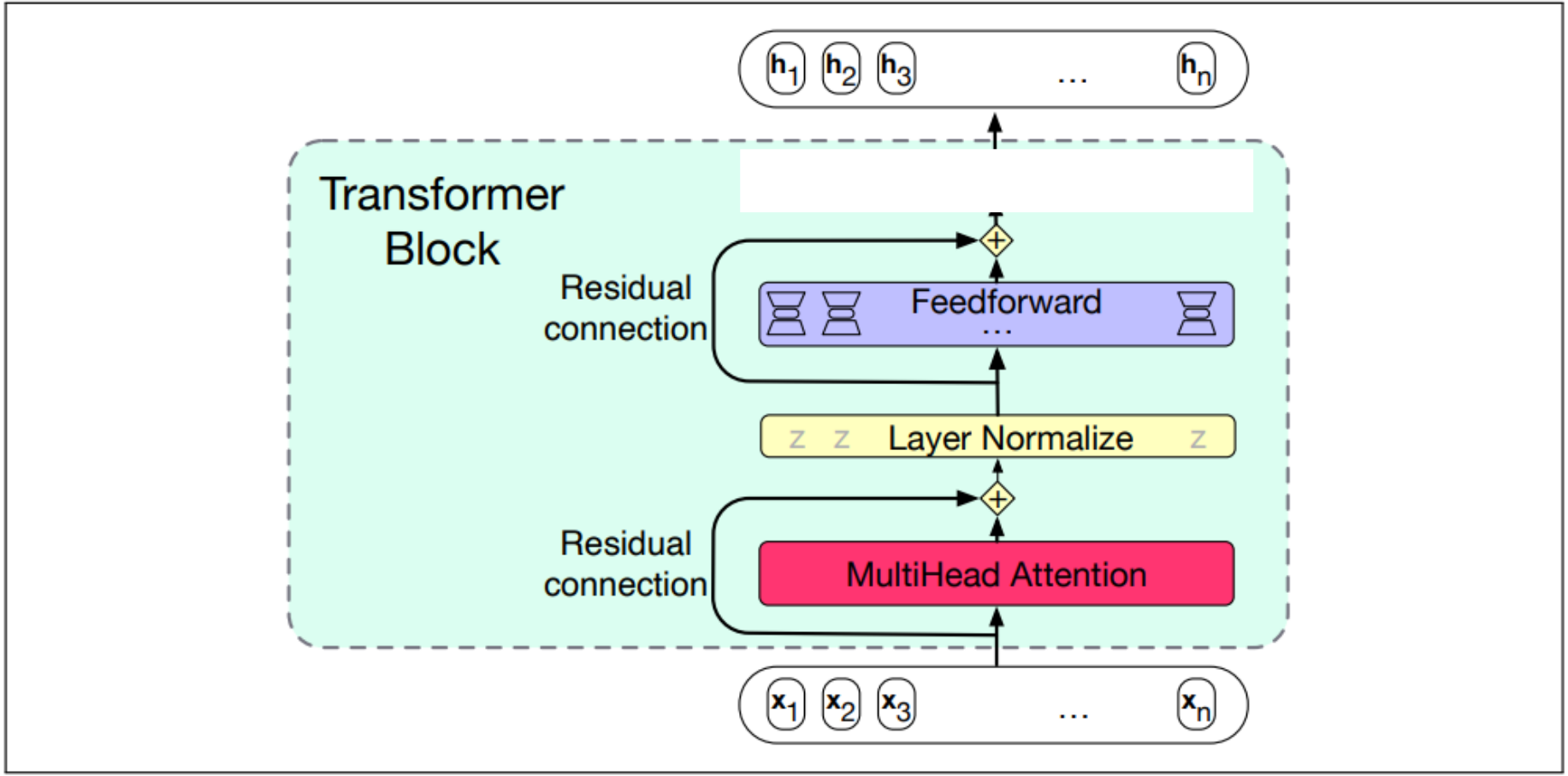


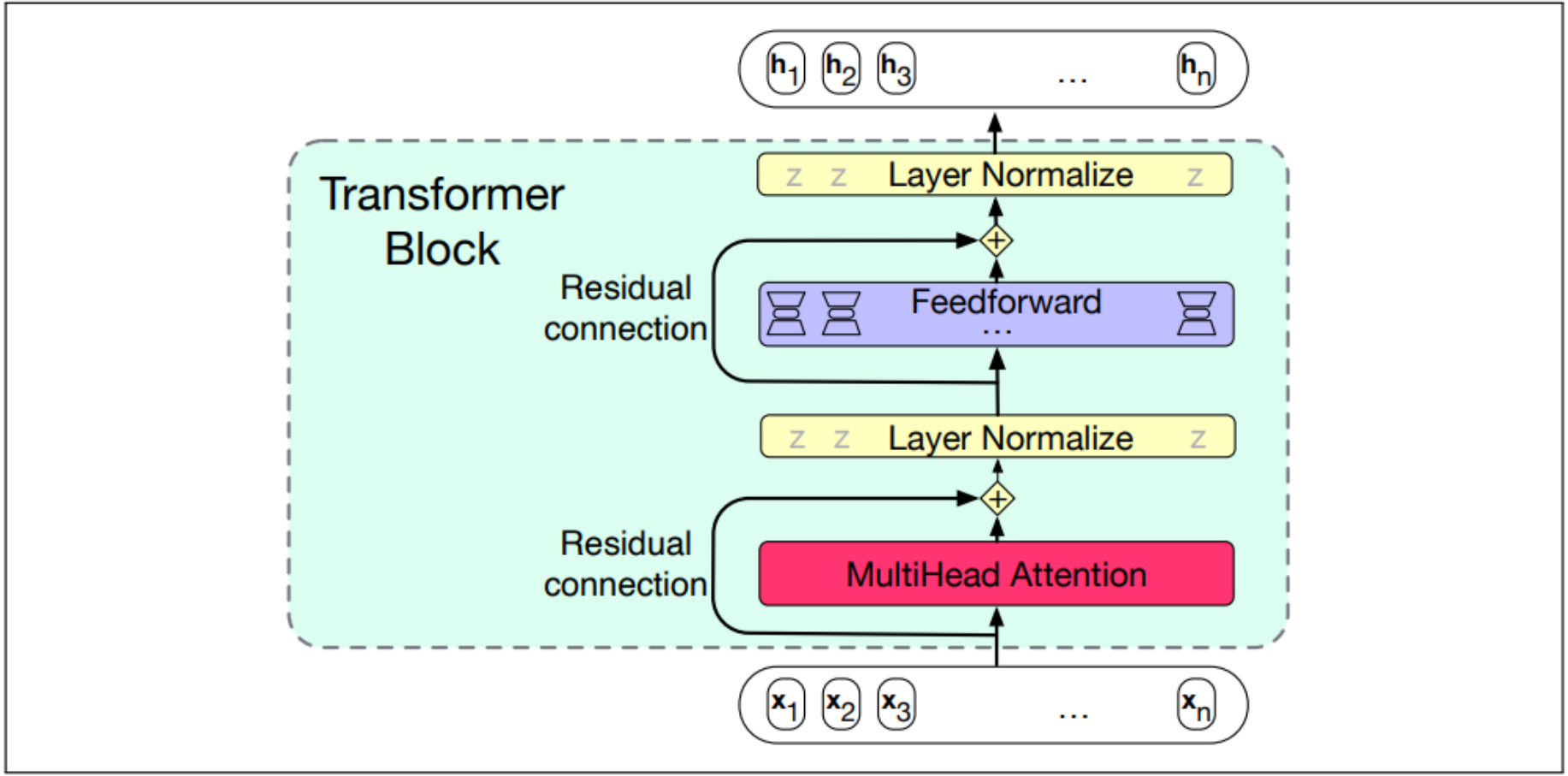
This attention distribution is **learned**, from text.


MultiHead Attention
x1
x2
x3
...
xn

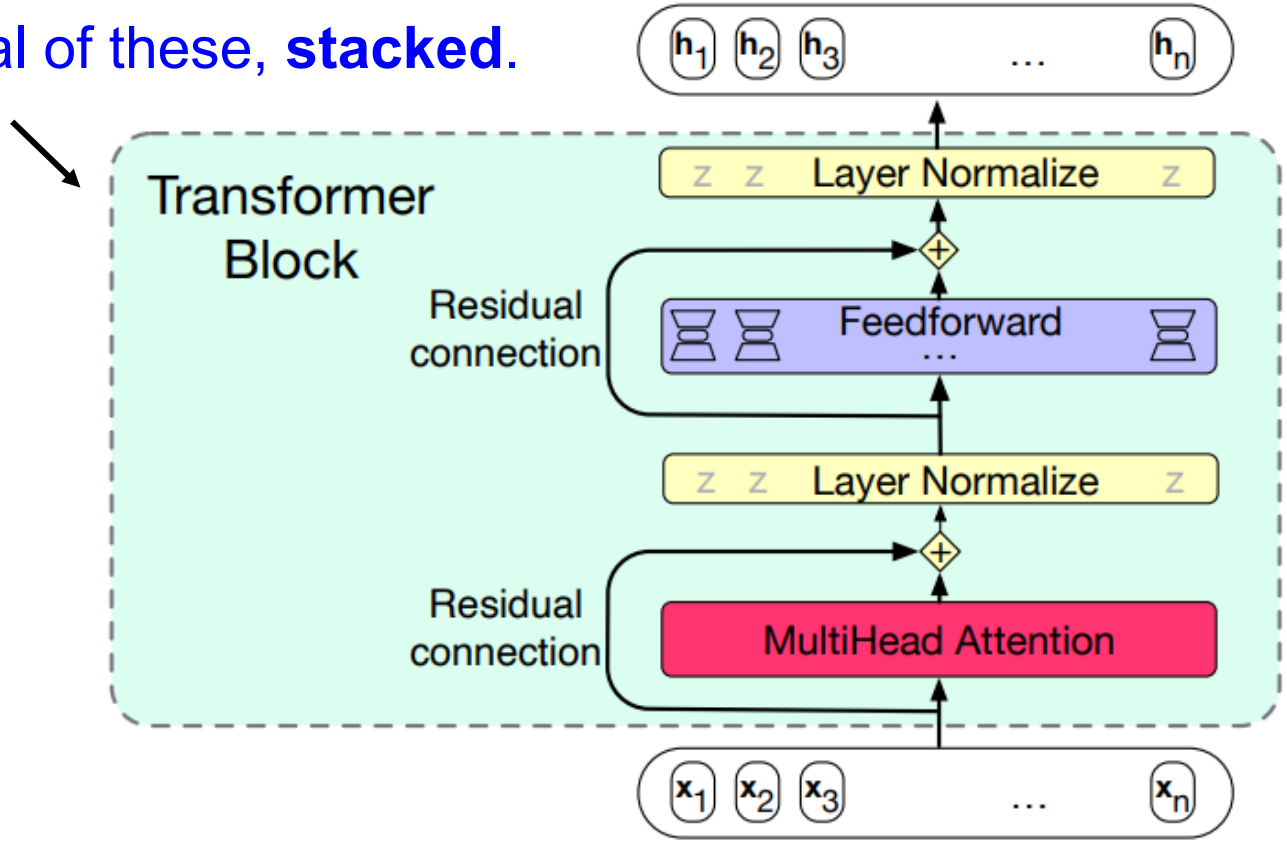

h_1 h_2 h_3 ... h_n
Transformer Block
Residual connection
MultiHead Attention
x_1 x_2 x_3 ... x_n


h1 h2 h3 ... hn
Transformer Block
z z Layer Normalize z
+
Residual connection
MultiHead Attention
x1 x2 x3 ... xn

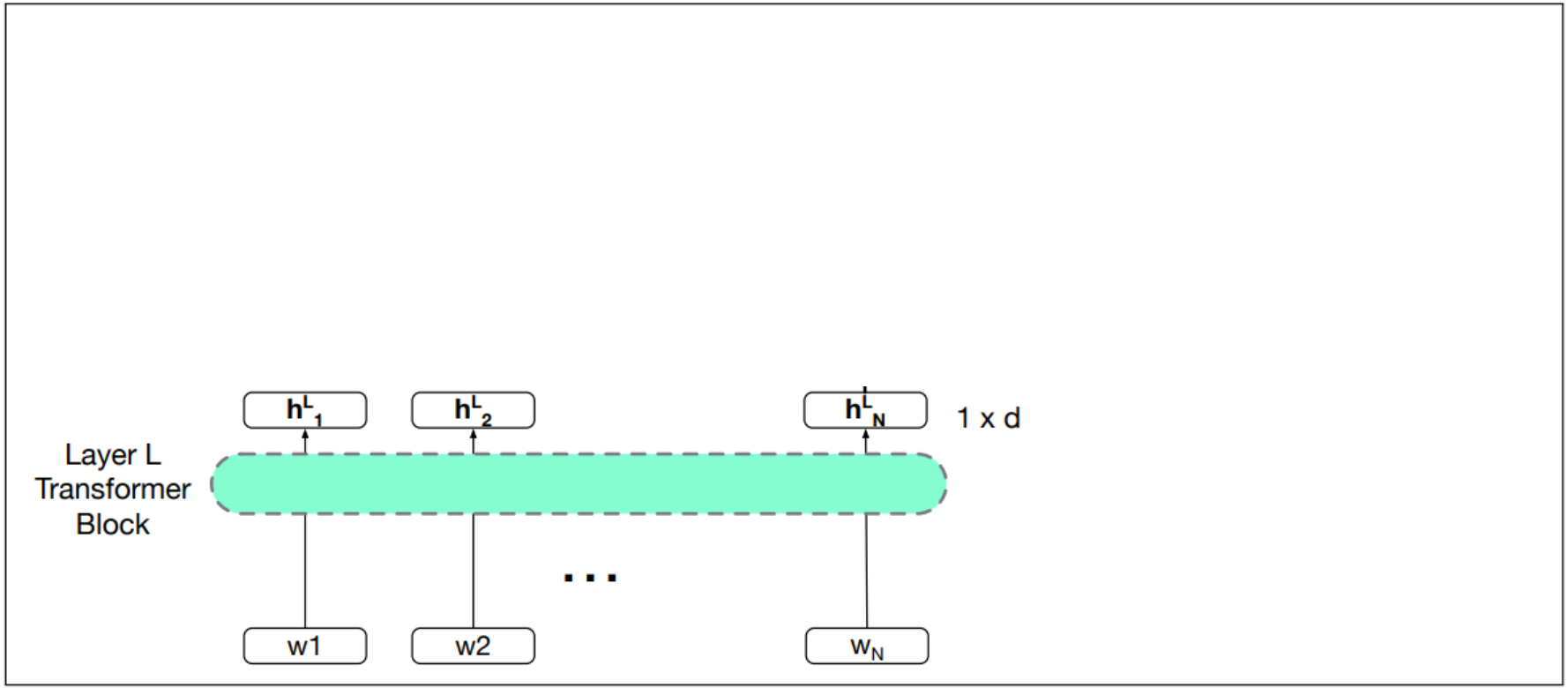

h1 h2 h3 … hn
Transformer Block
Residual connection
Feedforward
…
Layer Normalize
Residual connection
MultiHead Attention
x1 x2 x3 … xn


h1 h2 h3 … hn
Transformer Block
z z Layer Normalize z
Residual connection
Feedforward …
z z Layer Normalize z
Residual connection
MultiHead Attention
x1 x2 x3 … xn

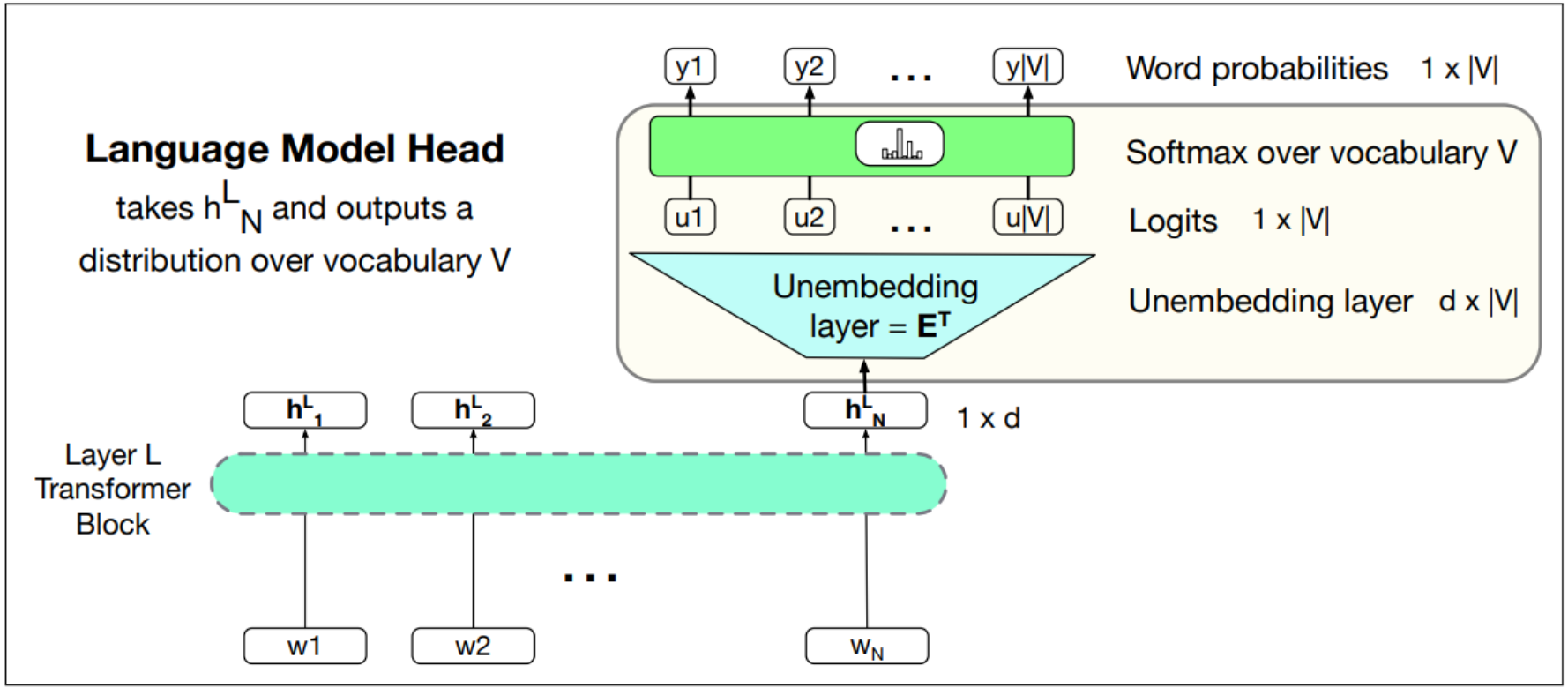
Several of these, **stacked**.



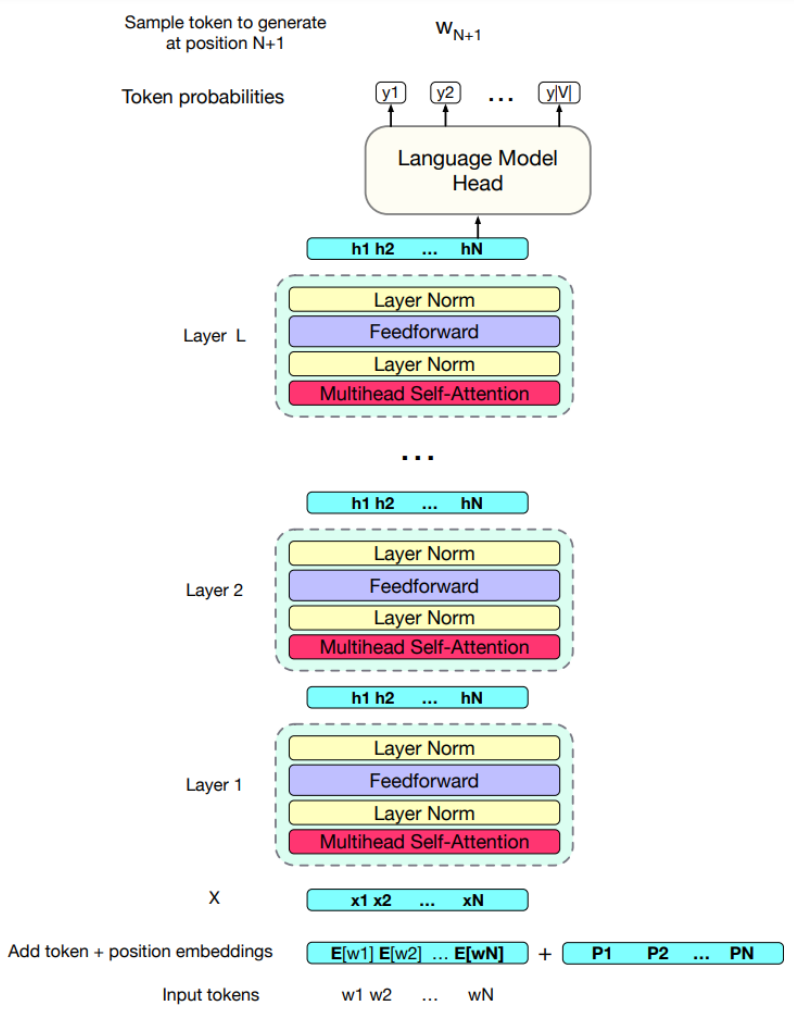
Producing word output from a stack of transformer blocks



Producing word output from a stack of transformer blocks

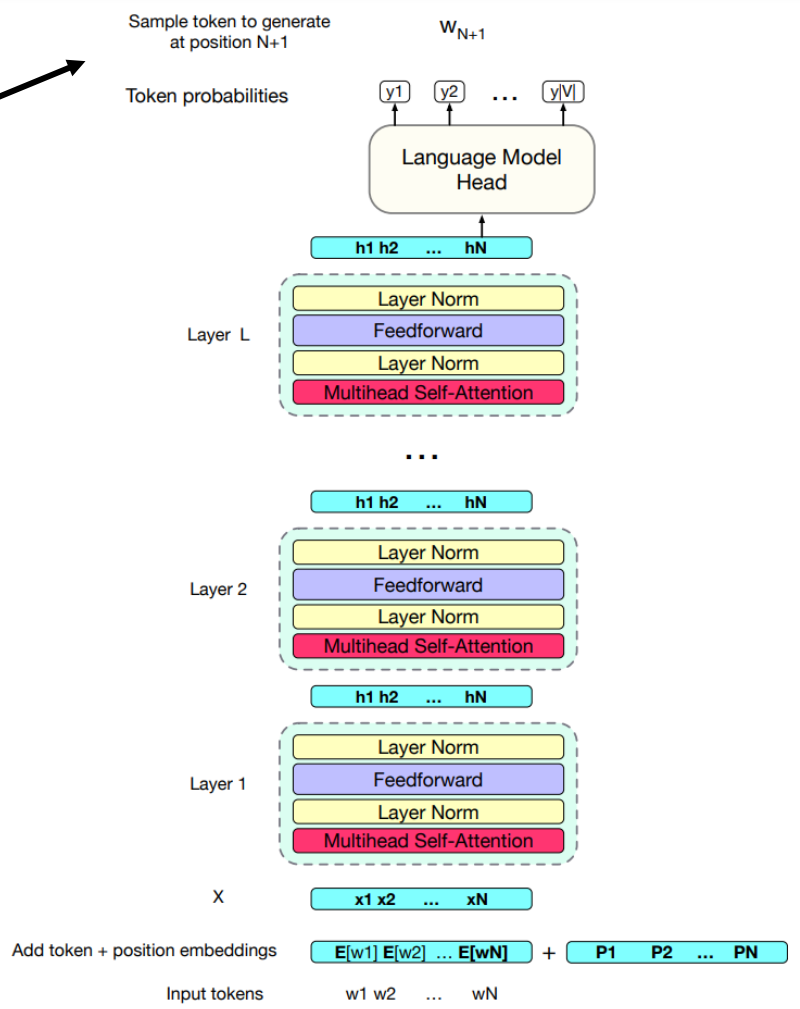


A transformer as a whole



A transformer as a whole

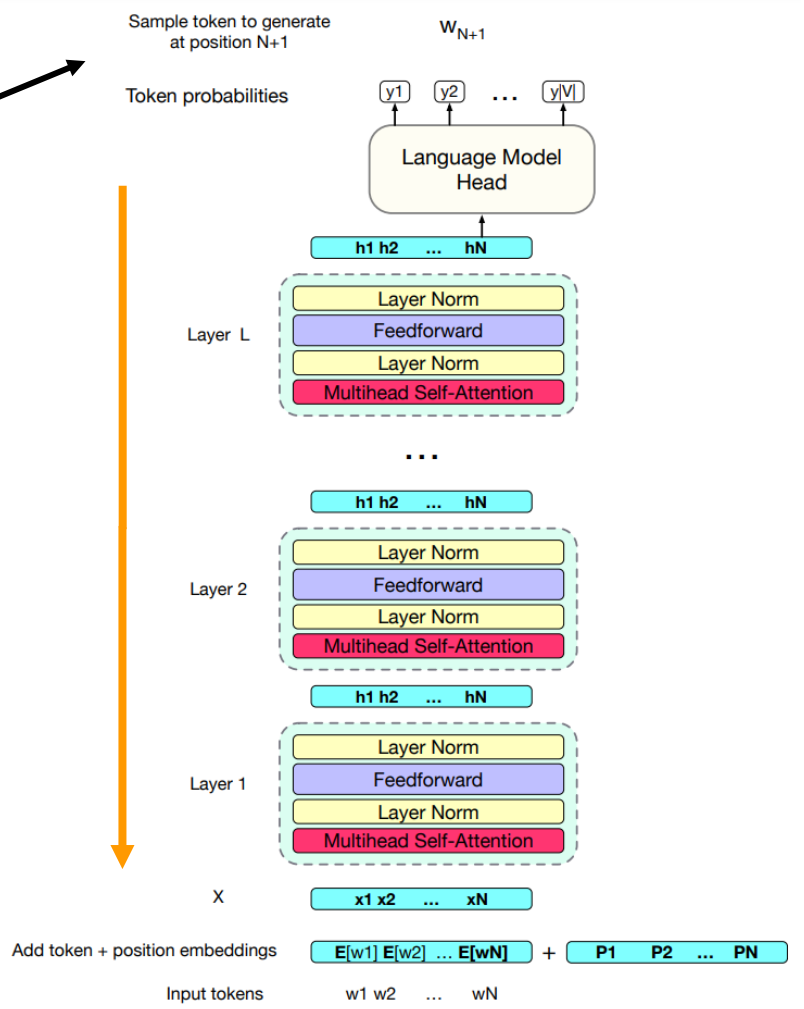
Cross-entropy loss for predicting the next word



A transformer as a whole

Cross-entropy loss for predicting the next word

Gradient descent, e.g. backpropagation



T **You**

Someone is famous for having found, when young, an efficient method for summing the natural numbers from 1 to 100. Who is that person, and what was their method?

You

Someone is famous for having found, when young, an efficient method for summing the natural numbers from 1 to 100. Who is that person, and what was their method?

ChatGPT

The person you're referring to is the mathematician Carl Friedrich Gauss. Legend has it that when Gauss was a child, his teacher tasked the class with adding up all the numbers from 1 to 100 as a way to keep them occupied. While his classmates were busy with the tedious task, Gauss found a clever method to solve the problem quickly.

His method involved pairing the first and last numbers in the sequence, then the second and second-to-last numbers, and so on. Each pair added up to 101 (1 + 100, 2 + 99, 3 + 98, and so forth), and since there were 50 pairs in total, the sum could be calculated as $50 \times 101 = 5050$.

T **You**

Someone is famous for having found, when young, an efficient method for summing the natural numbers from 3 to 45. Who is that person, and what was their method?

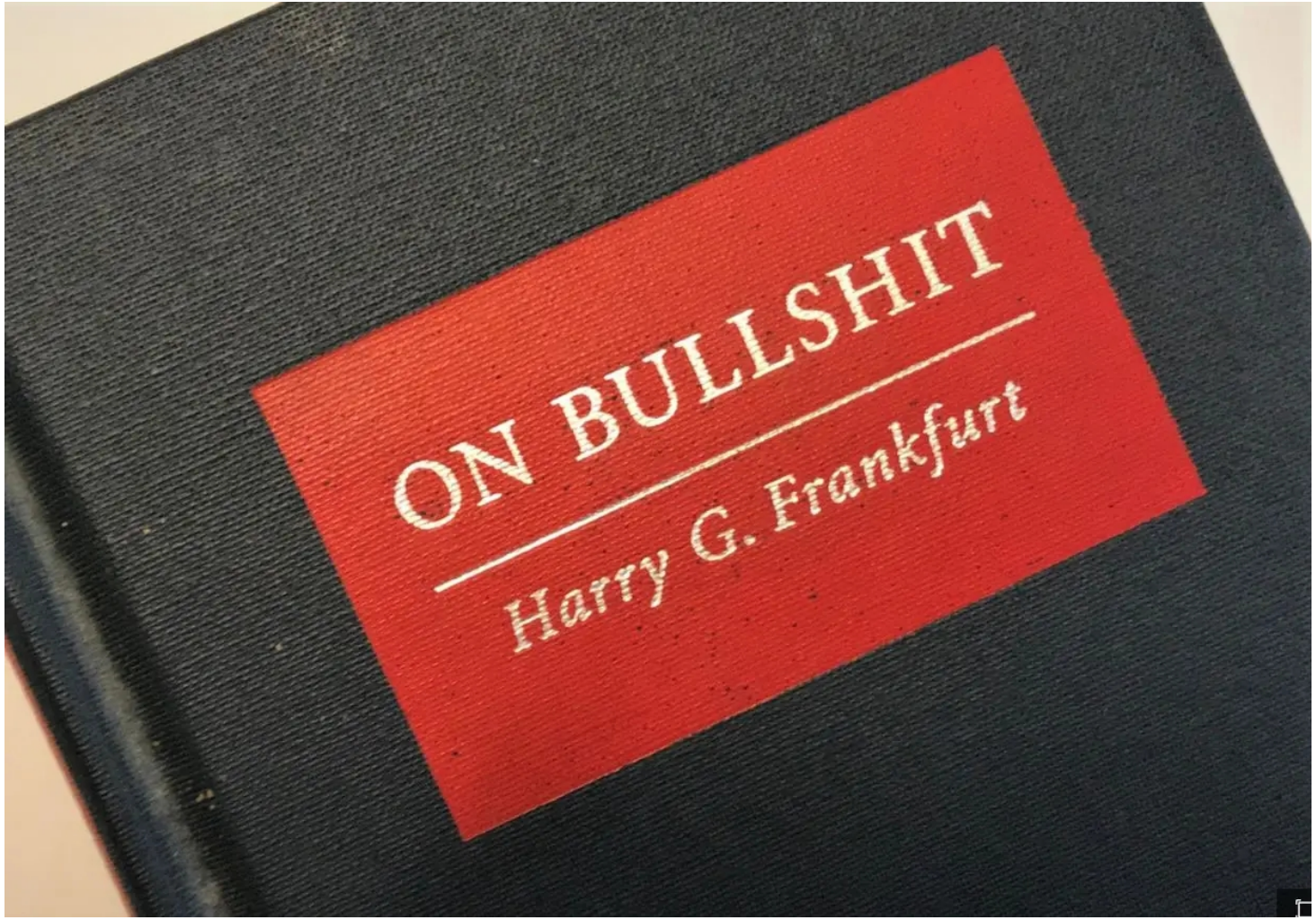
You

Someone is famous for having found, when young, an efficient method for summing the natural numbers from 3 to 45. Who is that person, and what was their method?

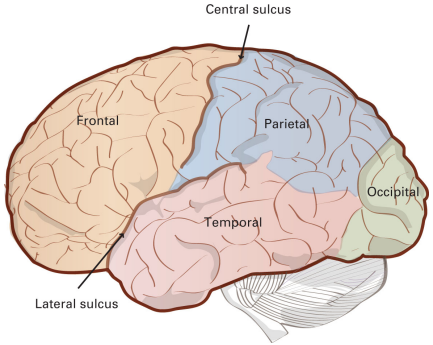
ChatGPT

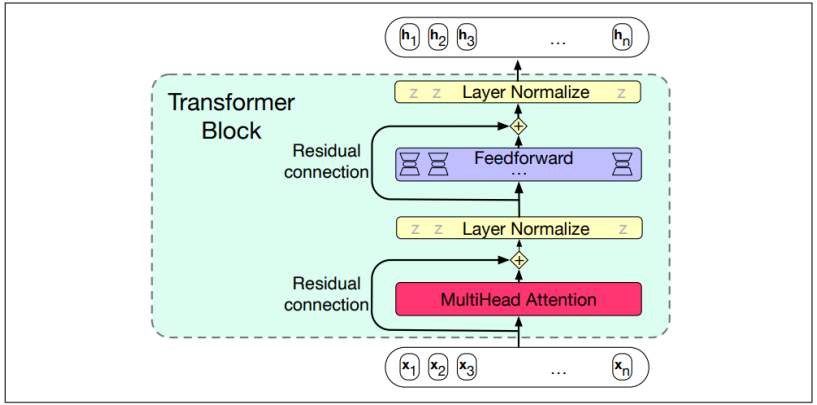
The person you are referring to is the mathematician Évariste Galois. He is known for his contributions to the development of group theory and for his work in abstract algebra, particularly his insights into the solvability of polynomial equations.

Galois did not specifically devise a method for summing the natural numbers from 3 to 45 when he was young. Instead, he made significant advancements in mathematics during his short life, which tragically ended at the age of 20. Galois' work laid the foundation for modern algebra and had a profound impact on various areas of mathematics.

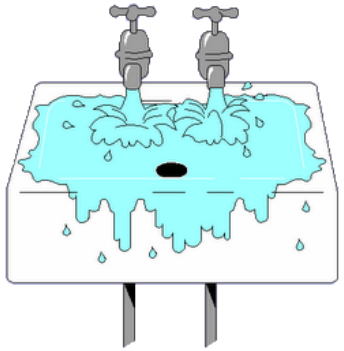

ON BULLSHIT
Harry G. Frankfurt

Questions? Comments?





$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$



Next: discussion

- [Optional] “Godfather of AI” Geoffrey Hinton: The 60 Minutes Interview [video].
- Welsh (2023). Large language models and the end of programming [video].
- McCoy et al. (2023). Embers of autoregression: Understanding large language models through the problem they are trained to solve.
- Futrell & Mahowald (2025). How linguistics learned to stop worrying and love the language models.